MANUAL DE CÓDIGO FUENTE

Agente negociador inteligente para la gestión y análisis de leads en LaPreDigital

Versión 1.0
Noviembre de 2026

ÍNDICE GENERAL

1 Presentación del sistema 4
2 Acceso e Inicio 7
2.1 Abrir la consola de negociación 7
2.2 Usar el menú y reconocer los modos de operación 9
3 Captación pública 11
3.1 Enviar el formulario de contacto 11
3.2 Continuar desde el comprobante de registro 14
3.3 Solicitar información de un plan 16
3.4 Aceptar la invitación del responsable económico 17
3.5 Activar Explora UNT desde el enlace personal 19
4 Pendientes del operador 20
4.1 Consultar las gestiones pendientes 20
4.2 Registrar la atención de una conversación derivada 22
4.3 Validar una contratación con pago registrado 23
4.4 Revisar la entrega de credenciales 24
5 LEADS y casos 25
5.1 Revisar una solicitud de captación 25
5.2 Registrar un contacto y su participación 27
5.3 Buscar y abrir una negociación 29
6 Expediente de negociación 30
6.1 Consultar y completar el perfil 30
6.2 Consultar la conversación y registrar un mensaje recibido 32
6.3 Revisar un mensaje con la inteligencia artificial 34
6.4 Revisar interés y prioridad de seguimiento 35
6.5 Registrar autorización o revocación de contacto 37
6.6 Preparar la prueba gratuita desde el expediente 39
6.7 Preparar una invitación al responsable económico 40
6.8 Registrar una aceptación documentada del responsable 42
7 Propuestas y servicio 44
7.1 Preparar una propuesta con tarifa vigente 44
7.2 Consultar el avance y compartir la oferta personal 46
7.3 Aceptar la propuesta y sus condiciones en el portal 48

7.4 Abrir el pago de prueba desde la oferta 50
7.5 Habilitar un servicio adquirido 52
7.6 Consultar el servicio habilitado 54
8 Catálogo interno 55
8.1 Consultar planes y tarifas mensuales 55
8.2 Comparar la modalidad semestral 57
9 Seguimiento autorizado 59
9.1 Consultar la agenda y revisar vencimientos 59
10 Indicadores de la fase LEADS 60
10.1 Consultar el resumen de indicadores 60
10.2 Aplicar filtros por periodo y dimensión 67
10.3 Interpretar causas y desgloses 69
10.4 Actualizar los datos y exportar el reporte 71
11 Agente e integraciones 75
11.1 Consultar el estado de los servicios 75
11.2 Vincular Telegram con un contacto existente 77
11.3 Configurar las recomendaciones con datos 78
12 Conocimiento del agente 80
12.1 Consultar y editar un documento 80
12.2 Crear o subir un documento 82
12.3 Probar una consulta 84
13 Reglas del agente 86
13.1 Configurar las preguntas y las acciones automáticas 86
14 Referencia de operación 88
14.1 Resolver situaciones frecuentes 88
14.2 Interpretar los indicadores 89

1 Presentación del sistema

Este manual explica la implementación de las mismas opciones del manual de usuario. Está dirigido a quien mantiene el sistema y necesita localizar sus componentes, validaciones, reglas y operaciones de datos. Cada opción conserva su pantalla y la relaciona con capturas reales del código y una explicación técnica breve.

Las pantallas se reutilizan del manual de usuario con sus datos de demostración. Las imágenes de código se obtuvieron de los archivos actuales abiertos en Visual Studio Code Server desde el navegador integrado, sin recrear un editor. Los nombres de funciones y las líneas corresponden a esa revisión; una modificación posterior puede desplazar su ubicación.

Arquitectura y responsabilidades

La interfaz React y TypeScript se compila con Vite. El estado de trabajo y los contratos HTTP conectan los componentes con una API Express en Node.js. Las rutas validan entradas con Zod; los servicios coordinan reglas y transacciones; los repositorios ejecutan SQL parametrizado. El adaptador Node inicia un trabajador PowerShell y utiliza System.Data.SqlClient para SQL Server. La lógica analítica consulta una base dimensional distinta del OLTP.

Capa	Archivos y responsabilidad
Interfaz	src/LiveApp.tsx, src/pages y src/features componen las pantallas y sus acciones.
Estado y contrato	src/state/Workspace.tsx y src/api.ts mantienen la selección y las peticiones HTTP.
API	server/app.mjs y routes.mjs de cada dominio validan y publican las operaciones.
Negocio	service.mjs coordina reglas, autorizaciones, evidencias y transacciones.
Persistencia	repository.mjs, server/db.mjs y server/sql-worker.ps1 ejecutan las consultas.
Integraciones	server/integrations y server/agent separan recepción, análisis, despacho y confirmación.
Análisis	server/datamart, server/kpis y sql separan ETL, hechos dimensionales y tasas.

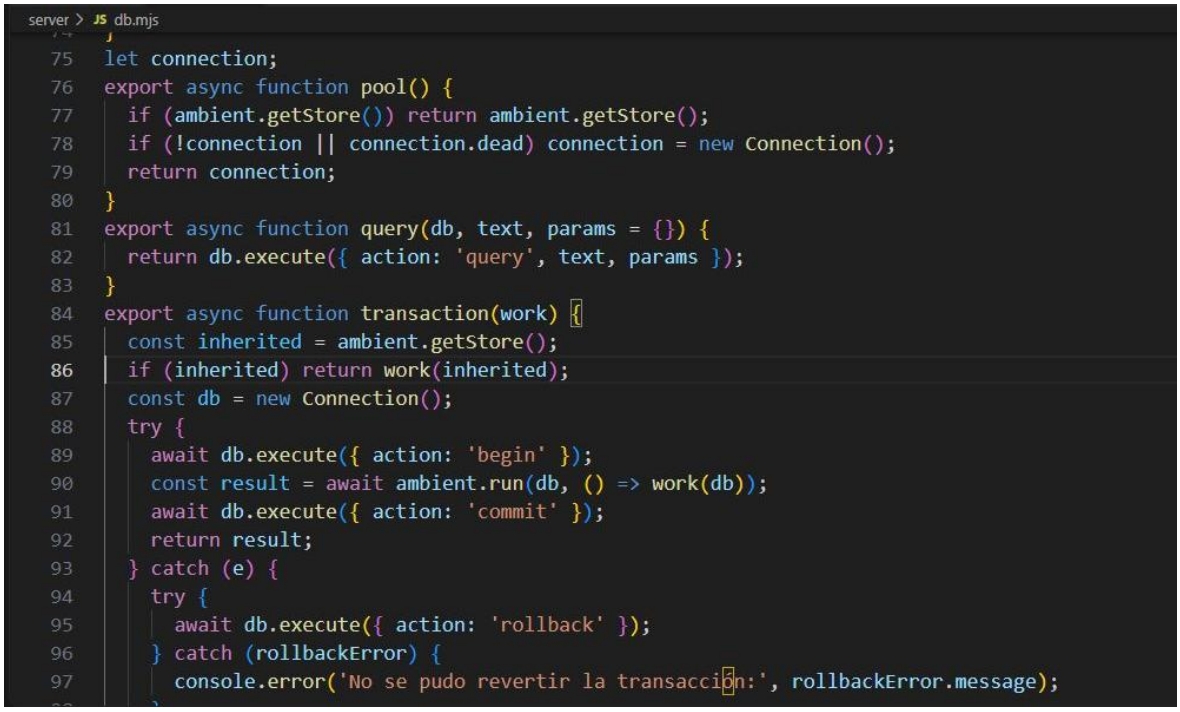
Requisitos y configuración

El README requiere Windows, Node.js 20 o superior, PowerShell con .NET, sqlcmd y SQL Server. package.json define npm install, npm run build y npm start para preparar y ejecutar la aplicación. npm run server y npm run dev separan API y Vite durante el desarrollo. npm run migrate aplica las migraciones del OLTP, incluido el acceso del administrador, las reglas del agente y los documentos de Conocimiento; npm run knowledge:init carga los documentos iniciales. npm run up inicia la API junto con el túnel público usado por los webhooks. El esquema OLTP reside en script/LaPreDigital_OLTP.sql; la creación y actualización del Data Mart se coordinan con las herramientas de tools/datamart.mjs.

La configuración de ejemplo está en .env.example. DB_SERVER y DB_DATABASE identifican el OLTP; DATAMART_DATABASE selecciona la base analítica. OPENAI_API_KEY y OPENAI_MODEL configuran el modelo principal; OLLAMA_ENABLED, OLLAMA_URL y OLLAMA_MODEL, el respaldo local. Los nombres de configuración de SMTP, Telegram, WhatsApp y Stripe se consultan en sus adaptadores. Este manual no incluye valores de claves, contraseñas ni tokens operativos. No se requiere exponer el editor o las bases mediante un túnel para consultar el código.

Figura 01

server/db.mjs líneas 75 a 97



pool reutiliza una conexión cuando corresponde y query delega la ejecución parametrizada. transaction conserva una conexión ambiental con AsyncLocalStorage para servicios anidados y coordina begin, commit, rollback y cierre.

Figura 02

server/app.mjs líneas 110 a 132



La API monta las rutas por dominio después de los controles de origen, host y tipo de contenido. /api/auth queda fuera de la sesión; requireSession protege el resto de /api y Conocimiento admite solicitudes mayores para subir archivos. Los repositorios ejecutan SQL mediante server/sql-worker.ps1 y System.Data.SqlClient, con parámetros y una conexión transaccional.

2 Acceso e Inicio

2.1 Abrir la consola de negociación

Figura 03

Inicio de sesión de la consola

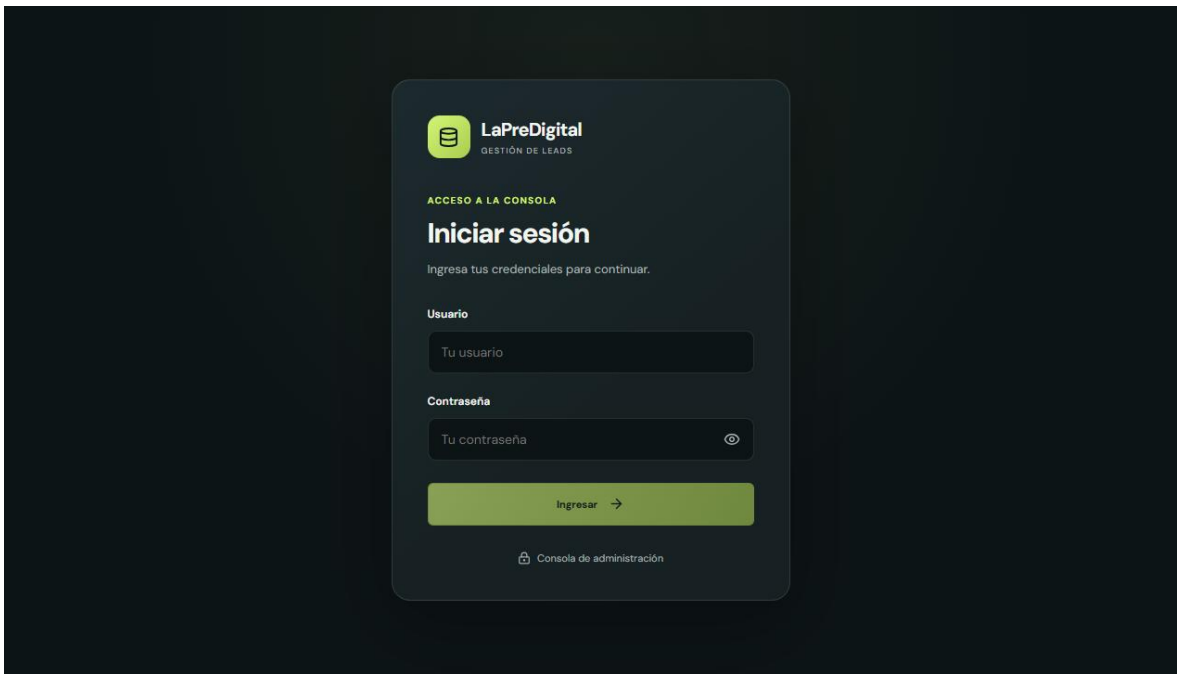


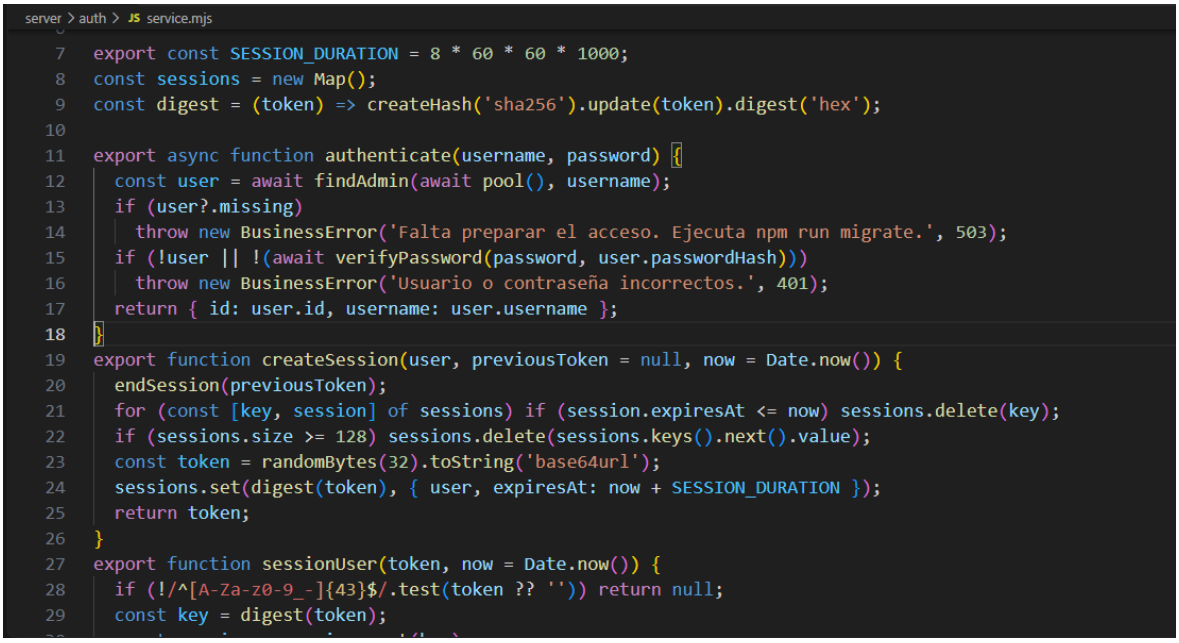
Figura 04

Pantalla de Inicio de la consola



Figura 05

server/auth/service.mjs líneas 7 a 29



Recorrido técnico Login → POST /api/auth/login → authenticate → createSession → cookie de sesión → requireSession

Implementación authenticate busca al administrador y compara la contraseña con su hash; si falta preparar el acceso indica ejecutar npm run migrate. createSession genera un token aleatorio de 32 bytes, guarda solo su hash SHA 256 en memoria con una vigencia de ocho horas y limita el número de sesiones simultáneas. La ruta de inicio admite diez intentos por minuto. app.mjs aplica requireSession a todas las rutas /api posteriores, de modo que la consola solo carga GET /api/bootstrap después de iniciar sesión. Workspace mantiene luego los datos iniciales, la selección y los estados de carga y error.

2.2 Usar el menú y reconocer los modos de operación

Figura 06

Menú de módulos de la consola

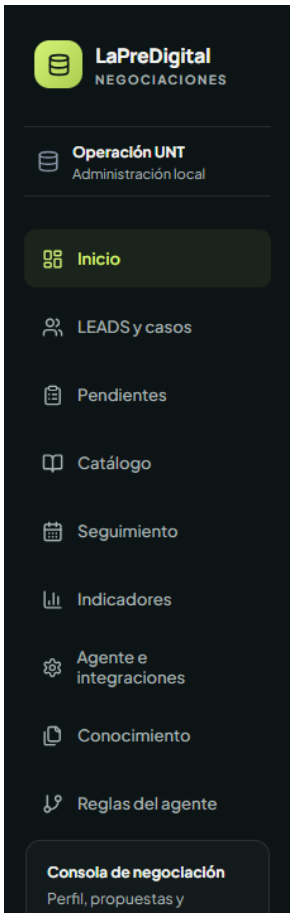
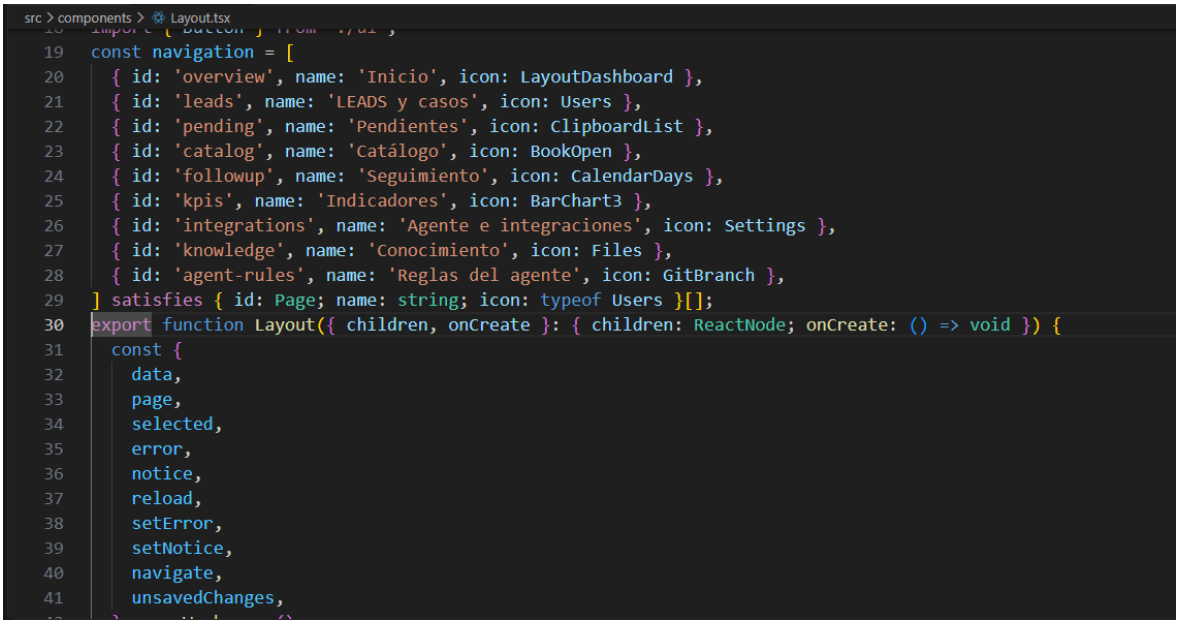


Figura 07

src/components/Layout.tsx líneas 19 a 41



Recorrido técnico Layout ⟶ navigation.map ⟶ navigate ⟶ WorkspaceView

Implementación La lista navigation define los identificadores, etiquetas e iconos de los nueve módulos del menú: Inicio, LEADS y casos, Pendientes, Catálogo, Seguimiento, Indicadores, Agente e integraciones, Conocimiento y Reglas del agente. Layout utiliza el estado page para marcar aria-current y aplica navigate al seleccionar una opción; si hay cambios sin guardar, pide confirmación antes de salir. LiveApp compone la vista correspondiente y Profile organiza las pestañas del expediente.

3 Captación pública

3.1 Enviar el formulario de contacto

Figura 08

Formulario público de contacto



Figura 09

Recomendación para el estudiante sin celular

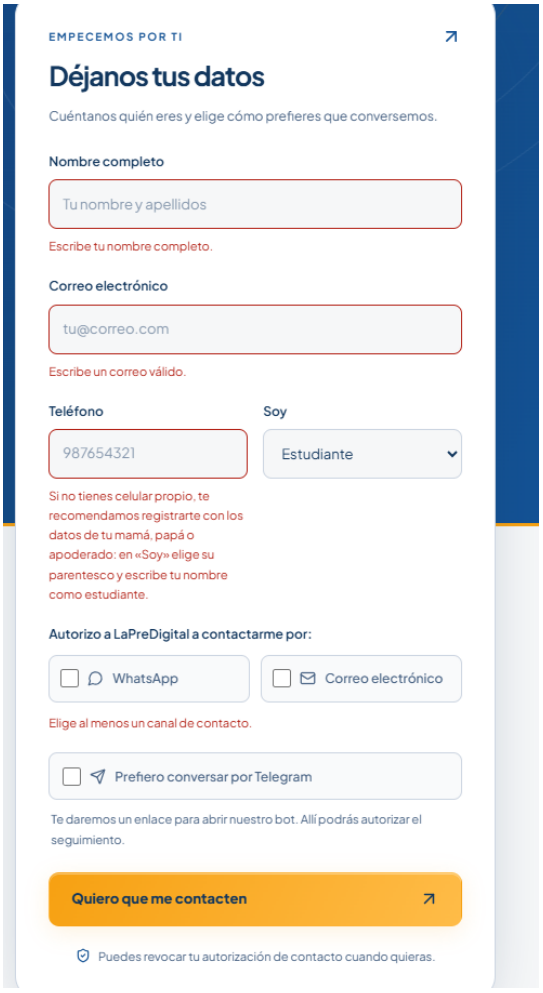
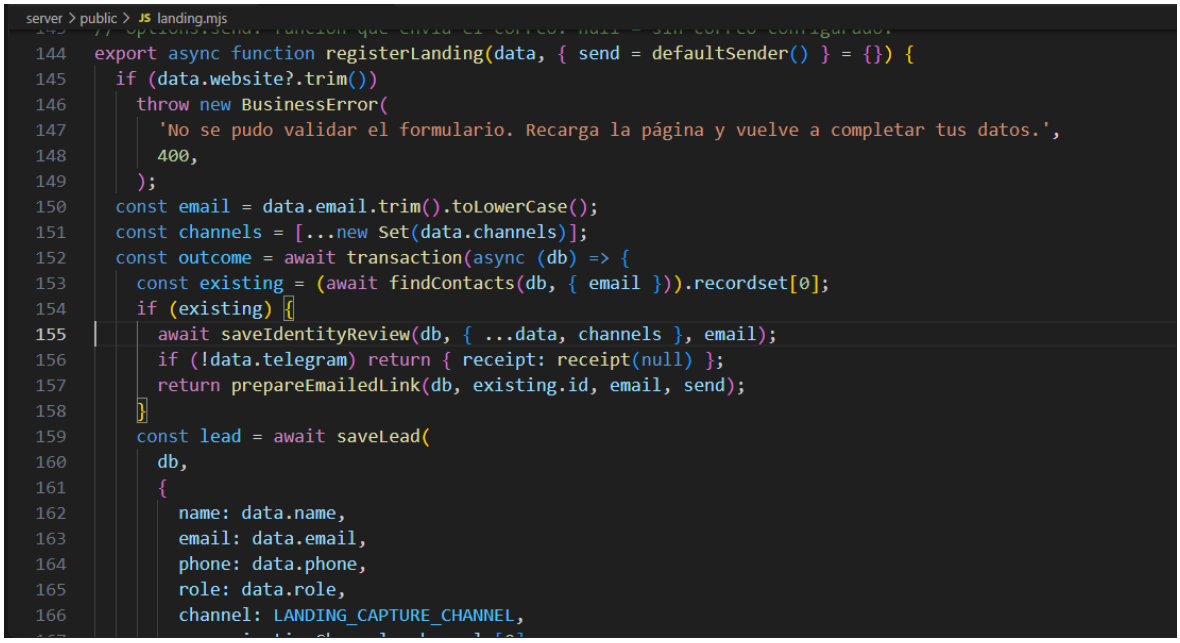


Figura 10

server/public/landing.mjs líneas 144 a 166



Recorrido técnico LandingForm → POST /api/public/landing → registerLanding → leads/service.mjs

Implementación La ruta valida landingForm y limita a ocho solicitudes por minuto. registerLanding normaliza los datos, comprueba identidades existentes y conserva solicitudes que requieren revisión. Los canales autorizados se transmiten explícitamente al alta comercial; saveLead crea o reutiliza Contacto, Beneficiario, Lead, Caso y LeadCaso. La recepción del formulario, la vinculación comercial y el correo de confirmación tienen resultados separados. Un fallo del conector de correo no debe interpretarse como una entrega realizada.

3.2 Continuar desde el comprobante de registro

Figura 11

Comprobante de recepción y continuación por canales

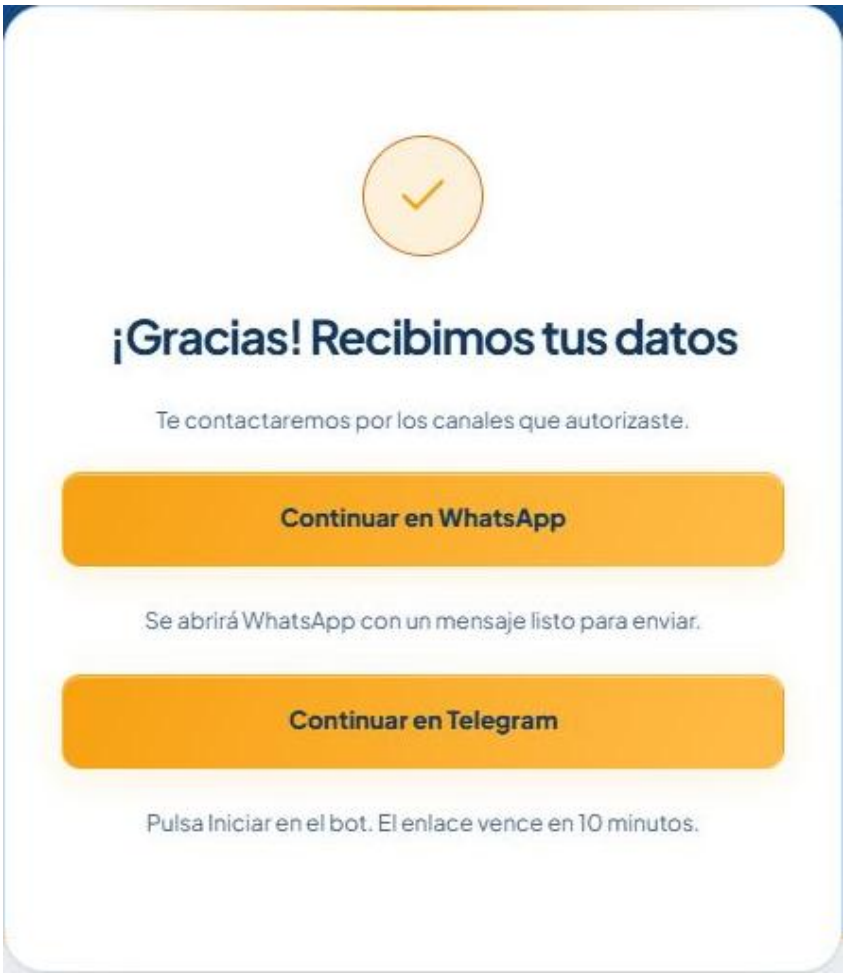
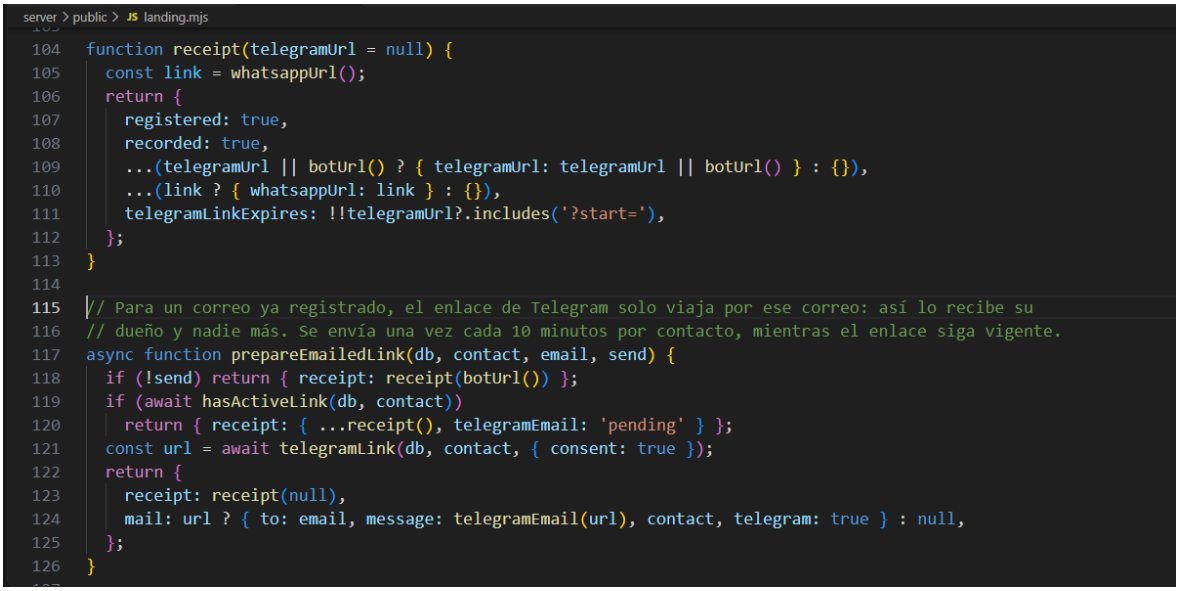


Figura 12

server/public/landing.mjs líneas 104 a 126



Recorrido técnico registerLanding → receipt → LandingForm y botones de continuación

Implementación receipt devuelve las opciones de continuación al formulario: el enlace wa.me configurado y el enlace al bot de Telegram. Si el correo ya pertenece a un contacto, prepareEmailedLink no muestra el enlace personal en la página: lo envía por correo a su dueño, como máximo una vez cada diez minutos mientras siga vigente. La vinculación Telegram usa un token aleatorio cuyo hash se guarda con su vigencia. La respuesta permite presentar los botones del comprobante, pero no acredita que el contacto haya iniciado el bot ni que se haya entregado un mensaje.

3.3 Solicitar información de un plan

Figura 13

Catálogo público y acceso a la solicitud de un plan

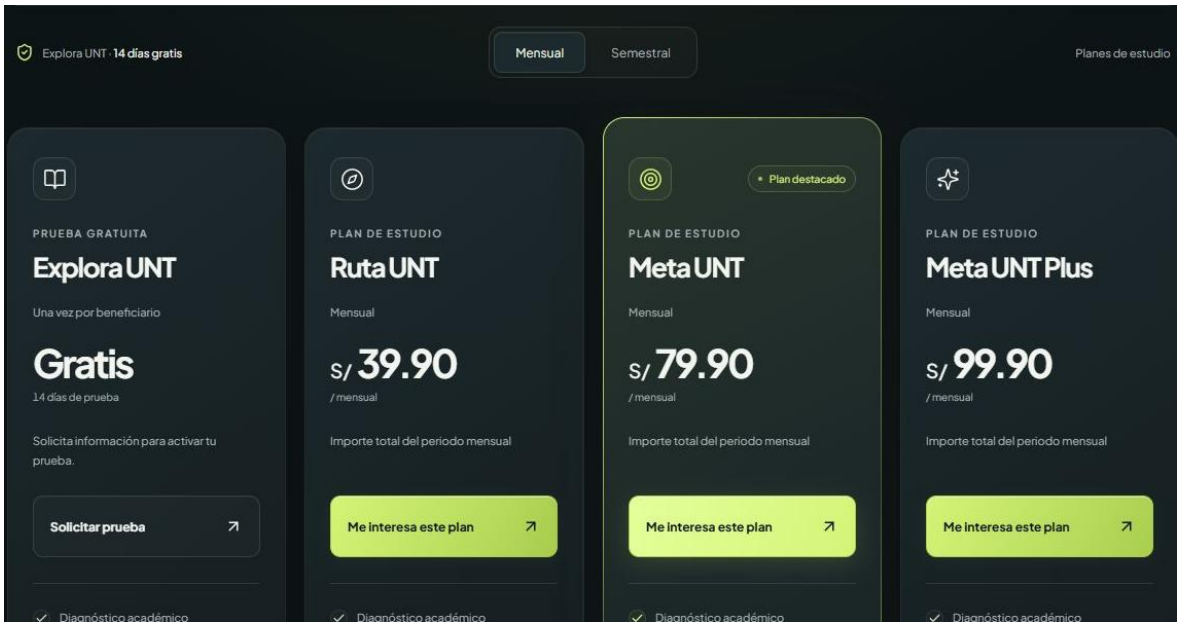
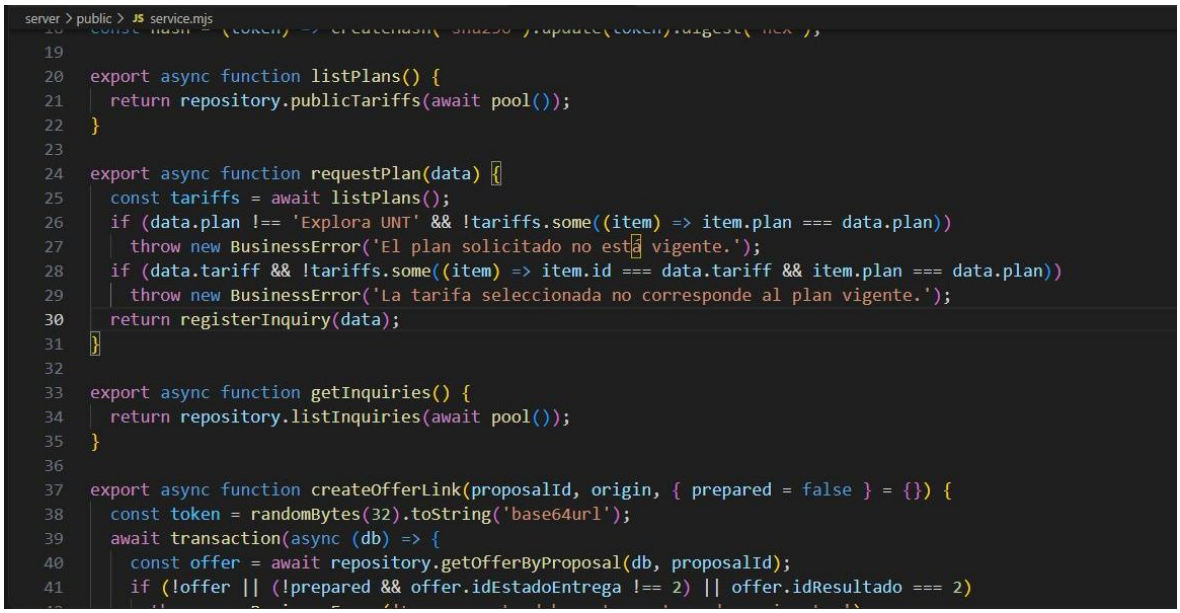


Figura 14

server/public/service.mjs líneas 19 a 41



Recorrido técnico PublicPlans → POST /api/public/interest → requestPlan → registerInquiry

Implementación requestPlan comprueba el plan y, si fue proporcionada, la tarifa contra publicTariffs. registerInquiry conserva la solicitud y resuelve estados Pendiente, Completar datos, Revisar identidad o Vinculada. La identidad existente no se asocia públicamente solo por escribir su correo. Cuando hay datos suficientes, el servicio llama a createLead y relaciona la solicitud con la participación. Esta operación registra interés, sin preparar un pago.

3.4 Aceptar la invitación del responsable económico

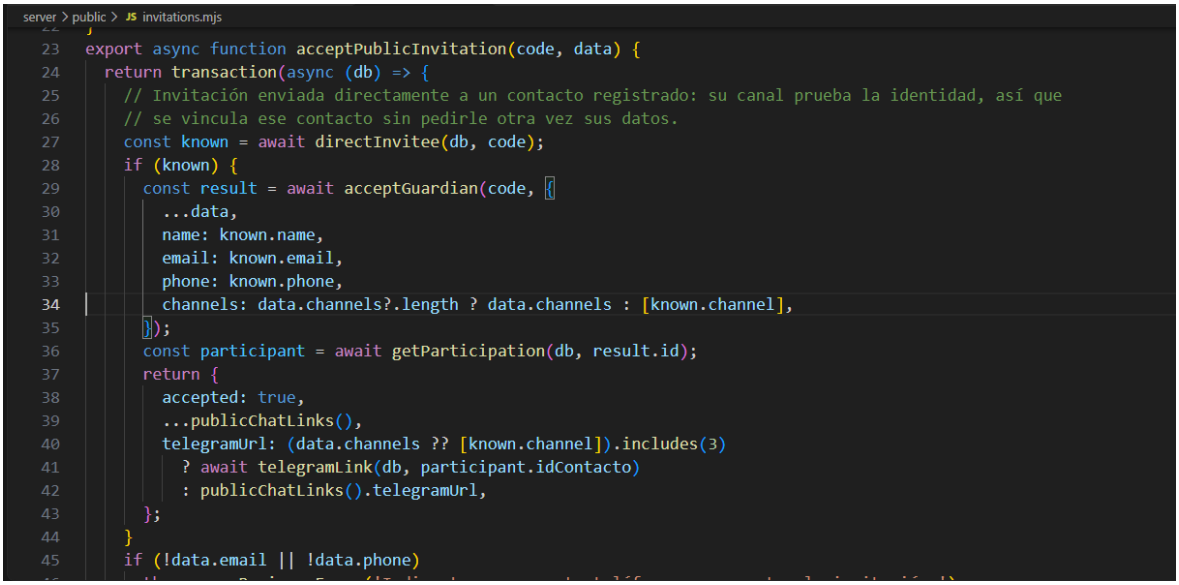
Figura 15

Aceptación pública de una invitación personal



Figura 16

server/public/invitations.mjs líneas 23 a 45



Recorrido técnico PublicGuardian → POST /api/public/invitations/:code → acceptPublicInvitation → acceptGuardian

Implementación Si la invitación se envió directamente a un contacto ya registrado, directInvitee lo identifica y acceptGuardian lo vincula con sus datos conocidos, sin volver a pedirlos; los canales elegidos se usan para continuar la conversación y, si incluye Telegram, se prepara su enlace personal. En otro caso se exigen correo y teléfono. El flujo público rechaza invitaciones vencidas o usadas, comprueba coincidencias de identidad y registra los dos consentimientos antes de vincular al responsable económico con el caso.

3.5 Activar Explora UNT desde el enlace personal

Figura 17

Pantalla de activación de Explora UNT

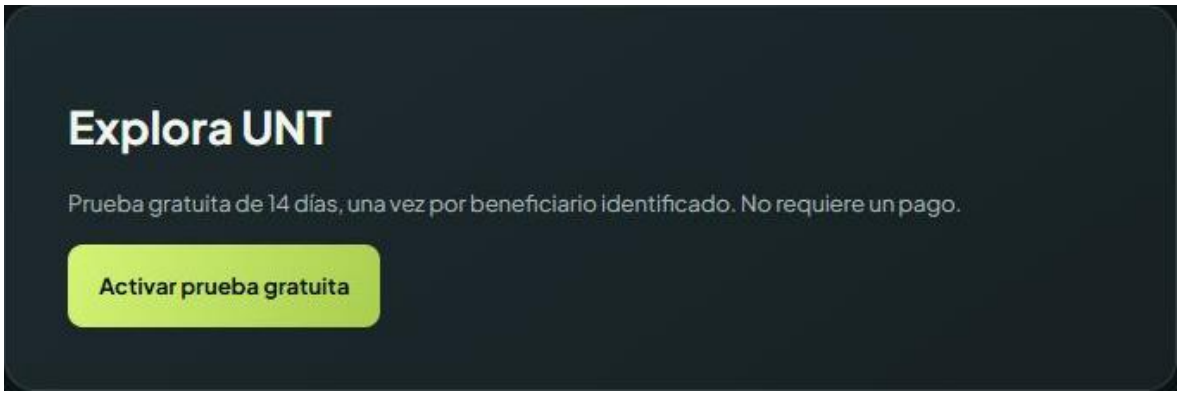
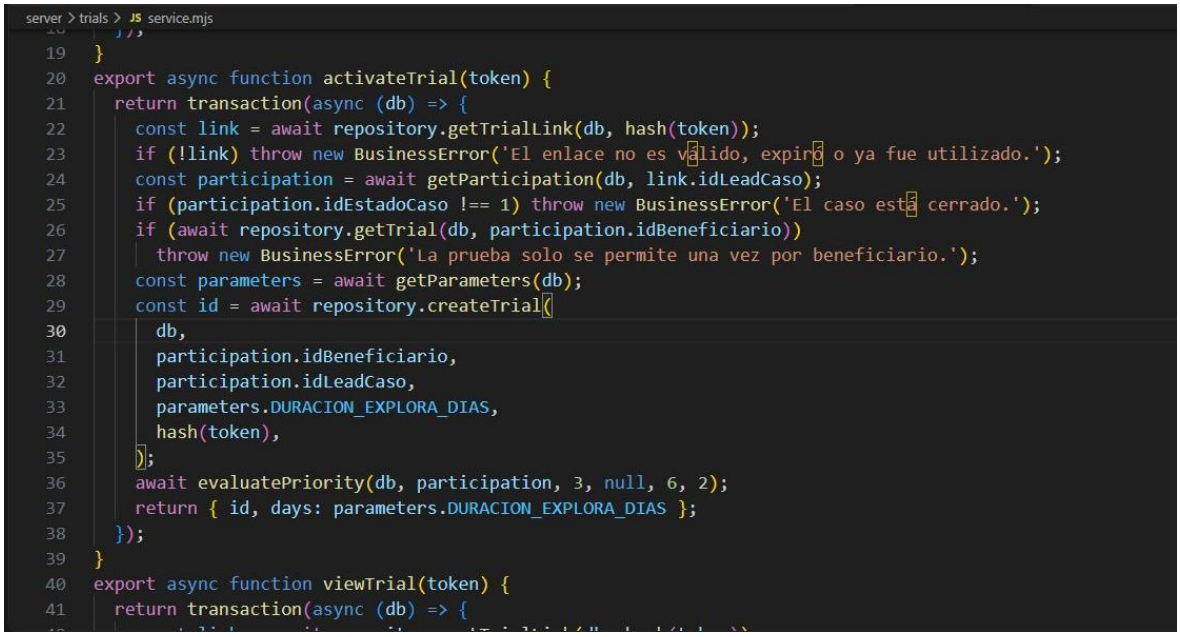


Figura 18

server/trials/service.mjs líneas 19 a 41



```
server > trials > JS service.mjs
19  }
20  export async function activateTrial(token) {
21    return transaction(async (db) => {
22      const link = await repository.getTrialLink(db, hash(token));
23      if (!link) throw new BusinessError('El enlace no es válido, expiró o ya fue utilizado.');
24      const participation = await getParticipation(db, link.idLeadCaso);
25      if (participation.idEstadoCaso !== 1) throw new BusinessError('El caso está cerrado.');
26      if (await repository.getTrial(db, participation.idBeneficiario))
27        throw new BusinessError('La prueba solo se permite una vez por beneficiario.');
28      const parameters = await getParameters(db);
29      const id = await repository.createTrial(
30        db,
31        participation.idBeneficiario,
32        participation.idLeadCaso,
33        parameters.DURACION_EXPLORA_DIAS,
34        hash(token),
35      );
36      await evaluatePriority(db, participation, 3, null, 6, 2);
37      return { id, days: parameters.DURACION_EXPLORA_DIAS };
38    });
39  }
40  export async function viewTrial(token) {
41    return transaction(async (db) => {
```

Recorrido técnico PublicTrial → POST /api/public/trials/:token → activateTrial → trials/repository.mjs

Implementación El token se compara por su hash y debe seguir vigente y sin uso. activateTrial exige un caso abierto y ausencia de una prueba anterior del beneficiario; la duración procede de DURACION_EXPLORA_DIAS. createTrial conserva la activación y el token consumido. La implementación también llama a evaluatePriority con nivel 3. Esto reevalúa la prioridad de la participación; no registra un pago ni una contratación y no equivale a disponer de una plataforma educativa integrada.

4 Pendientes del operador

4.1 Consultar las gestiones pendientes

Figura 19

Gestiones de atención del operador

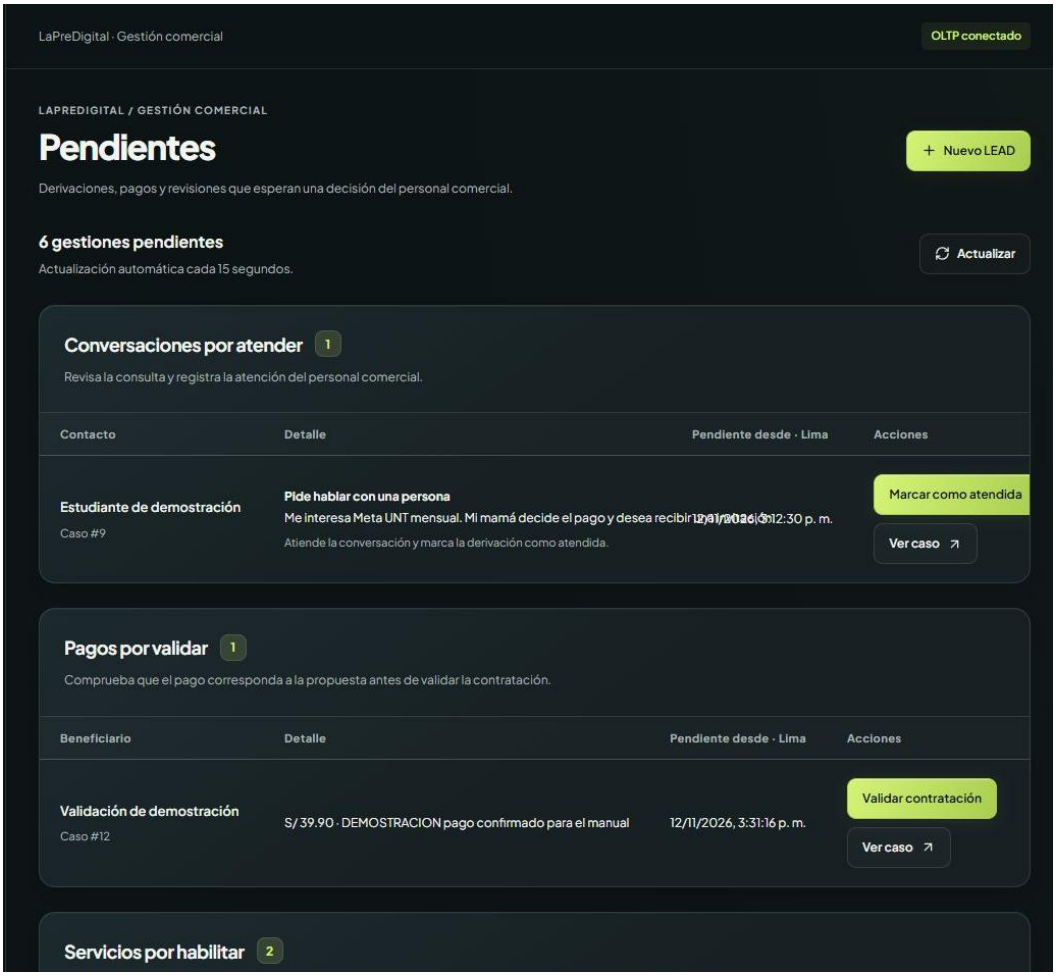


Figura 20

server/pending/service.mjs líneas 128 a 138



Recorrido técnico Pending → GET /api/pending → listPending → consultas de tareas pendientes

Implementación listPending reúne los grupos de atención comercial a partir de condiciones de sus registros. La vista Pending actualiza la consulta mientras está visible y muestra las acciones que corresponden a cada grupo. Conversaciones, pagos, habilitaciones, credenciales e identidades conservan identidades y estados diferentes. La acción completada deja de cumplir la condición de su bandeja. Abrir el expediente consulta evidencia y no resuelve por sí mismo una tarea.

4.2 Registrar la atención de una conversación derivada

Figura 21

Conversaciones derivadas al personal comercial

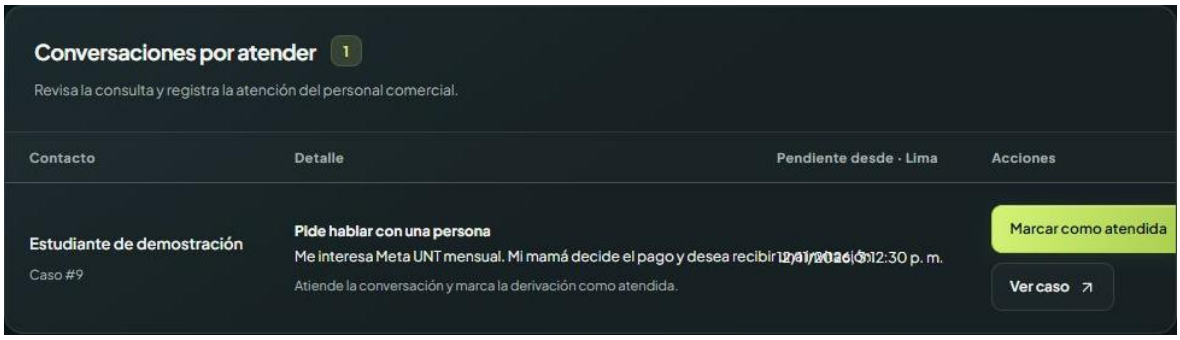


Figura 22

server/handoffs/service.mjs líneas 19 a 29



Recorrido técnico Pending → POST /api/participations/:id/handoff/attend → attendHandoff

Implementación El servicio obtiene la participación y busca una derivación abierta del contacto. Si no existe, devuelve BusinessError. En una transacción registra la atención con el identificador del operador y devuelve attended. El repositorio actualiza Derivacion; el resultado no constituye un envío al contacto. La creación y la retoma de derivaciones se implementan en openHandoff, el repositorio de handoffs y el mantenimiento de seguimiento.

4.3 Validar una contratación con pago registrado

Figura 23

Pago registrado pendiente de validación

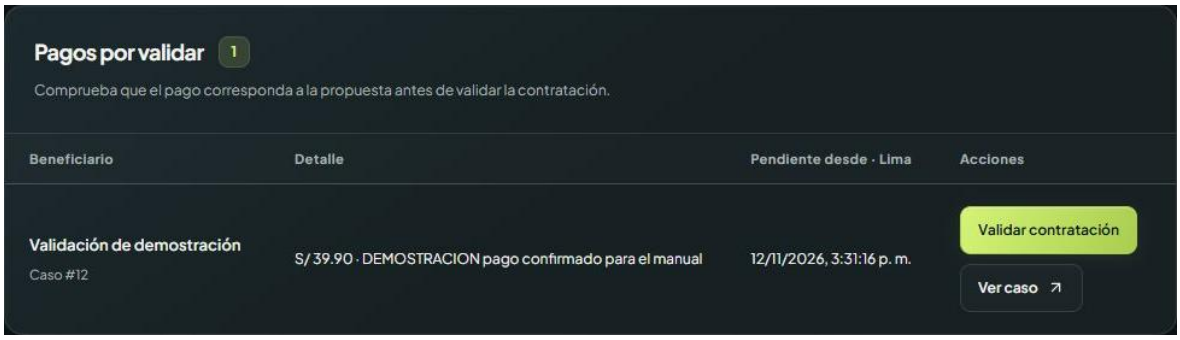
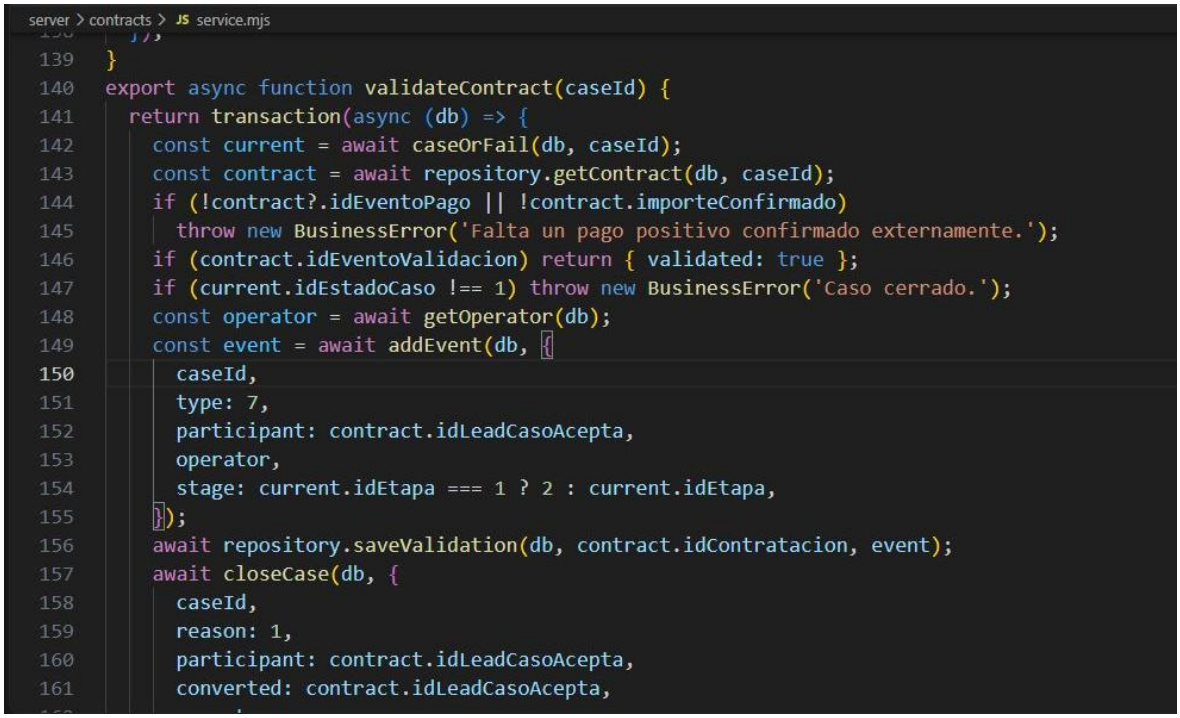


Figura 24

server/contracts/service.mjs líneas 139 a 161



Recorrido técnico Pending → contracts/routes.mjs → validateContract → contracts y cases/repository.mjs

Implementación validateContract exige un pago positivo registrado y un caso abierto; una validación existente se devuelve de forma idempotente. Registra el evento contractual con el operador, guarda la validación y llama a closeCase con motivo de contratación pagada. La participación aceptante se convierte y el caso se cierra dentro de la misma transacción. La habilitación y el mensaje de credenciales siguen siendo operaciones posteriores.

4.4 Revisar la entrega de credenciales

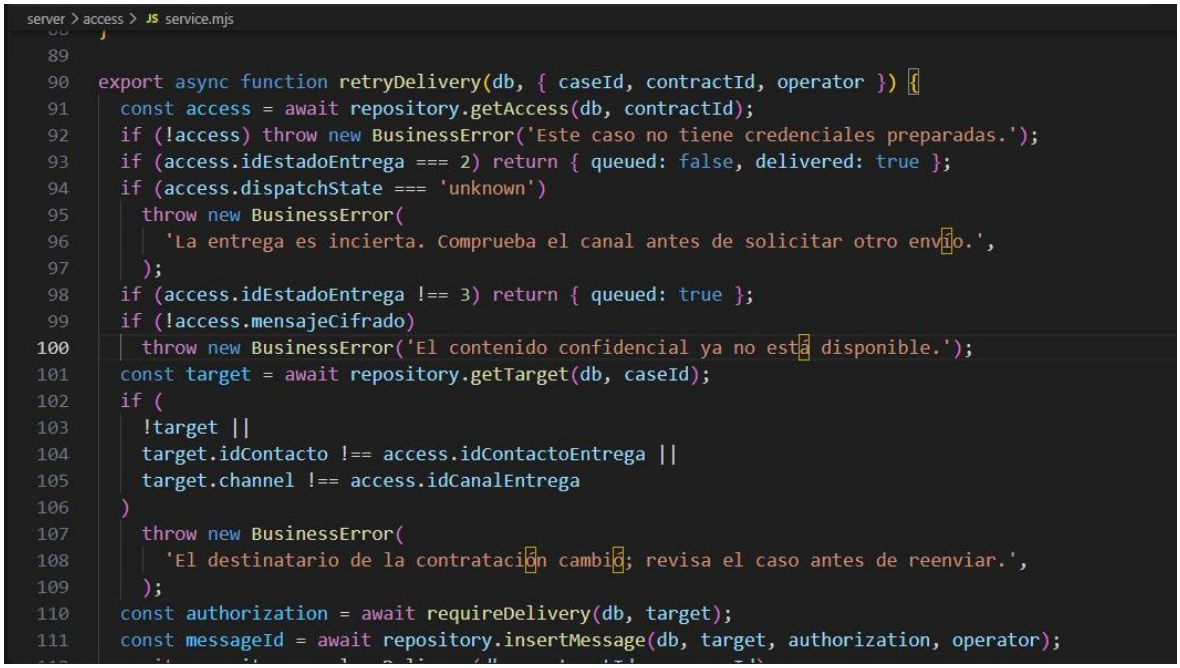
Figura 25

Estado y recuperación de la entrega de credenciales



Figura 26

server/access/service.mjs líneas 89 a 111



Recorrido técnico Pending → retryAccessDelivery → retryDelivery → replaceDelivery

Implementación El reintento comprueba que la contratación esté habilitada y no invalidada. retryDelivery reutiliza el acceso existente: una entrega confirmada no se repite; una incierta se bloquea para revisión; un mensaje que sigue en cola no genera otro. Solo el fallo confirmado con contenido cifrado disponible permite preparar una nueva entrega. Antes verifica destinatario, canal y autorización. No se crea otra cuenta ni se sustituye la contraseña existente.

5 LEADS y casos

5.1 Revisar una solicitud de captación

Figura 27

Solicitudes por revisar y negociaciones registradas

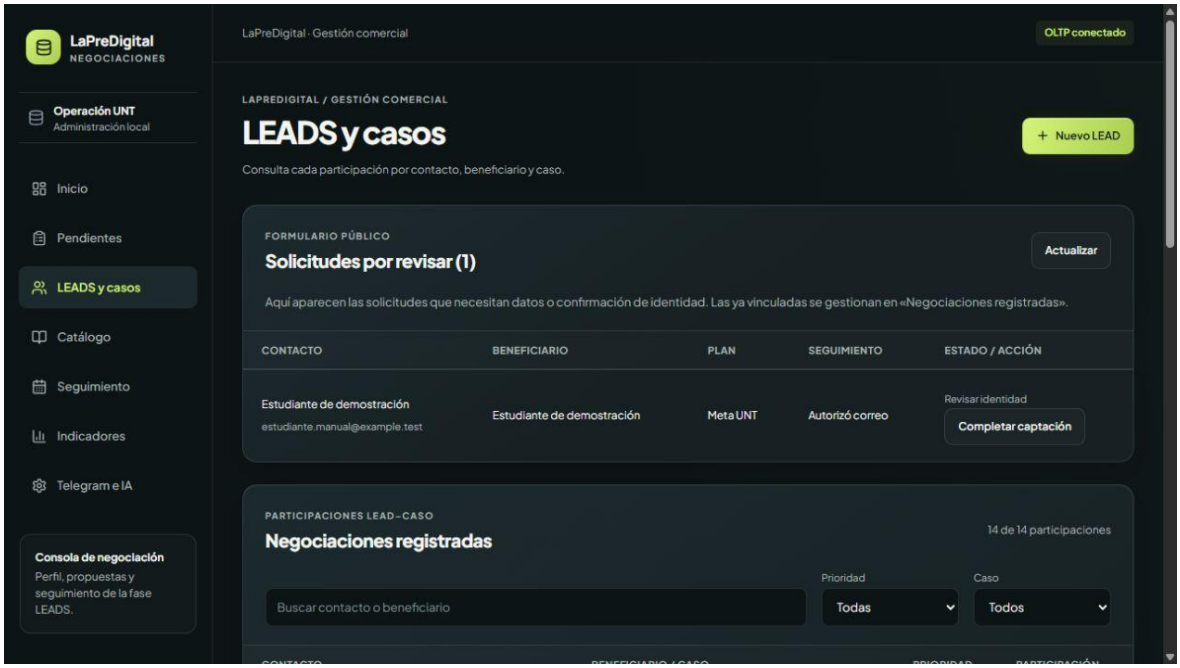


Figura 28

server/public/intake.mjs líneas 68 a 90



Recorrido técnico LeadTable y LeadForm → POST /api/public-inquiries/:id/complete → registerInquiry

Implementación El modo operator permite completar una solicitud identificada. Antes de vincular, el servicio exige resolver coincidencias de beneficiario mediante su identificador y reutiliza el contacto identificado. linkInquiry conserva la participación y el estado Vinculada. LeadTable filtra las solicitudes que ya tienen participationId y las que están Vinculada o Descartada; por eso salen de la revisión sin eliminar su historial. Las captaciones provisionales utilizan además completeOnboarding.

5.2 Registrar un contacto y su participación

Figura 29

Registro manual en dos vistas continuas del formulario

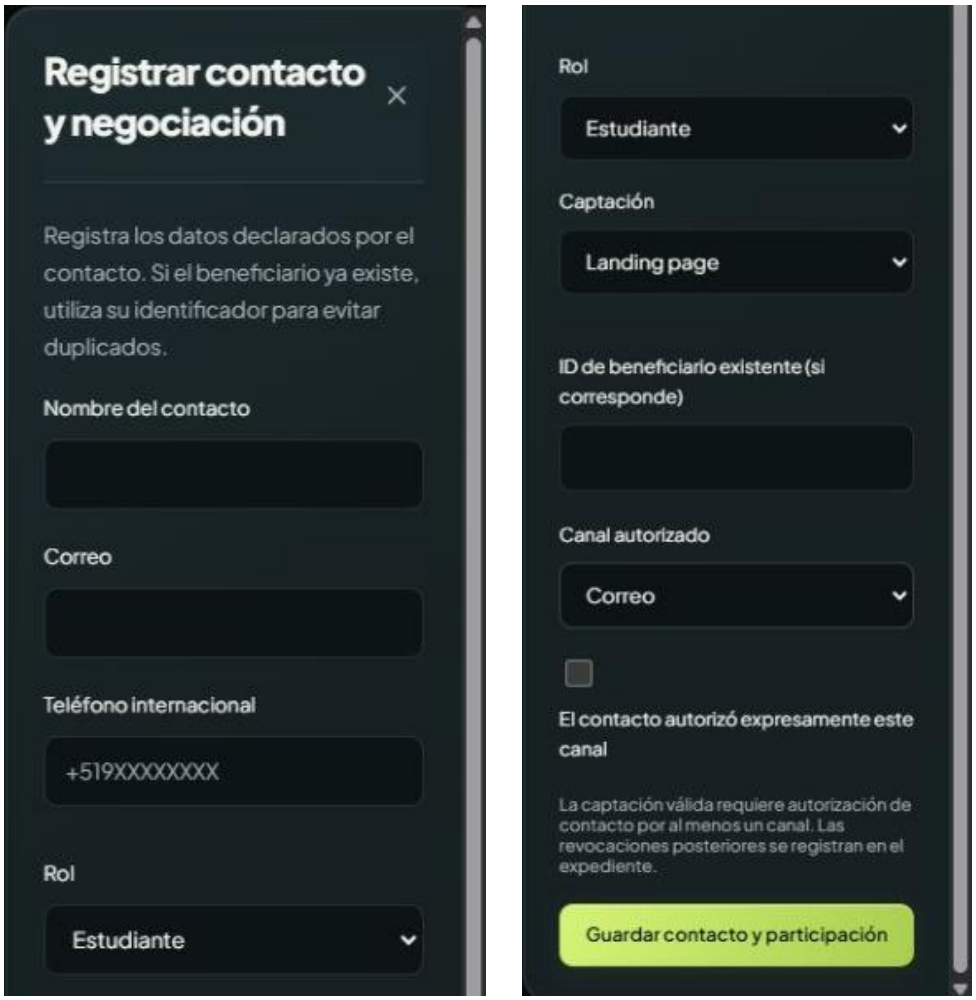
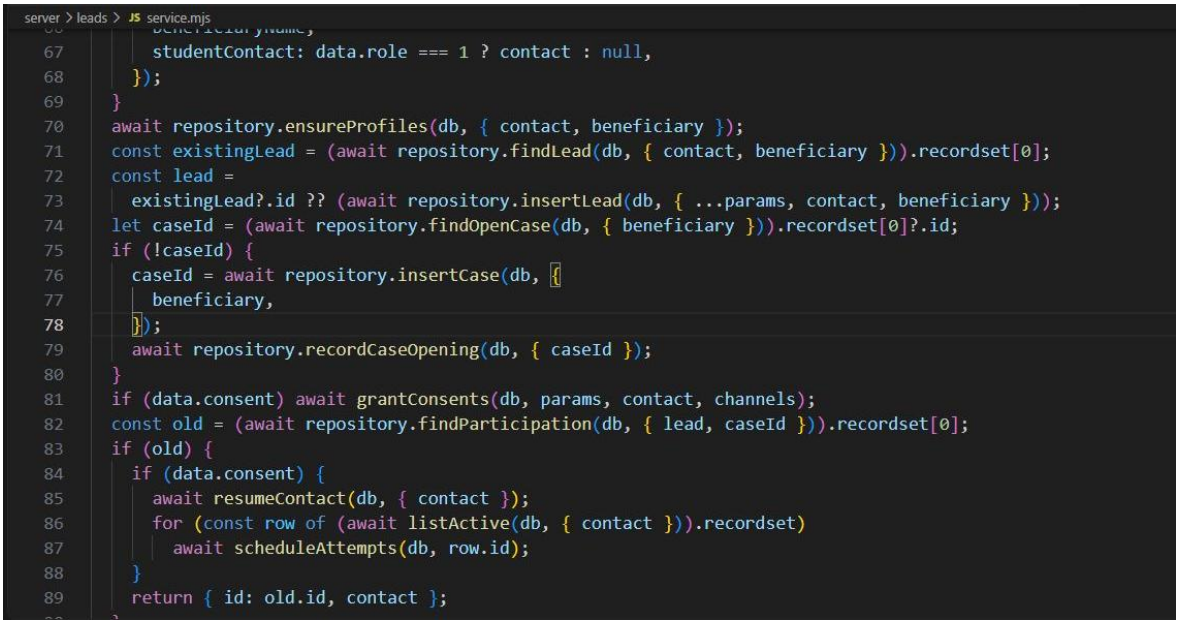


Figura 30

server/leads/service.mjs líneas 67 a 89



Recorrido técnico LeadForm → POST /api/leads → createLead → saveLead → leads/repository.mjs

Implementación La entrada se valida con Zod: nombre, datos de contacto, rol, origen, beneficiario y canal. saveLead normaliza correo y teléfono y comprueba que no correspondan a contactos diferentes. Reutiliza la relación contacto–beneficiario y el caso abierto, y evita duplicar una participación existente. Las autorizaciones válidas determinan estado y elegibilidad; scheduleAttempts conserva la agenda única. Todas las escrituras coordinadas heredan la misma transacción.

5.3 Buscar y abrir una negociación

Figura 31

Identificación del contacto beneficiario y caso en el expediente



Figura 32

src/features/leads/LeadTable.tsx líneas 5 a 27

src > features > leads > LeadTable.tsx

```
5  import type { PublicInquiry } from '../../api';
6  export function LeadTable() {
7    const { data, open, inquiries, inquiryError, refreshInquiries } = useWorkspace();
8    const [search, setSearch] = useState('');
9    const [priority, setPriority] = useState('');
10   const [caseState, setCaseState] = useState('');
11   const [intake, setIntake] = useState<PublicInquiry | null>(null);
12   const pendingInquiries = inquiries?.filter(
13     (item) => !item.participationId && item.status !== 'Vinculada' && item.status !== 'Descartada',
14   );
15   const rows =
16     data?.leads.filter(
17       (lead) =>
18         `${lead.name} ${lead.email || ''} ${lead.beneficiary} ${lead.idCaso}`
19           .toLowerCase()
20           .includes(search.toLowerCase()) &&
21         (!priority || lead.priority === priority) &&
22         (!caseState || lead.idEstadoCaso === Number(caseState)),
23     ) || [];
24   return (
25     <>
26       {intake && (
27         <LeadForm
```

Recorrido técnico LeadTable → open(id) → GET /api/participations/:id → getDetails

Implementación Los filtros locales aplican búsqueda de contacto, correo, beneficiario y número de caso, además de prioridad y estado del caso. La fila representa una participación. Workspace conserva selectedId y solicita el detalle por ese identificador; getDetails reúne perfil, mensajes, propuestas, autorizaciones, evaluaciones y contratos asociados. El contador de filas no debe interpretarse como personas únicas. El expediente permite seguir un contacto en el contexto del beneficiario y caso correctos.

6 Expediente de negociación

6.1 Consultar y completar el perfil

Figura 33

Perfil académico y económico del contacto



Figura 34

server/profiles/service.mjs líneas 75 a 97



Recorrido técnico ProfileCard → profiles/routes.mjs → saveProfile → profiles/repository.mjs

Implementación saveProfile distingue los campos del beneficiario de los del contacto mediante un mapa fijo de tablas y columnas. Cursos y postulaciones tienen tratamiento específico; los valores sin cambio no crean historial. Cada modificación conserva valor anterior, nuevo, canal, rol, tipo de cambio y operador o mensaje de procedencia. El servicio exige un caso abierto y ejecuta la actualización y su trazabilidad en una transacción. No completa campos desconocidos con información inferida.

6.2 Consultar la conversación y registrar un mensaje recibido

Figura 35

Historial y registro de interacciones

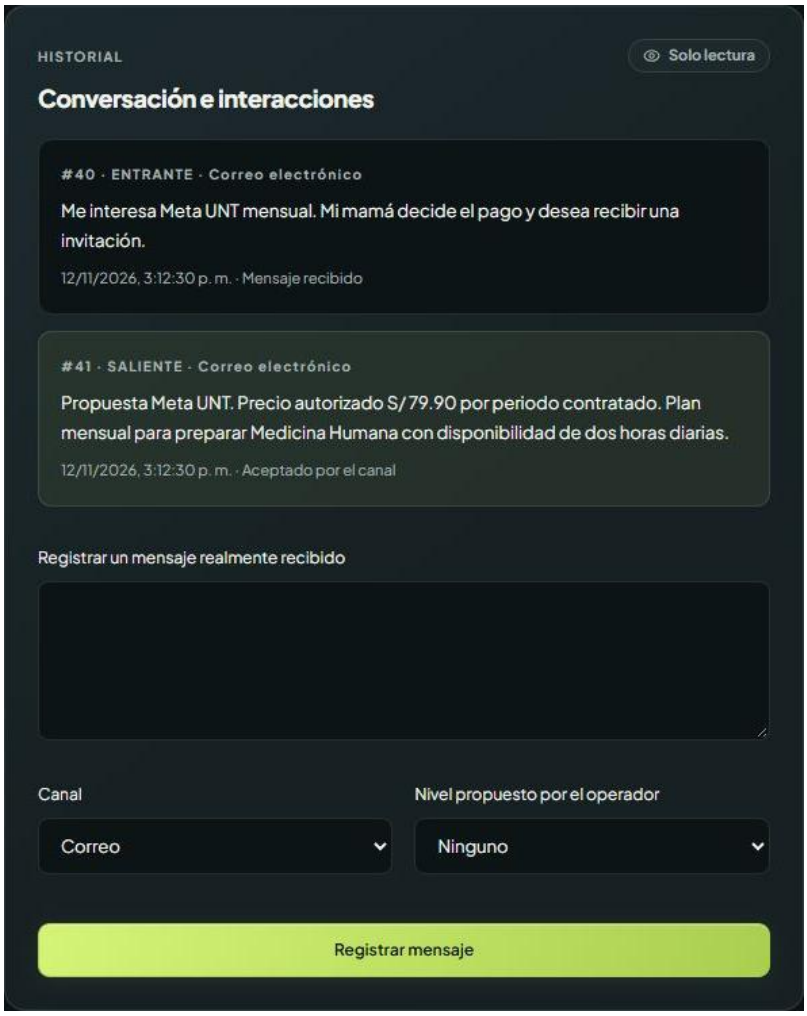
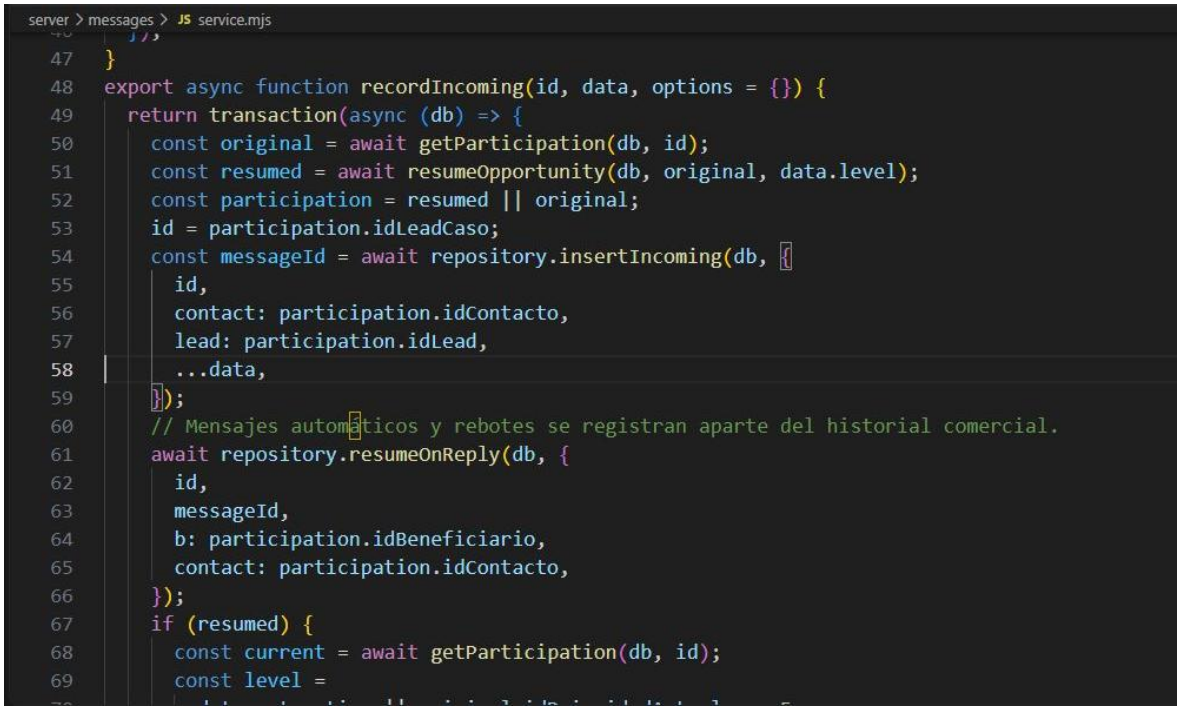


Figura 36

server/messages/service.mjs líneas 47 a 69



Recorrido técnico Conversation → messages/routes.mjs → recordIncoming → messages/repository.mjs

Implementación recordIncoming obtiene la participación, aplica las reglas de reactivación de resumeOpportunity e inserta el mensaje entrante. resumeOnReply relaciona la respuesta con el seguimiento y se reevalúa la prioridad cuando corresponde. La función devuelve messageId y participationId, porque una reapertura puede cambiar el contexto seleccionado. Los mensajes automáticos y rebotes no se consideran respuestas comerciales. Este formulario registra evidencia recibida; no llama a un conector para enviar el texto escrito.

6.3 Revisar un mensaje con la inteligencia artificial

Figura 37

Selección de un mensaje para revisión asistida



Figura 38

server/integrations/service.mjs líneas 37 a 50

```
server > integrations > JS service.mjs
37  }
38  export async function analyze(id, data) {
39    const context = await getWorkspace();
40    if (!context.leads.some((lead) => lead.id === id))
41      throw new BusinessError('Participación no encontrada', 404);
42    const participant = context.leads.find((lead) => lead.id === id);
43    return analyzeMessage(data.text, context.tariffs, {
44      context: {
45        contact: participant.name,
46        beneficiary: participant.beneficiary,
47        role: participant.role,
48      },
49    });
50  }
51
```

Recorrido técnico AgentCard → POST /api/participations/:id/agent/analyze → analyze → analyzeMessage

Implementación analyze verifica que la participación figure en el espacio de trabajo y entrega al modelo las tarifas y el contexto de contacto, beneficiario y rol. agent/ollama.mjs selecciona OpenAI y el respaldo Ollama según configuración y disponibilidad, valida la salida estructurada y las citas de respaldo. Este endpoint devuelve una propuesta de análisis. La aplicación de acciones automáticas se coordina separadamente en agent/service.mjs; los modelos no acreditan pagos ni aprueban descuentos.

6.4 Revisar interés y prioridad de seguimiento

Figura 39

Revisión de interés y prioridad vista superior e inferior

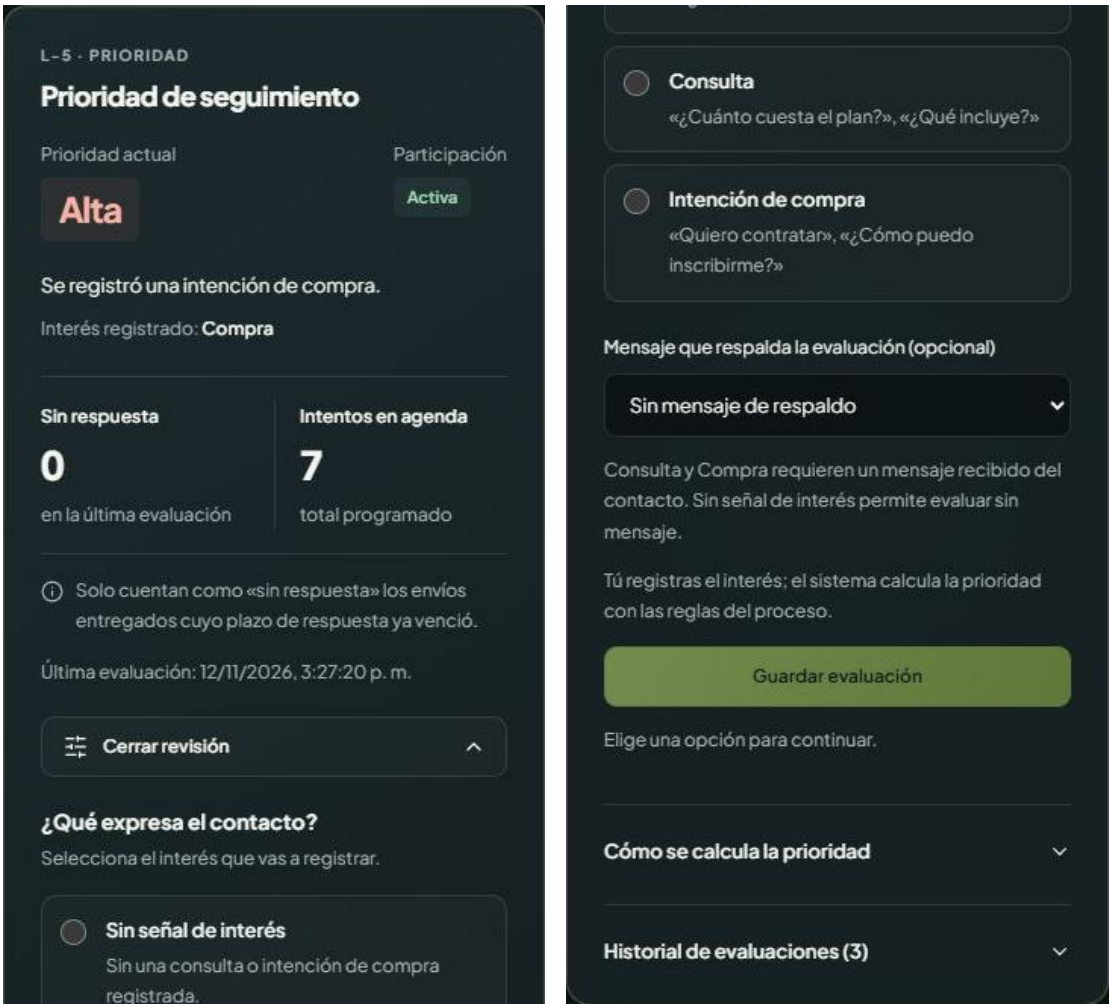


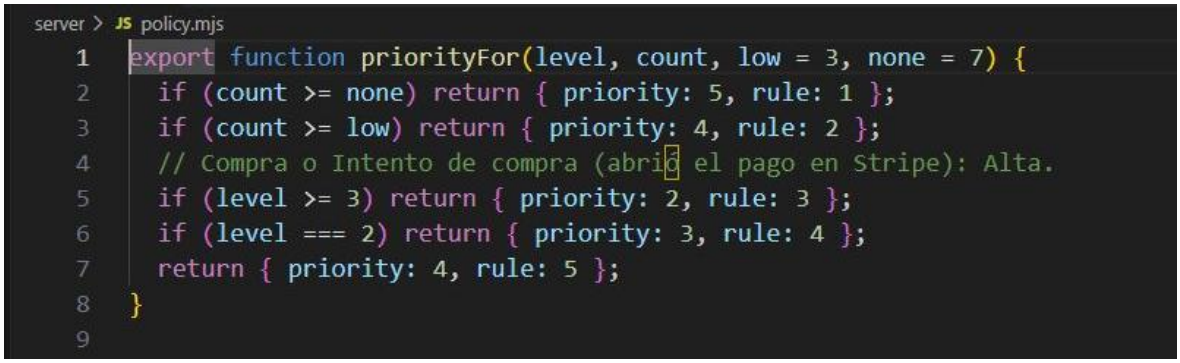
Figura 40

server/priorities/service.mjs líneas 40 a 53

```
server > priorities > JS service.mjs
40  }
41  export async function saveEvaluation(id, data) {
42    return transaction(async (db) => {
43      const participation = await getParticipation(db, id);
44      if (
45        data.messageId &&
46        !(await repository.findEvidence(db, { m: data.messageId, id })).recordset.length
47      )
48        throw new BusinessError('La evidencia no pertenece a esta participación.');
49      if (data.level > 1 && !data.messageId)
50        throw new BusinessError('Consulta o Compra requieren mensaje de evidencia.');
51      return evaluatePriority(db, participation, data.level, data.messageId ?? null, 7);
52    });
53  }
54
```

Figura 41

server/policy.mjs líneas 1 a 8



Recorrido técnico PriorityCard → priorities/routes.mjs → saveEvaluation → evaluatePriority → priorityFor

Implementación saveEvaluation comprueba que el mensaje pertenezca a la participación y exige evidencia cuando el nivel supera Ninguno. evaluatePriority consulta parámetros y cuenta intentos entregados cuyo plazo venció; después registra la evaluación con regla, procedencia y contador. priorityFor aplica primero los umbrales Nula y Baja; por debajo de ellos, nivel mayor o igual a 3 produce Alta, nivel 2 Media y el resto Baja. La versión actual incluye también el nivel 4 de intento de compra al abrir Stripe.

6.5 Registrar autorización o revocación de contacto

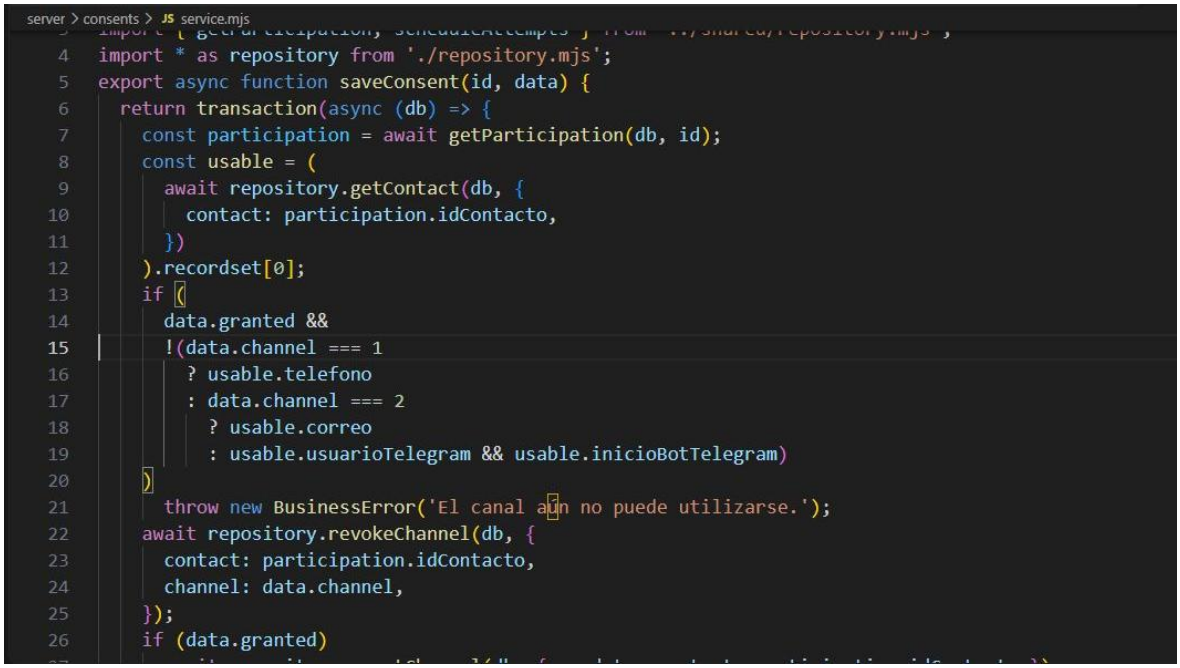
Figura 42

Permisos independientes por canal de contacto



Figura 43

server/consents/service.mjs líneas 4 a 26



Recorrido técnico ConsentCard → consents/routes.mjs → saveConsent → consents/repository.mjs

Implementación Antes de otorgar una autorización, saveConsent comprueba que el canal tenga datos utilizables; Telegram requiere usuario vinculado e inicio del bot. Revoca la autorización vigente de ese contacto y canal y, si corresponde, registra la nueva. Si no queda ningún canal vigente suspende sus participaciones; si quedan canales, las reanuda y conserva la agenda mediante scheduleAttempts. La prioridad no se sustituye automáticamente por Nula al revocar.

6.6 Preparar la prueba gratuita desde el expediente

Figura 44

Prueba gratuita disponible en el expediente

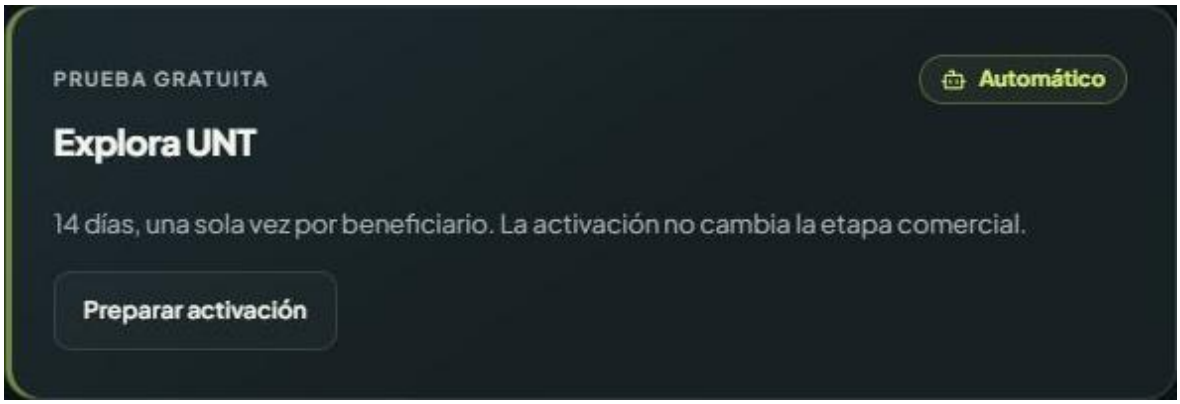
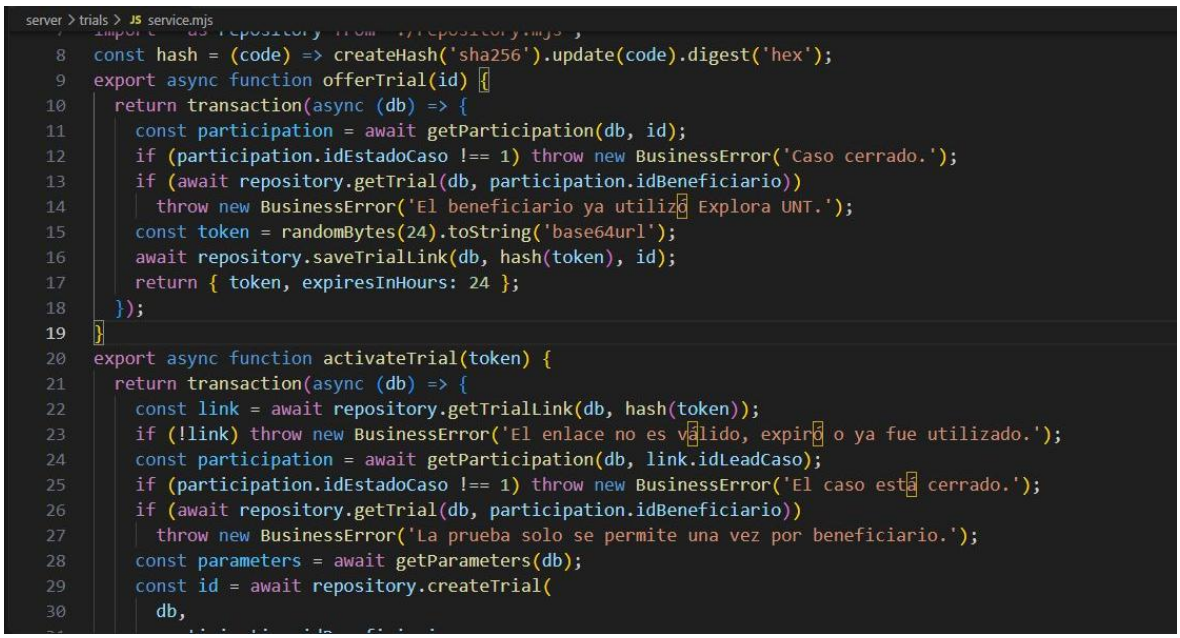


Figura 45

server/trials/service.mjs líneas 8 a 30



```
server > trials > JS service.mjs
8  const hash = (code) => createHash('sha256').update(code).digest('hex');
9  export async function offerTrial(id) {
10   return transaction(async (db) => {
11     const participation = await getParticipation(db, id);
12     if (participation.idEstadoCaso !== 1) throw new BusinessError('Caso cerrado.');
13     if (await repository.getTrial(db, participation.idBeneficiario))
14       throw new BusinessError('El beneficiario ya utilizó Explora UNT.');
15     const token = randomBytes(24).toString('base64url');
16     await repository.saveTrialLink(db, hash(token), id);
17     return { token, expiresInHours: 24 };
18   });
19 }
20 export async function activateTrial(token) {
21   return transaction(async (db) => {
22     const link = await repository.getTrialLink(db, hash(token));
23     if (!link) throw new BusinessError('El enlace no es válido, expiró o ya fue utilizado.');
24     const participation = await getParticipation(db, link.idLeadCaso);
25     if (participation.idEstadoCaso !== 1) throw new BusinessError('El caso está cerrado.');
26     if (await repository.getTrial(db, participation.idBeneficiario))
27       throw new BusinessError('La prueba solo se permite una vez por beneficiario.');
28     const parameters = await getParameters(db);
29     const id = await repository.createTrial(
30       db,
```

Recorrido técnico TrialCard → POST /api/participations/:id/trial-link → offerTrial → activateTrial

Implementación offerTrial comprueba caso abierto y ausencia de uso previo, genera un token aleatorio y almacena su hash. Devuelve un enlace con vigencia de 24 horas. La activación se realiza después mediante el endpoint correspondiente y crea una única prueba por beneficiario con la duración parametrizada. El bloque del expediente utiliza estos servicios para preparar y registrar una activación solicitada; no añade una compra al catálogo.

6.7 Preparar una invitación al responsable económico

Figura 46

Preparación de la invitación al responsable

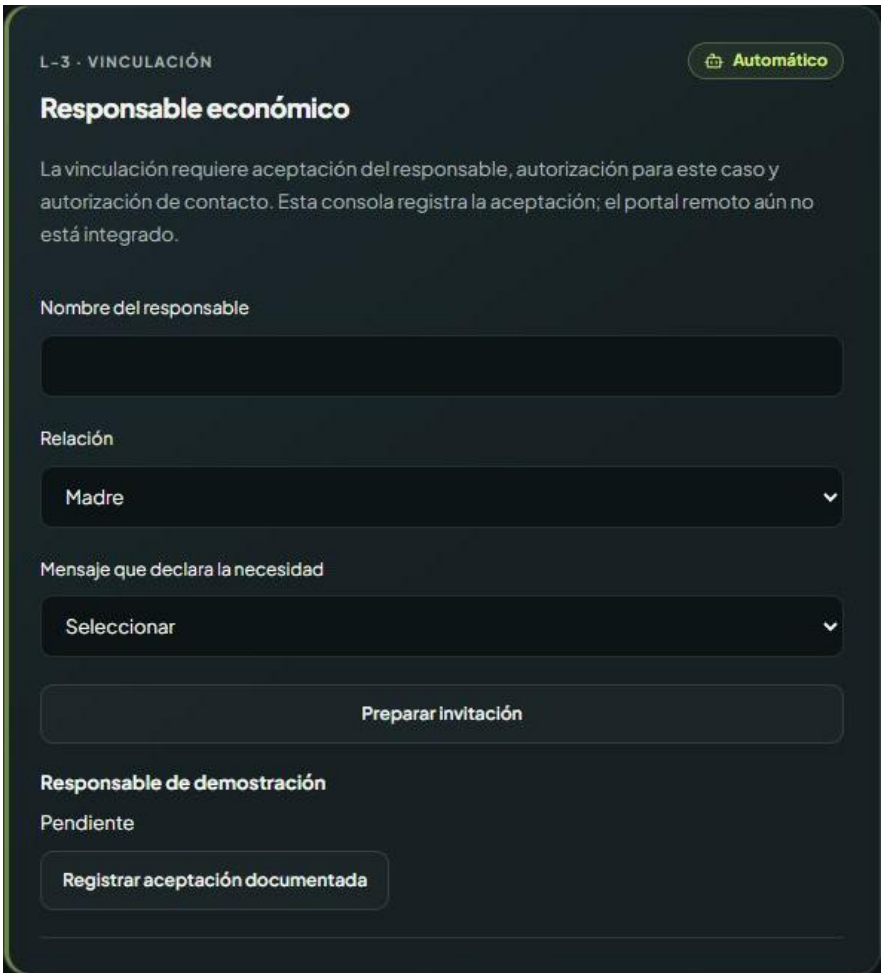
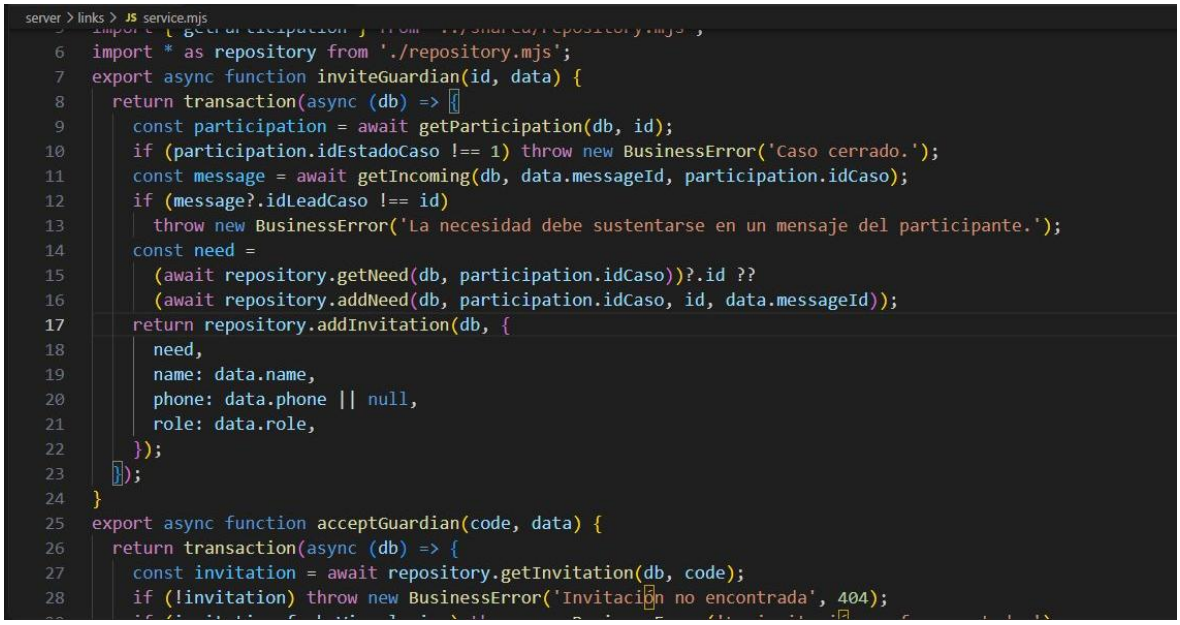


Figura 47

server/links/service.mjs líneas 6 a 28



Recorrido técnico GuardianCard → POST /api/participations/:id/invitations → inviteGuardian

Implementación La invitación requiere un mensaje entrante del mismo participante que declare la necesidad de responsable. inviteGuardian reutiliza o crea NecesidadVinculacion y registra InvitacionVinculacion con nombre, relación y teléfono disponible. Una invitación preparada queda pendiente; todavía no acredita vínculo ni autorización de contacto. La aceptación posterior debe corresponder al mismo caso y guardar las dos decisiones expresas del responsable.

6.8 Registrar una aceptación documentada del responsable

Figura 48

Registro de las dos autorizaciones del responsable

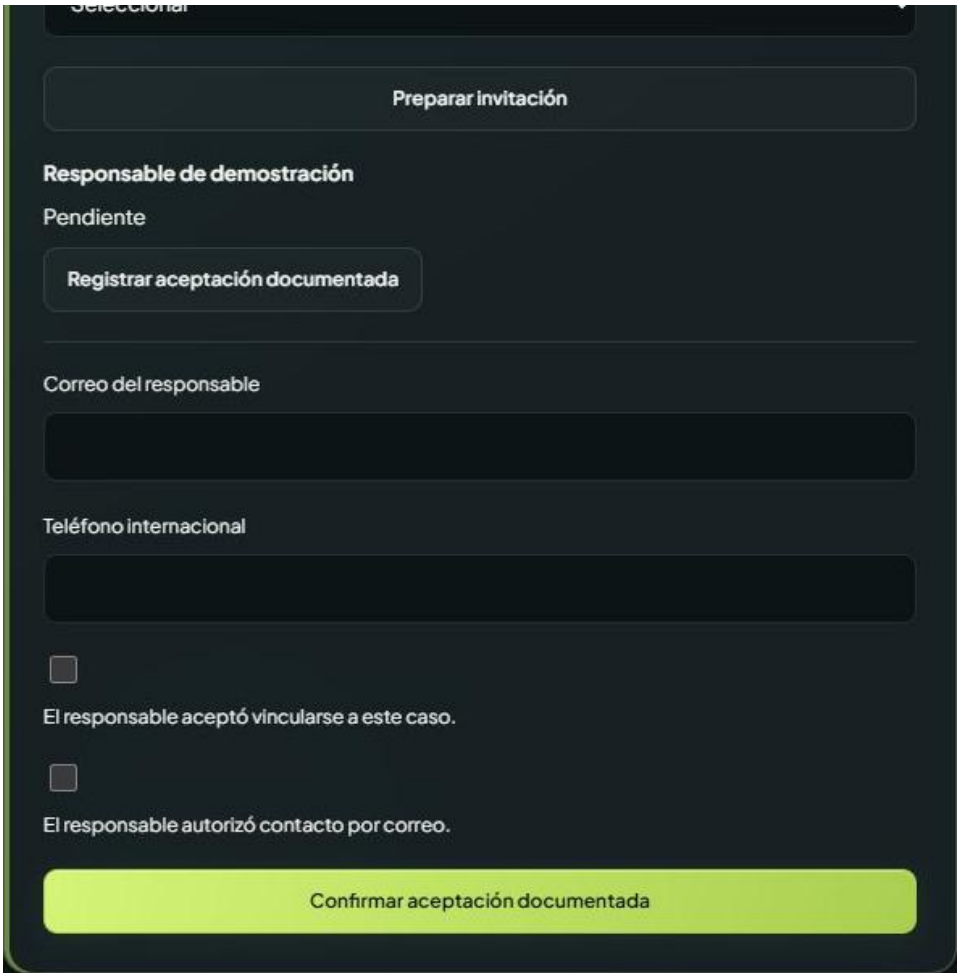


Figura 49

server/links/service.mjs líneas 24 a 46



Recorrido técnico GuardianCard → POST /api/invitations/accept → acceptGuardian → acceptInvitation

Implementación La ruta exige un código UUID, datos del responsable y los dos consentimientos verdaderos. acceptGuardian rechaza invitaciones inexistentes, ya aceptadas o pertenecientes a casos cerrados. createLead crea o reutiliza la participación del responsable para el beneficiario invitado y confirma que el caso no cambió. acceptInvitation registra el vínculo y sus autorizaciones. La entrada manual representa una aceptación documentada por el operador; no debe deducirse del mero envío de una invitación.

7 Propuestas y servicio

7.1 Preparar una propuesta con tarifa vigente

Figura 50

Tarifa contexto y justificación de una propuesta

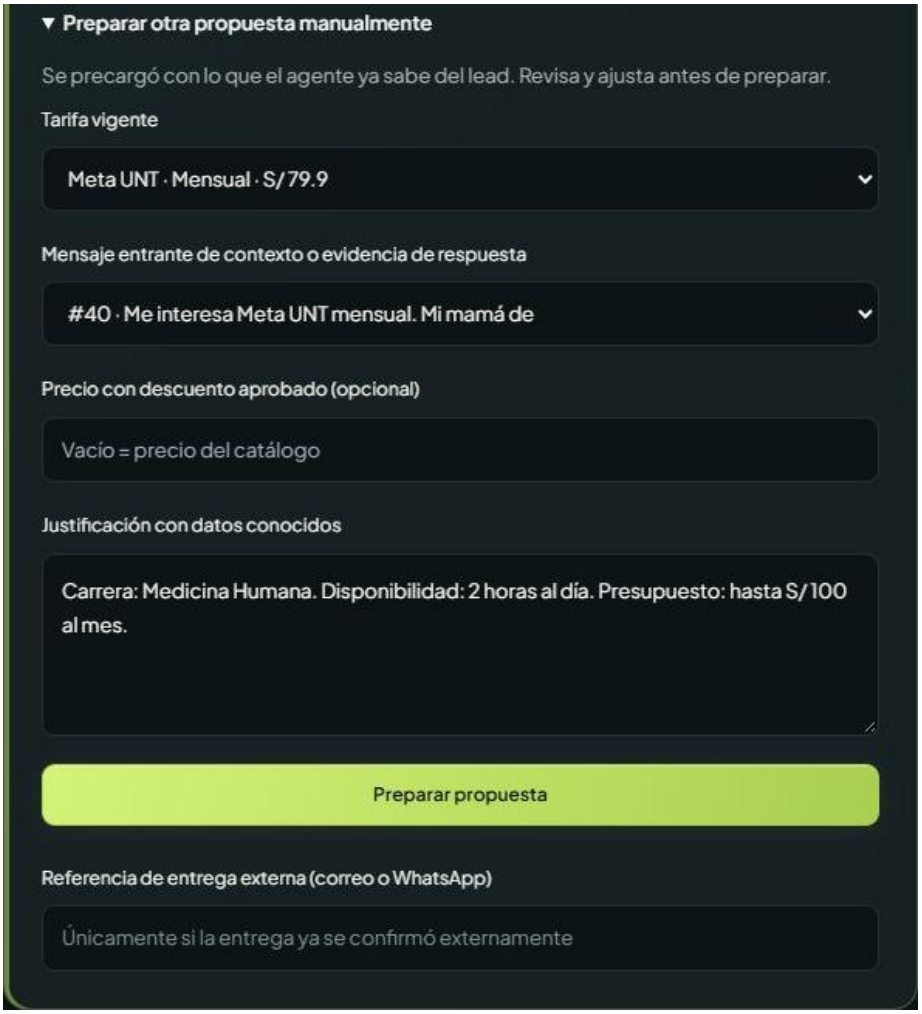
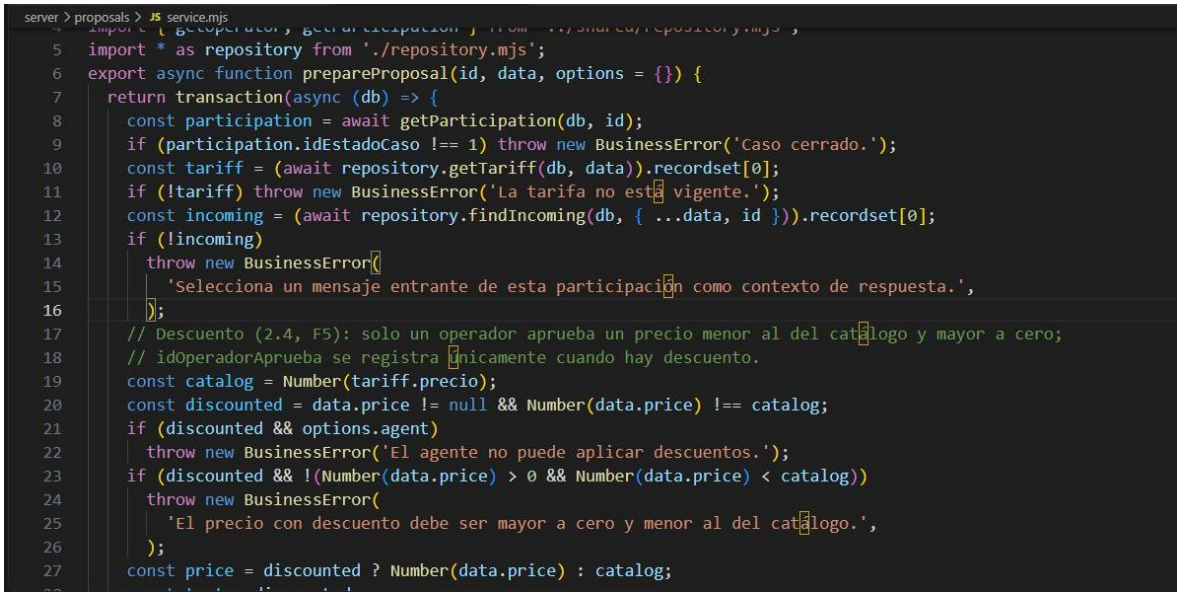


Figura 51

server/proposals/service.mjs líneas 5 a 27



Recorrido técnico ProposalCard ⟶ proposals/routes.mjs ⟶ prepareProposal ⟶ proposals/repository.mjs

Implementación prepareProposal exige caso abierto, tarifa vigente y mensaje entrante de contexto de esa participación. El precio autorizado procede del catálogo. Un descuento debe ser positivo, menor al precio de catálogo y aprobado por un operador; el agente no puede aplicarlo. El servicio inserta un mensaje pendiente y la propuesta vinculada a su tarifa y caso. Devuelve los identificadores y declara que la entrega externa sigue pendiente.

7.2 Consultar el avance y compartir la oferta personal

Figura 52

Avance de una propuesta entregada y enlace personal

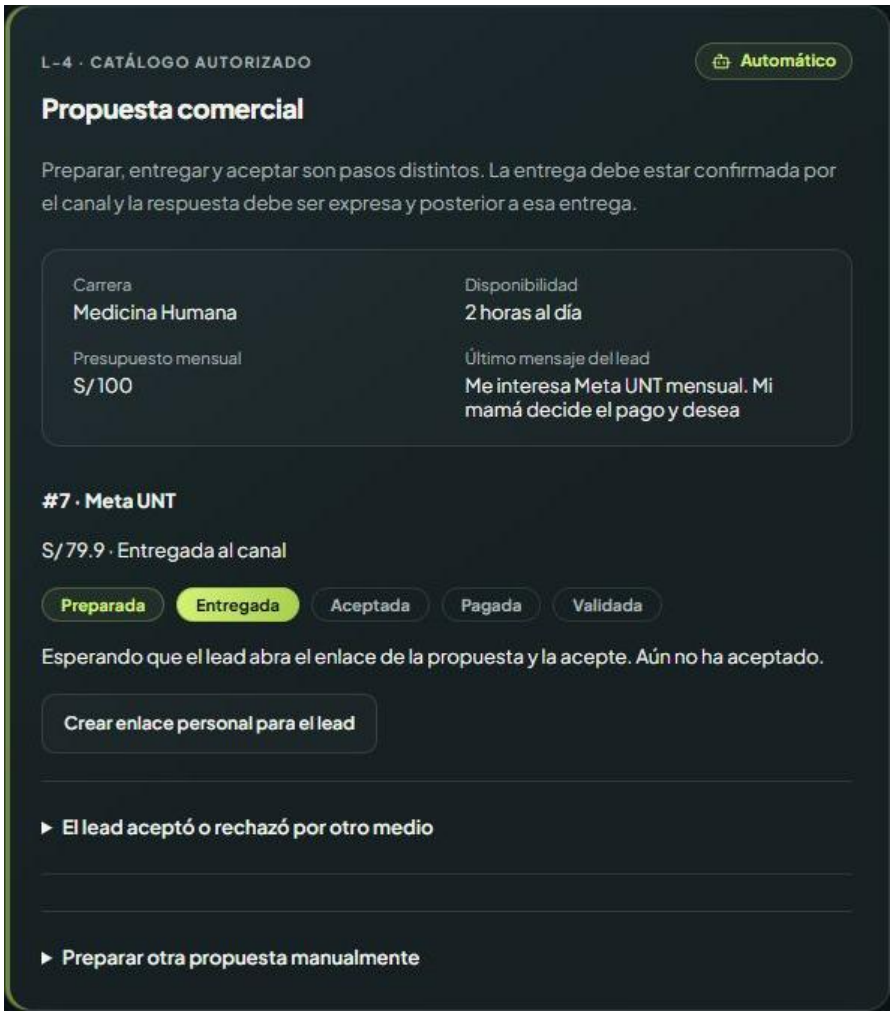
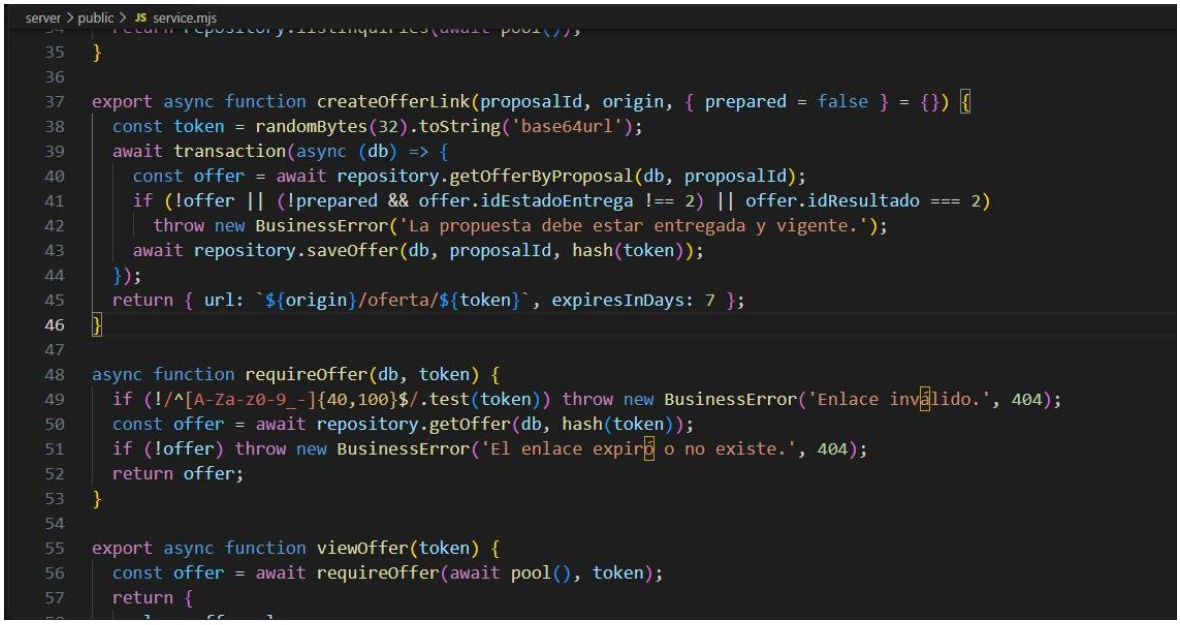


Figura 53

server/public/service.mjs líneas 35 a 57



Recorrido técnico ProposalCard → POST /api/proposals/:id/public-link → createOfferLink

Implementación createOfferLink genera un token aleatorio, conserva su hash y asocia el enlace a una propuesta disponible. Desde la consola se exige entrega confirmada y ausencia de rechazo. La vigencia devuelta es de siete días. La respuesta por otro medio utiliza recordProposalResponse, que valida una evidencia entrante del caso y posterior a la entrega. La aceptación se conserva como evento comercial y no confirma un pago.

7.3 Aceptar la propuesta y sus condiciones en el portal

Figura 54

Oferta personal y aceptación expresa de condiciones

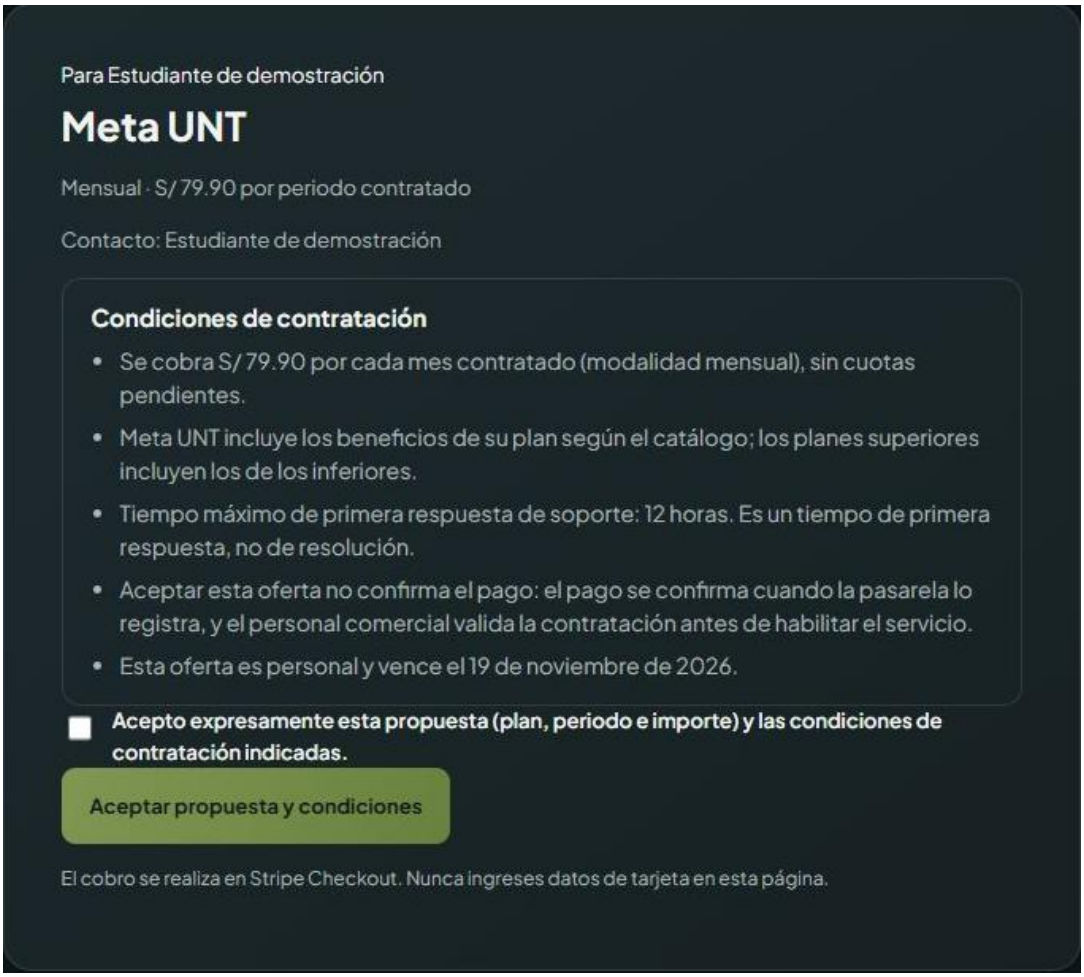
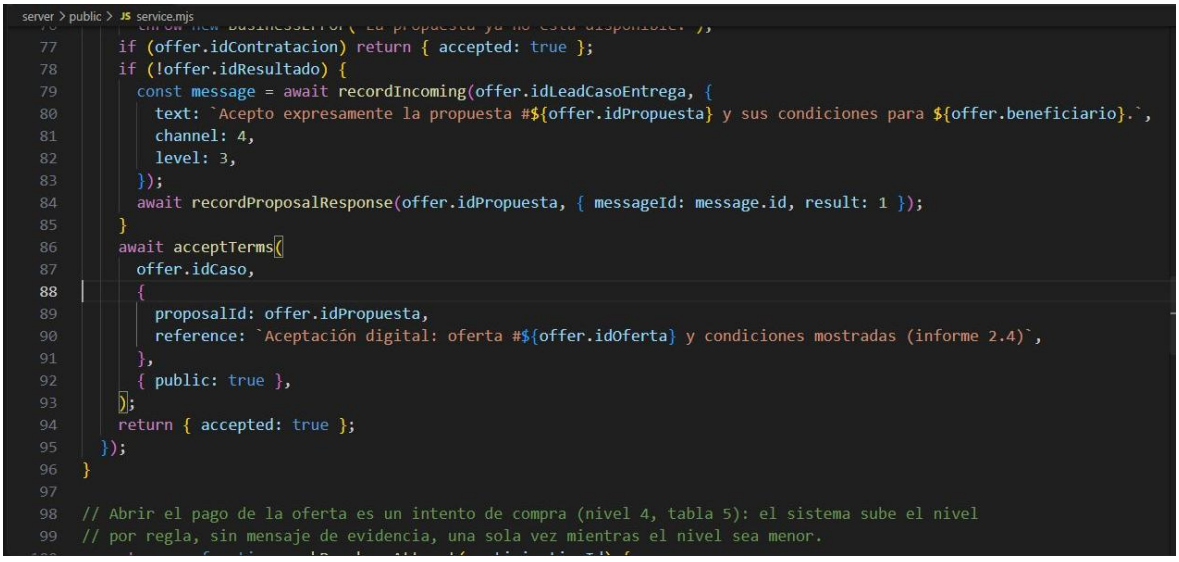


Figura 55

server/public/service.mjs líneas 77 a 99



Recorrido técnico PublicOffer → POST /api/public/offers/:token/accept → acceptOffer → acceptTerms

Implementación El token debe resolver a una oferta vigente y entregada. acceptOffer registra una manifestación expresa por Portal web, conserva la respuesta de la propuesta y llama a acceptTerms para crear la contratación y su evidencia. Una contratación ya registrada se devuelve sin duplicarla. La transacción ambiental de db.mjs permite que los servicios anidados participen en la misma unidad de trabajo. La respuesta accepted no significa pago ni habilitación.

7.4 Abrir el pago de prueba desde la oferta

Figura 56

Opciones de pago después de aceptar las condiciones



Figura 57

server/contracts/service.mjs líneas 84 a 106



Recorrido técnico PublicOffer → payOffer → markPurchaseAttempt → startStripeCheckout → stripe.mjs

Implementación El checkout requiere condiciones aceptadas, caso abierto y ausencia de pago. createSession utiliza el importe de la contratación, no un precio libre enviado por el navegador. La confirmación verifica modo de prueba, moneda PEN, estado paid, metadatos e importe en céntimos antes de recordPayment. payOffer registra además el nivel 4 de intento de compra, una sola vez mientras el nivel sea menor. La simulación exige DEMO_PAYMENTS=1 y no acredita un cobro real.

7.5 Habilitar un servicio adquirido

Figura 58

Servicio adquirido pendiente de habilitación

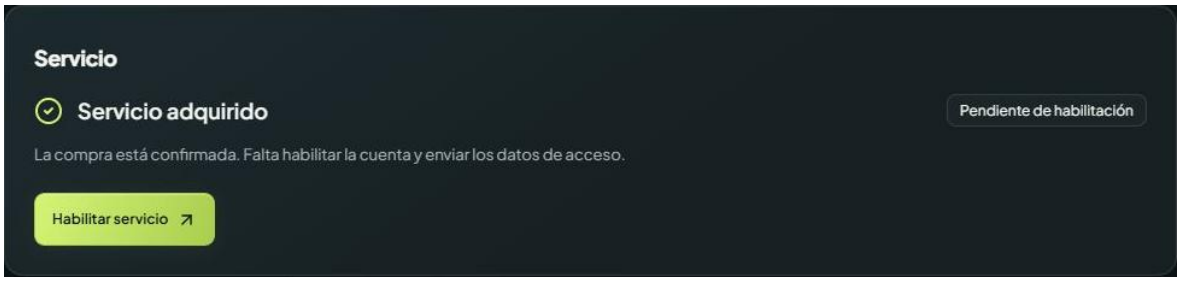
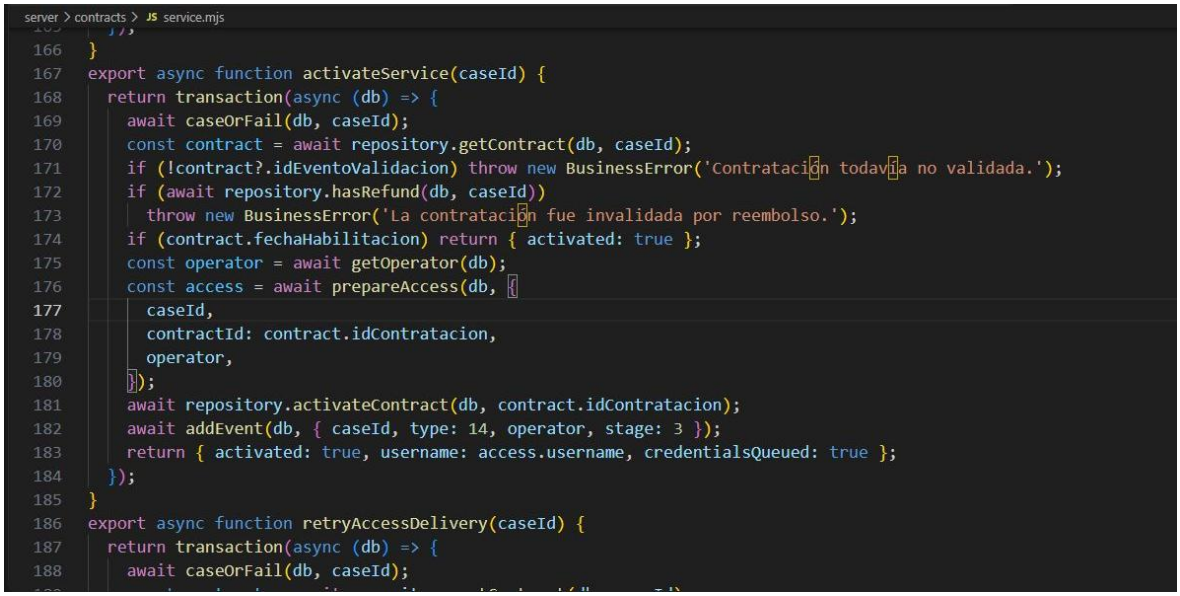


Figura 59

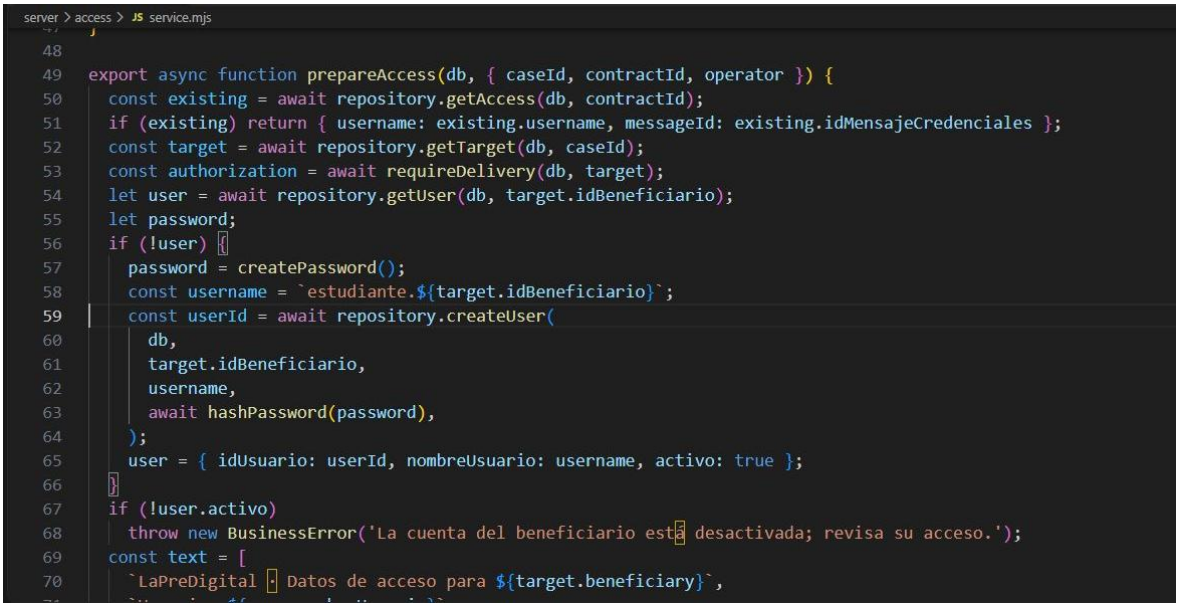
server/contracts/service.mjs líneas 166 a 188



```
server > contracts > JS service.mjs
166  }
167  export async function activateService(caseId) {
168    return transaction(async (db) => {
169      await caseOrFail(db, caseId);
170      const contract = await repository.getContract(db, caseId);
171      if (!contract?.idEventoValidacion) throw new BusinessError('Contratación todavía no validada.');
172      if (await repository.hasRefund(db, caseId))
173        throw new BusinessError('La contratación fue invalidada por reembolso.');
174      if (contract.fechaHabilitacion) return { activated: true };
175      const operator = await getOperator(db);
176      const access = await prepareAccess(db, {
177        caseId,
178        contractId: contract.idContratacion,
179        operator,
180      });
181      await repository.activateContract(db, contract.idContratacion);
182      await addEvent(db, { caseId, type: 14, operator, stage: 3 });
183      return { activated: true, username: access.username, credentialsQueued: true };
184    });
185  }
186  export async function retryAccessDelivery(caseId) {
187    return transaction(async (db) => {
188      await caseOrFail(db, caseId);
```

Figura 60

server/access/service.mjs líneas 48 a 70



Recorrido técnico ContractCard o Pending → activateService → prepareAccess → access/repository.mjs

Implementación activateService exige validación contractual y ausencia de reembolso invalidante; una habilitación existente es idempotente. prepareAccess verifica el destinatario y su canal autorizado y configurado, crea o reutiliza UsuarioPlataforma y vincula HabilitacionAcceso a la contratación y mensaje. Una cuenta nueva guarda una contraseña con hash scrypt y sal; el contenido temporal de entrega se cifra. La habilitación devuelve credentialsQueued: el envío se procesa separadamente y el historial no expone la contraseña.

prepareAccess reutiliza el acceso por contratación y el usuario por beneficiario. Solo una cuenta nueva genera contraseña; la persistencia recibe su hash, mientras el mensaje temporal se cifra por separado.

7.6 Consultar el servicio habilitado

Figura 61

Resumen del servicio adquirido y habilitado

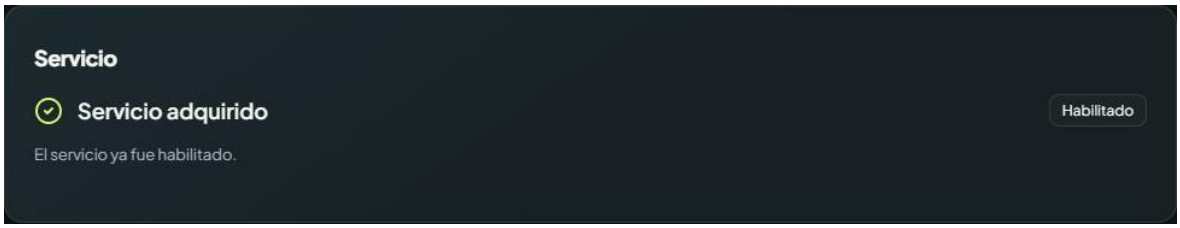


Figura 62

src/features/contracts/ContractCard.tsx líneas 4 a 26



Recorrido técnico Profile → ContractCard → detail.contracts → resumen Servicio

Implementación ContractCard calcula acquired cuando hay evento de pago, evento de validación y ausencia de invalidación. enabled añade fechaHabilitacion. La descripción distingue compra ausente, pago pendiente de validación, servicio pendiente y servicio habilitado. Solo acquired sin enabled ofrece el botón que llama a /cases/:id/activate. Esta presentación no crea una cuenta al consultarse ni demuestra que las credenciales hayan llegado; la entrega se revisa en Pendientes.

8 Catálogo interno

8.1 Consultar planes y tarifas mensuales

Figura 63

Tarifas mensuales y selección del contacto en el catálogo interno

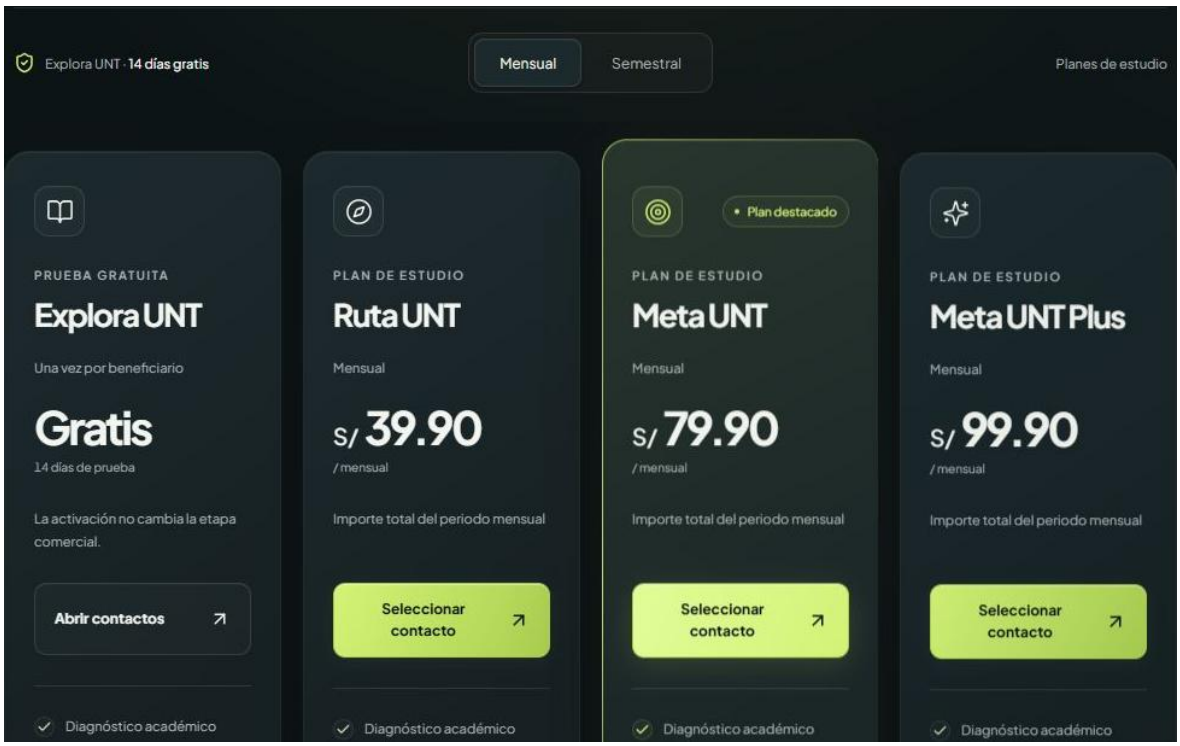


Figura 64

src/pages/Catalog.tsx líneas 128 a 150



Recorrido técnico Catalog → useWorkspace → data.tariffs → PlanCard y TrialPlanCard

Implementación La vista utiliza las tarifas cargadas por bootstrap y los beneficios definidos en src/shared/offer.json. Agrupa las alternativas comerciales y conserva Explora como prueba gratuita. Seleccionar contacto cambia al contexto de una participación para preparar una propuesta; consultar una tarjeta no inserta una venta. El precio contractual se vuelve a validar en el backend contra PlanTarifa, por lo que el estado del catálogo en el navegador no autoriza por sí mismo un importe.

8.2 Comparar la modalidad semestral

Figura 65

Comparación de planes con modalidad semestral



Figura 66

src/pages/Catalog.tsx líneas 21 a 43



Recorrido técnico Catalog.period → PlanCard → tariffs.find por modalidad

Implementación Las modalidades disponibles se obtienen de data.tariffs. PlanCard busca la tarifa del periodo elegido y, si el plan no la ofrece, utiliza su tarifa existente y muestra que no hay modalidad semestral. No calcula una tarifa semestral multiplicando meses ni inventa financiación. Los importes corresponden al periodo contratado. El selector modifica presentación y selección; prepareProposal vuelve a comprobar la vigencia de la tarifa elegida.

9 Seguimiento autorizado

9.1 Consultar la agenda y revisar vencimientos

Figura 67

Agenda y revisión manual de vencimientos

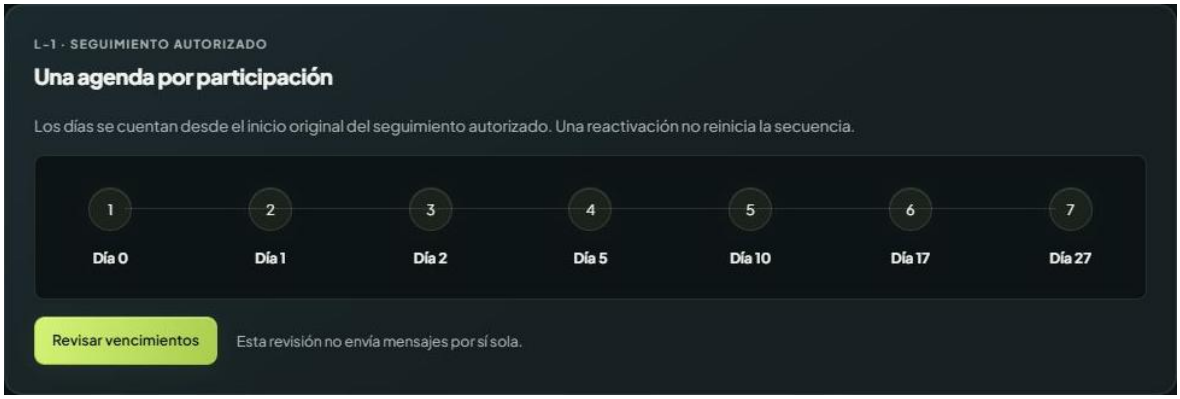


Figura 68

server/followup/service.mjs líneas 32 a 54

```
server > followup > JS service.mjs
// así que los intentos vencidos quedan pendientes para el worker en lugar de omitirse.
export async function reviewFollowup({ canSend = () => false, prepare = true } = {}) {
  return transaction(async (db) => {
    const result = { prepared: 0, omitted: 0, evaluated: 0, closed: 0 };
    await reviewMaintenance();
    const parameters = await getParameters(db);
    const pending = prepare ? await repository.listDue(db) : [];
    for (const attempt of pending) {
      const deadline = attempt.nextAttempt
        ? new Date(attempt.nextAttempt + '-05:00')
        : new Date(
            new Date(attempt.fechaProgramada + '-05:00').getTime() +
              parameters.PLAZO_RESPUESTA_TRAS_SEPTIMO_DIAS * 86400000,
          );
      const expired = deadline <= new Date();
      if (
        attempt.idEstadoCaso !== 1 ||
        attempt.idEstadoParticipacion !== 1 ||
        attempt.idPrioridadActual === 5 ||
        attempt.handoff ||
        attempt.blocked ||
        attempt.scholarshipPending ||
        expired
```

Recorrido técnico Followup ⟶ POST /api/followup/run ⟶ reviewFollowup({prepare:false})

Implementación La revisión manual ejecuta mantenimiento, reevaluaciones y cierres aplicables, sin preparar mensajes. El worker llama al mismo servicio con preparación habilitada y un predicado canSend por canal. Antes de preparar un intento controla caso, participación, prioridad, derivación, bloqueo, beca pendiente, vencimiento y autorización. La fecha límite respeta hora local UTC menos cinco. La agenda y los conteos dependen de IntentoContacto y Mensaje; programar o preparar no acredita entrega.

10 Indicadores de la fase LEADS

10.1 Consultar el resumen de indicadores

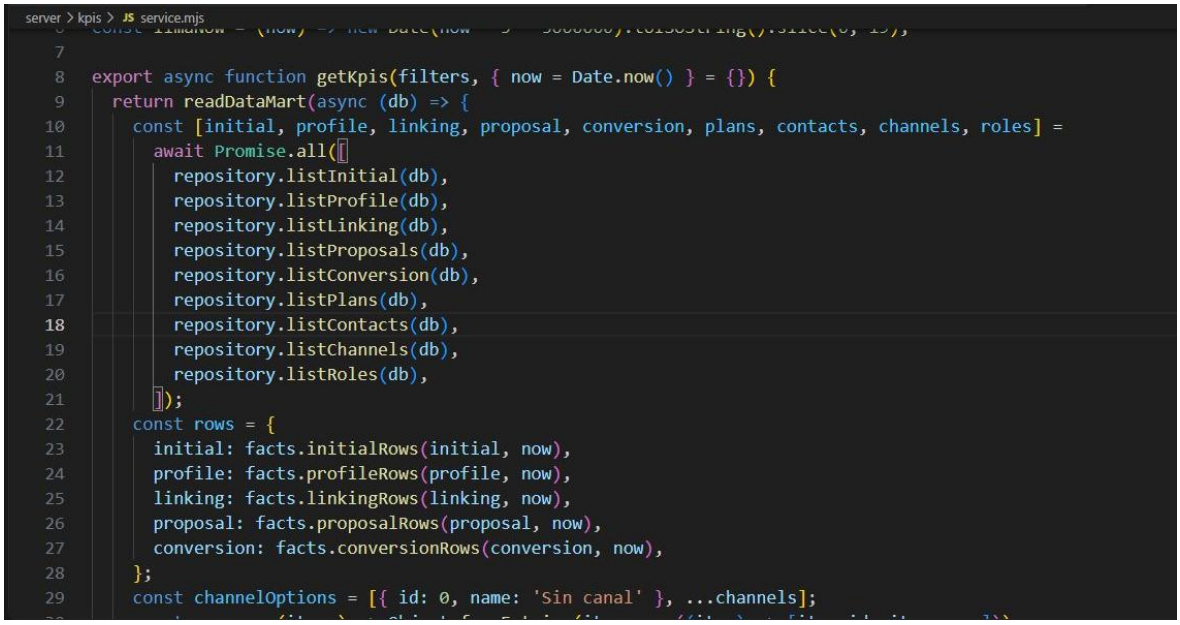
Figura 69

Resumen de los seis indicadores de la fase LEADS



Figura 70

server/kpis/service.mjs líneas 7 a 29



Recorrido técnico Kpis → GET /api/kpis → getKpis → readDataMart → kpis/repository.mjs

Implementación getKpis lee los cinco hechos y los catálogos dimensionales mediante consultas al Data Mart. datamart-rows.mjs convierte claves, instantes y medidas en filas de análisis y KPI_DEFINITIONS aplica las reglas de cada tasa. La respuesta identifica la base fuente y el momento de consulta. Los recuentos de Inicio provienen de bootstrap y son independientes. La versión actual incorpora DimCaso además de las seis dimensiones originales del informe; esta ampliación permite relacionar los hechos mediante KeyCaso.

Dimensiones y claves del modelo vigente

Dimensión	Identificador de origen y atributos	Clave de relación
DimTiempo	fechaHora única; fecha, anio, trimestre, mes y dia	KeyTiempo
DimContacto	idContacto; nombreCompleto	KeyContacto
DimCanal	idCanal; nombre	KeyCanal
DimRolContacto	idRolContacto; nombre	KeyRolContacto
DimPlan	idPlan; nombre	KeyPlan
DimPrioridad	idPrioridad; nombre	KeyPrioridad
DimCaso	idCaso; beneficiario, estado y motivoCierre	KeyCaso

Todas las claves de relación se definen como INT IDENTITY. Los identificadores de origen se conservan como atributos únicos y no sustituyen las claves en las relaciones de hechos. DimTiempo conserva instantes DATETIME2 de precisión de segundos: sus roles permiten comparar elegibilidad, entrega, respuesta, pago y validación. La jerarquía temporal es año, trimestre, mes y día.

Figura 71

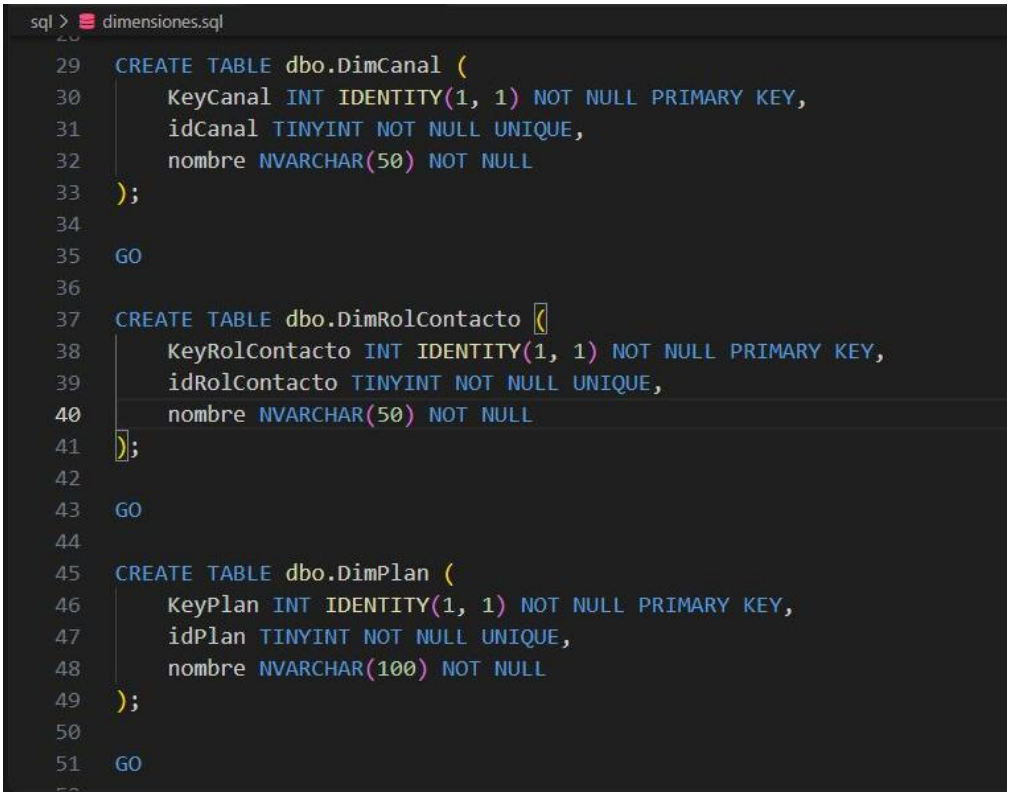
sql/dimensiones.sql líneas 8 a 30

```
sql > dimensiones.sql
 8
 9  CREATE TABLE dbo.DimTiempo (
10      KeyTiempo INT IDENTITY(1, 1) NOT NULL PRIMARY KEY,
11      fechaHora DATETIME2(0) NOT NULL UNIQUE,
12      fecha DATE NOT NULL,
13      anio SMALLINT NOT NULL,
14      trimestre TINYINT NOT NULL CHECK (trimestre BETWEEN 1 AND 4),
15      mes TINYINT NOT NULL CHECK (mes BETWEEN 1 AND 12),
16      dia TINYINT NOT NULL CHECK (dia BETWEEN 1 AND 31)
17  );
18
19  GO
20
21  CREATE TABLE dbo.DimContacto (
22      KeyContacto INT IDENTITY(1, 1) NOT NULL PRIMARY KEY,
23      idContacto BIGINT NOT NULL UNIQUE,
24      nombreCompleto NVARCHAR(150) NOT NULL
25  );
26
27  GO
28
29  CREATE TABLE dbo.DimCanal (
30      KeyCanal INT IDENTITY(1, 1) NOT NULL PRIMARY KEY,
```

DimTiempo conserva fechaHora única y su jerarquía año, trimestre, mes y día. DimContacto mantiene idContacto como identificador de origen. Las relaciones del Data Mart utilizan las claves surrogate identity KeyTiempo y KeyContacto.

Figura 72

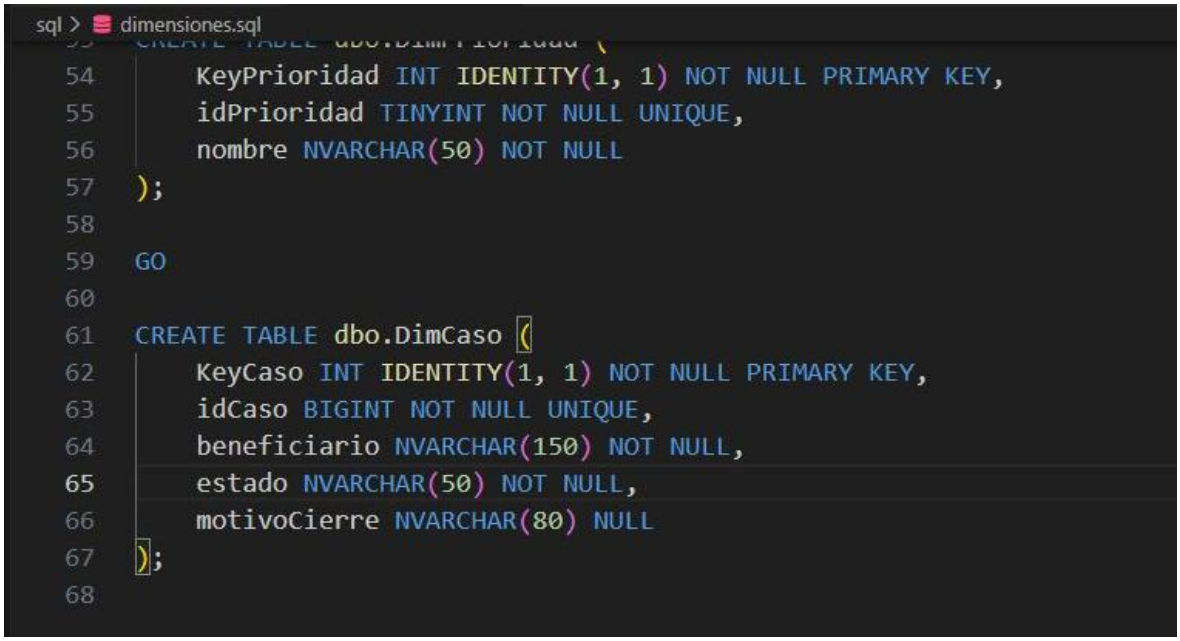
sql/dimensiones.sql líneas 29 a 51



DimCanal, DimRolContacto y DimPlan relacionan la clave sustituta con el identificador natural y el nombre del catálogo. DimPrioridad sigue el mismo patrón. Estas seis dimensiones corresponden al conjunto original solicitado.

Figura 73

sql/dimensiones.sql líneas 54 a 67



La implementación actual añade DimCaso con KeyCaso identity, idCaso único y atributos de beneficiario y estado. Es una ampliación del conjunto de seis dimensiones originales y se documenta como tal.

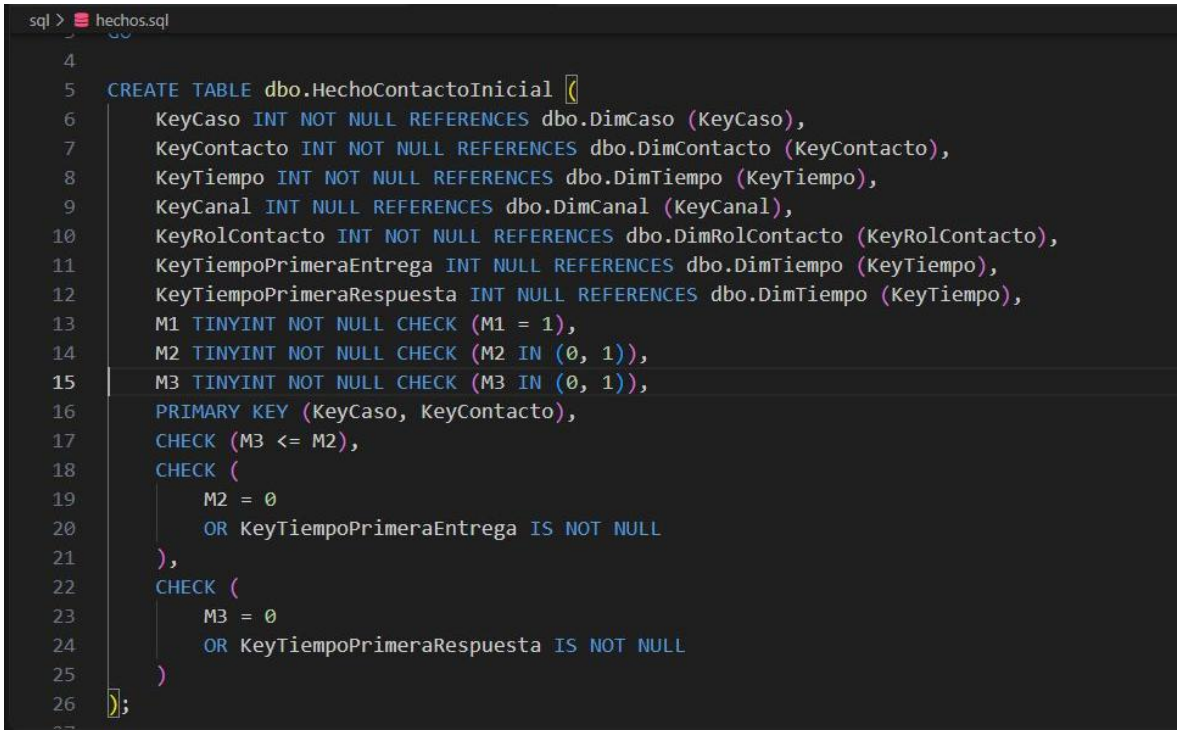
Hechos y unidad de almacenamiento

Hecho	Clave primaria y medidas
HechoContactoInicial	KeyCaso y KeyContacto; M1, M2 y M3.
HechoAmpliacionPerfil	KeyContacto, KeyTiempo, KeyCanal y KeyRolContacto; M4 y M5.
HechoVinculacionEconomica	KeyCaso y KeyTiempo; M6 y M7.
HechoPropuestaComercial	KeyCaso, KeyTiempo, KeyCanal y KeyPlan; M8 y M9.
HechoConversionPagada	KeyCaso; M10 y M11.

Los hechos contienen claves y medidas. Las claves temporales adicionales son distintos roles de DimTiempo. Las restricciones impiden, por ejemplo, que M3 supere M2 o que M11 supere M10 y exigen fechas cuando existe un logro. La unidad almacenada y la unidad del KPI se relacionan mediante la deduplicación de datamart-rows.mjs y count; una fila no equivale necesariamente a una persona distinta.

Figura 74

sql/hechos.sql líneas 4 a 26



HechoContactoInicial utiliza únicamente claves dimensionales y M1, M2 y M3. La clave compuesta es KeyCaso con KeyContacto. Las fechas de entrega y respuesta son roles de DimTiempo, no atributos descriptivos libres del hecho.

Origen transaccional de las medidas

sql/etl_hechos.sql deriva las medidas de eventos del OLTP. Los valores almacenados son indicadores de pertenencia o logro; datamart-rows.mjs y count agrupan las unidades antes de calcular las tasas, por lo que no se suman filas sin considerar su granularidad.

Medidas	Tablas de origen en el OLTP	Condición implementada
Contacto inicial M1, M2 y M3	LeadCaso, Lead, IntentoContacto y Mensaje	M1 identifica la participación con fechaInicioElegibilidad. M2 exige primera entrega de la agenda dentro de las 168 horas desde la elegibilidad. M3 exige M2 y un mensaje entrante no automático, posterior a esa entrega y dentro de las siguientes 168 horas.
Perfil M4 y M5	PreguntaPerfil, Mensaje, LeadCaso y CambioPerfil	M4 identifica una pregunta entregada, agrupada por contacto, instante, canal y rol. M5 exige un CambioPerfil de tipo 1 o 2, sin idOperadorCorrige, cuya fechaRespuesta esté dentro de las 168 horas desde la entrega de la pregunta.
Vinculación M6 y M7	NecesidadVinculacion, InvitacionVinculacion, AutorizacionVinculacion, AutorizacionContacto, LeadCaso, Lead y Mensaje	M6 identifica la primera necesidad nueva del caso en el mes, sin duplicar una necesidad anterior pendiente. M7 exige vínculo y las dos autorizaciones coincidentes y no revocadas al lograrse, dentro de las 168 horas desde la necesidad. fechaLogro toma el último de los tres instantes requeridos.
Propuesta M8 y M9	PropuestaComercial, Mensaje, PlanTarifa y EventoCaso	M8 identifica una propuesta entregada, agrupada por caso, instante, canal y plan. M9 exige aceptación expresa dentro de las 168 horas desde la entrega: mensaje entrante no automático o evento contractual de tipo 5 sin corrección.
Conversión M10 y M11	EvaluacionPrioridad, LeadCaso, Lead, Contratacion y EventoCaso	Se parte de la primera Alta válida y no corregida del caso. M10 vale 1 si no hay beca completa en su ventana de 168 horas. M11 exige además importe confirmado positivo, pago y validación contractual dentro de esa ventana y ausencia de reembolso invalidante en ella.

Las marcas de tiempo se relacionan mediante los distintos roles de DimTiempo. Las correcciones y exclusiones se resuelven en las vistas auxiliares del ETL; el repositorio de KPI recupera las claves y las medidas ya cargadas en el Data Mart.

10.2 Aplicar filtros por periodo y dimensión

Figura 75

Filtros de periodo y dimensiones aplicables



Figura 76

Indicadores que admiten el filtro de canal

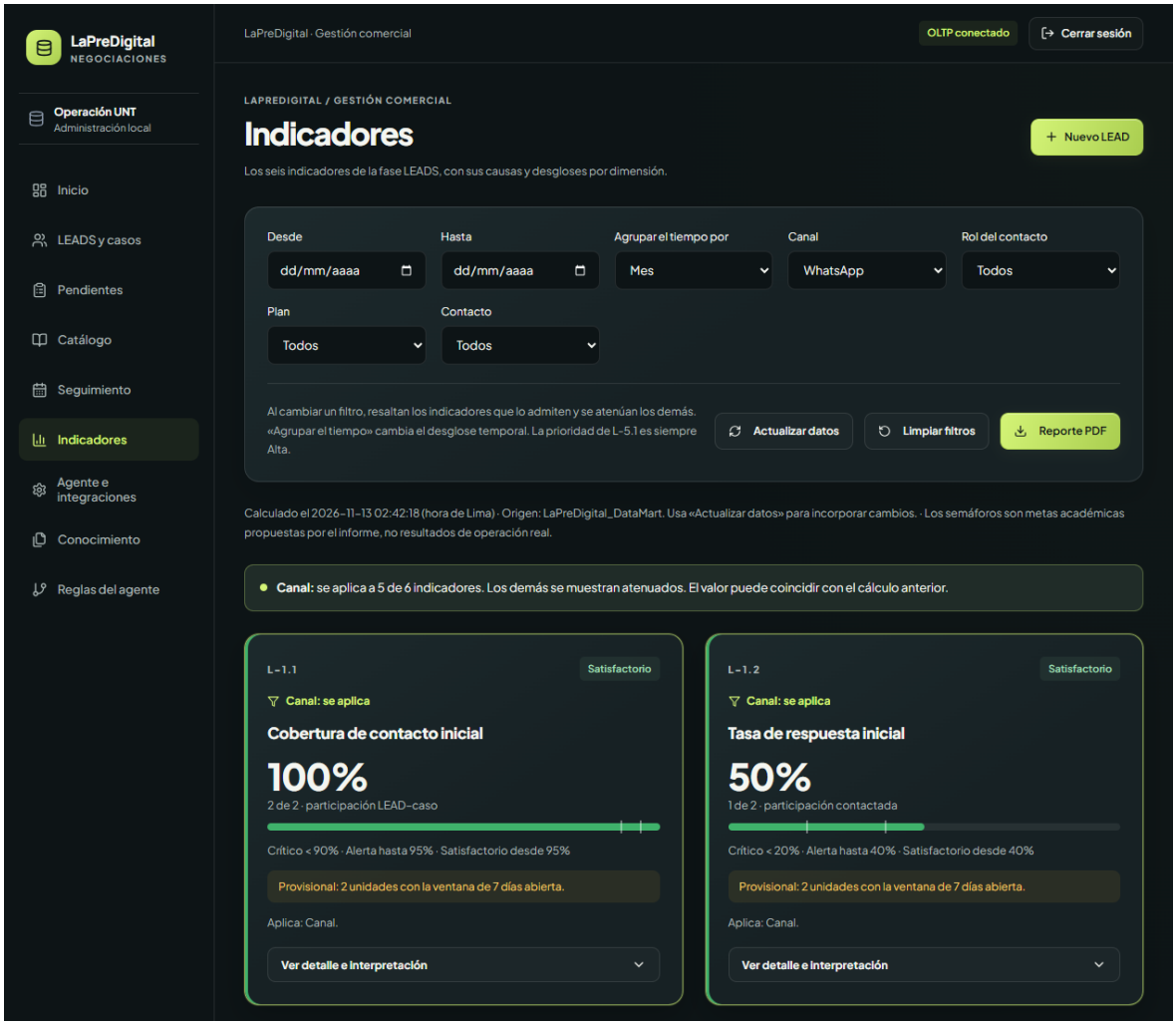
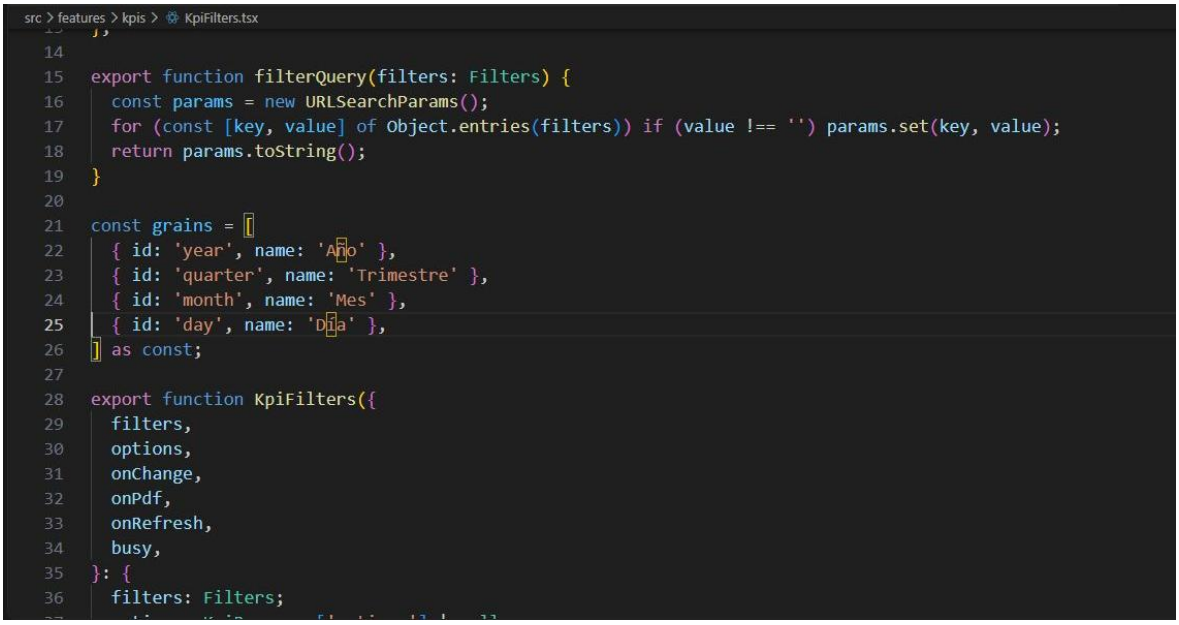


Figura 77

src/features/kpis/KpiFilters.tsx líneas 14 a 36



Recorrido técnico KpiFilters → filterQuery → kpis/routes.mjs → matchesFilters

Implementación filterQuery codifica fechas, agrupación y dimensiones elegidas en la consulta HTTP. La ruta valida los tipos y que la fecha inicial no supere la final. matchesFilters aplica solo las dimensiones declaradas por cada KPI: L 1 a L 3 usan tiempo, contacto, canal y rol; L 4 usa tiempo, canal y plan; L 5 usa tiempo, contacto y rol de la primera Alta. La prioridad Alta es condición de su cohorte. Cambiar filtros no ejecuta el ETL.

10.3 Interpretar causas y desgloses

Figura 78

Detalle causas y desglose por canal

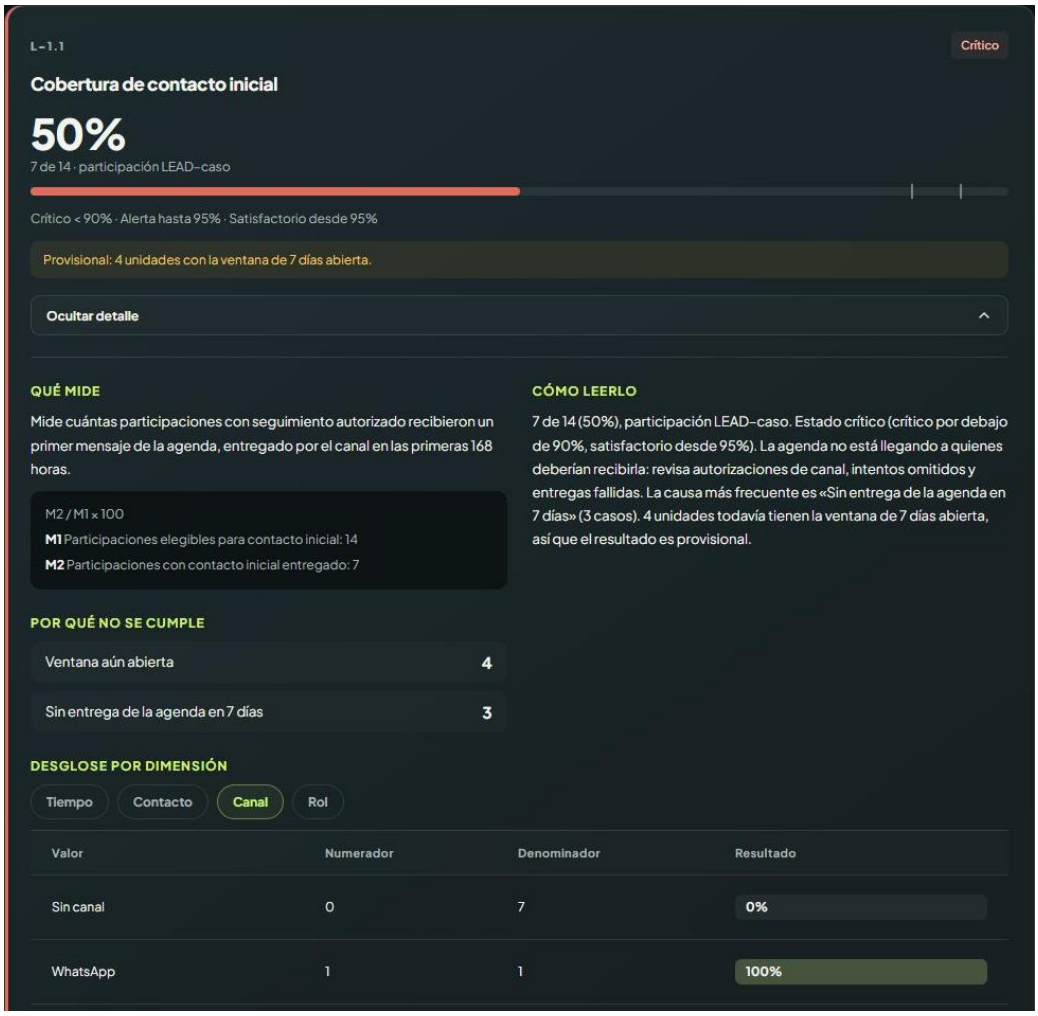
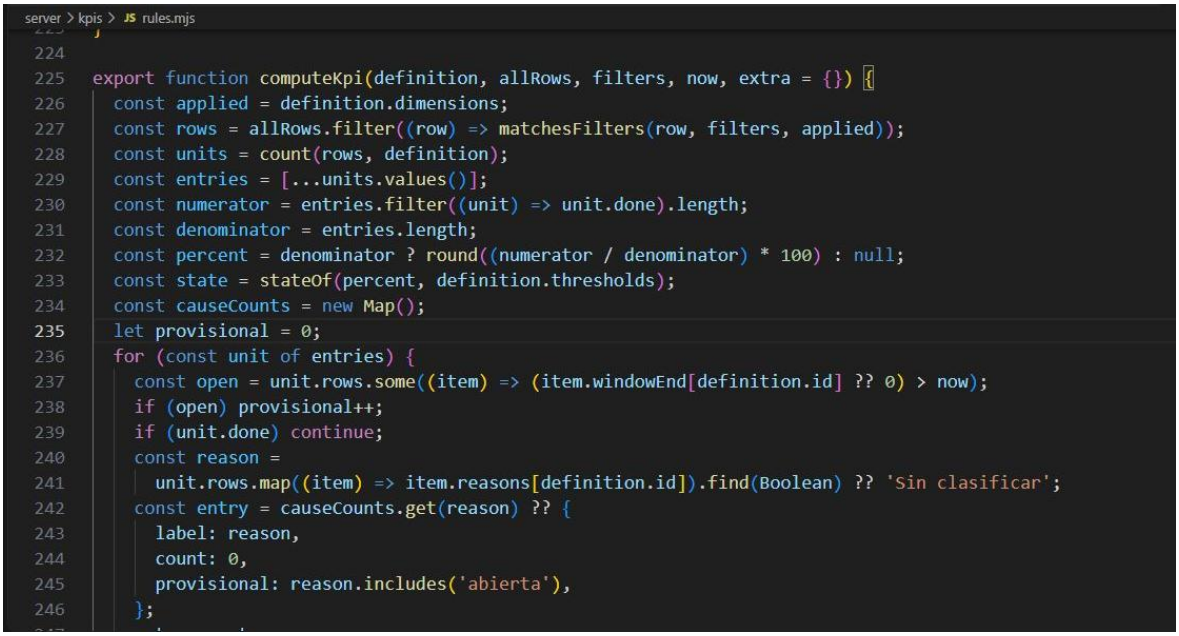


Figura 79

server/kpis/rules.mjs líneas 224 a 246



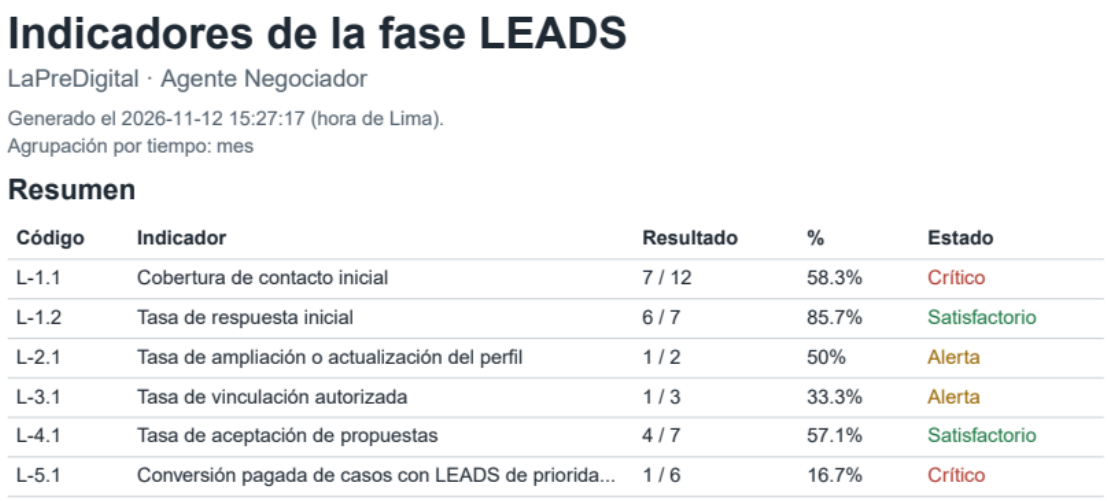
Recorrido técnico KpiCard → datos de computeKpi → count, group e interpret

Implementación computeKpi filtra las filas, agrupa por unitKey y cuenta una unidad una vez aunque tenga varias filas. El numerador se cumple si alguna fila elegible logra la condición. Un denominador cero produce null y estado No aplicable; el porcentaje se redondea a un decimal. El servicio calcula causas, ventanas abiertas y desgloses con las dimensiones permitidas. Los segmentos no deben sumarse ignorando la unidad de análisis, porque pueden compartir casos o contactos.

10.4 Actualizar los datos y exportar el reporte

Figura 80

Resumen del reporte PDF generado por la aplicación



Indicadores de la fase LEADS

LaPreDigital · Agente Negociador

Generado el 2026-11-12 15:27:17 (hora de Lima).
Agrupación por tiempo: mes

Resumen

Código	Indicador	Resultado	%	Estado
L-1.1	Cobertura de contacto inicial	7 / 12	58.3%	Crítico
L-1.2	Tasa de respuesta inicial	6 / 7	85.7%	Satisfactorio
L-2.1	Tasa de ampliación o actualización del perfil	1 / 2	50%	Alerta
L-3.1	Tasa de vinculación autorizada	1 / 3	33.3%	Alerta
L-4.1	Tasa de aceptación de propuestas	4 / 7	57.1%	Satisfactorio
L-5.1	Conversión pagada de casos con LEADS de priorida...	1 / 6	16.7%	Crítico

Figura 81

server/datamart/service.mjs líneas 110 a 132

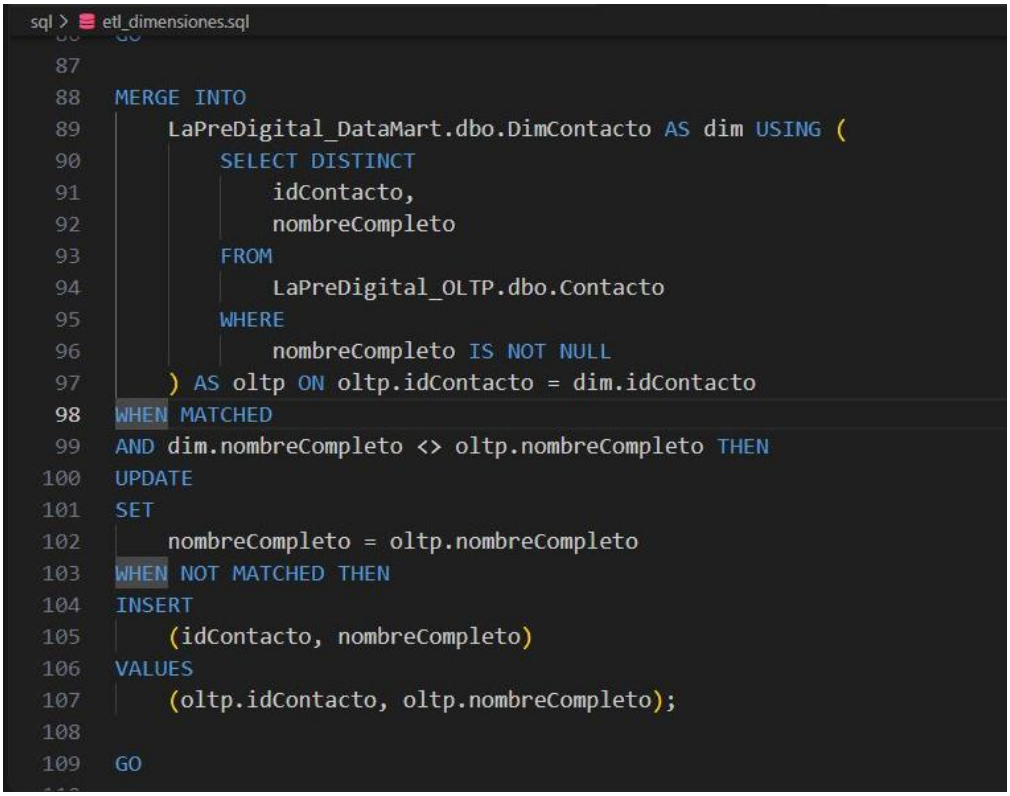


Recorrido técnico POST /api/kpis/refresh → refreshDataMart → load; GET /api/kpis/report.pdf → createKpiReport

Implementación load ejecuta los lotes separados por GO de etl_dimensiones.sql y etl_hechos.sql. refreshDataMart utiliza una transacción con bloqueo exclusivo; las consultas obtienen bloqueo compartido para evitar lecturas de cargas parciales. ready comprueba tablas, claves identity y columnas permitidas en hechos. La exportación vuelve a obtener los KPI con los mismos filtros y transmite el PDF generado por PDFKit. Consultar o exportar no ejecuta automáticamente una nueva carga.

Figura 82

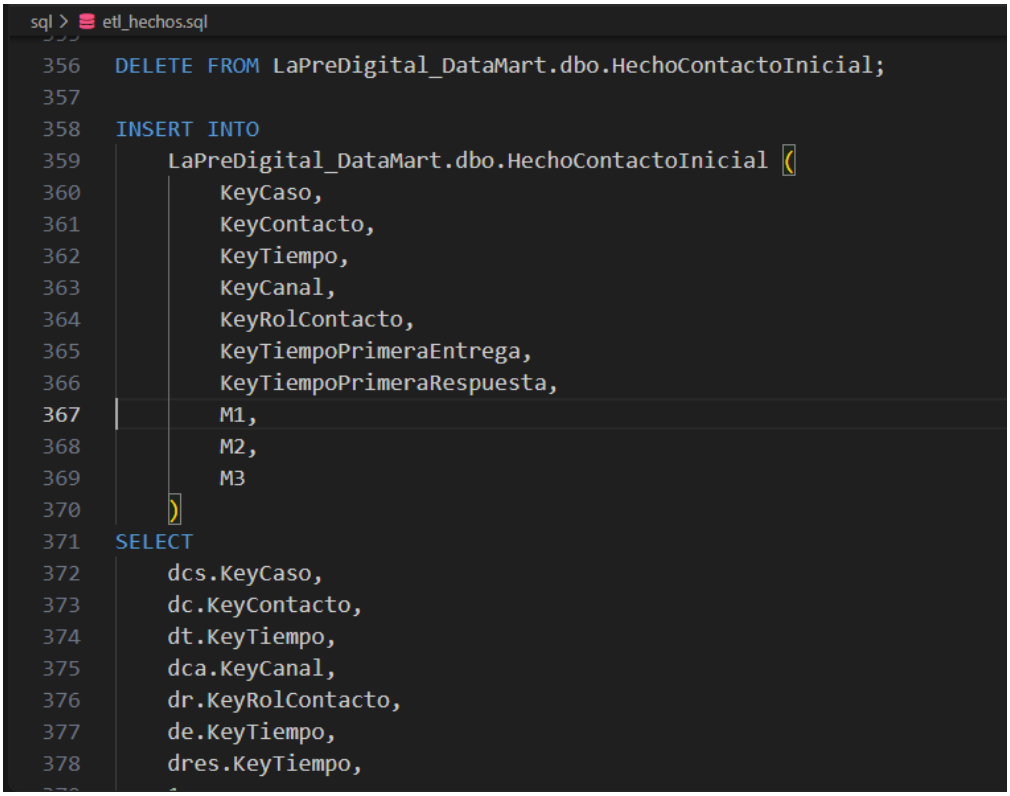
sql/etl_dimensiones.sql líneas 87 a 109



Los MERGE comparan el identificador de origen e insertan filas no encontradas, dejando que SQL Server genere las claves identity. La carga de tiempo unifica los instantes necesarios y evita duplicados; el script también carga DimCaso en la versión vigente.

Figura 83

sql/etl_hechos.sql líneas 356 a 378



El script prepara vistas auxiliares de eventos, elimina el contenido de cada hecho y vuelve a insertarlo mediante uniones con las dimensiones. Las claves se resuelven desde los identificadores del OLTP. Las medidas son indicadores de pertenencia y logro; las fechas conservan el contexto para la ventana.

La carga de hechos usa vistas auxiliares definidas en el propio script y un patrón DELETE seguido de INSERT. No utiliza OUTER APPLY. Los instantes requeridos por los hechos se cargan antes en DimTiempo; después las uniones resuelven sus claves. El cargador omite los lotes USE y ejecuta cada lote en la base analítica configurada. Este manual documenta el proceso sin ejecutarlo sobre los datos operativos.

11 Agente e integraciones

11.1 Consultar el estado de los servicios

Figura 84

Consulta del estado de integraciones y modelos

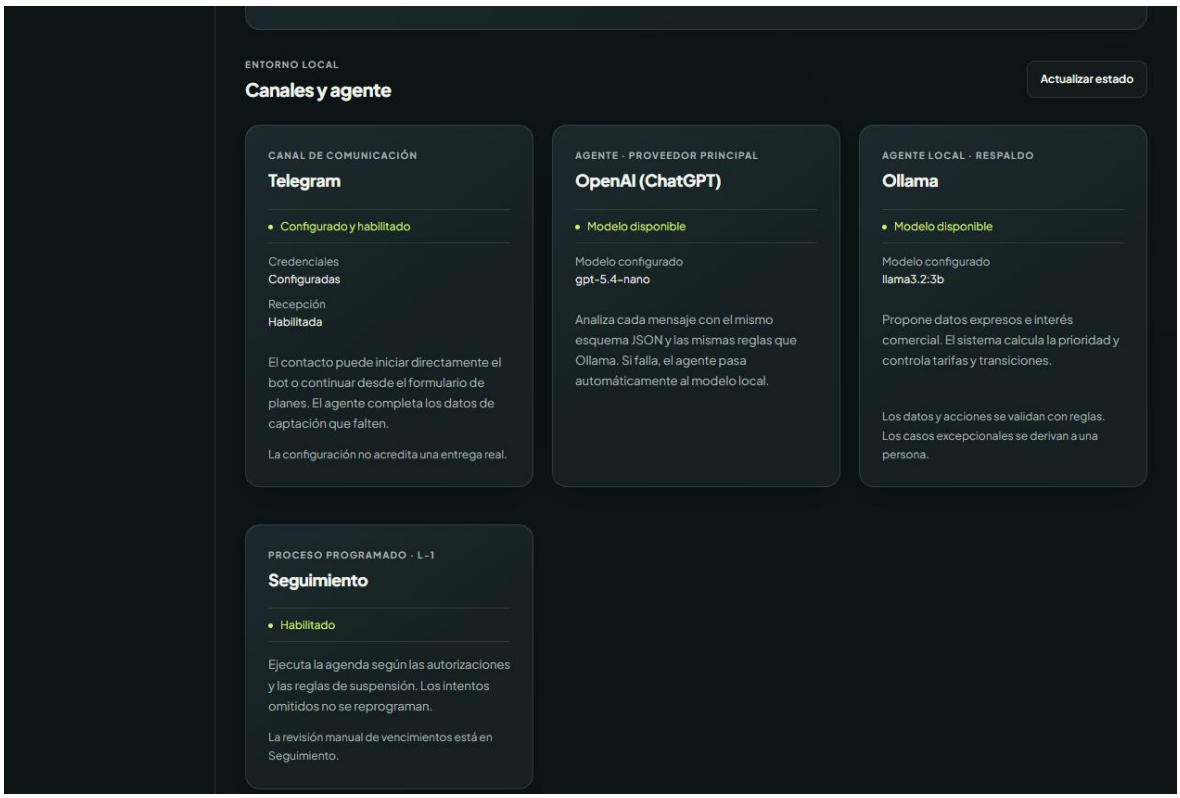
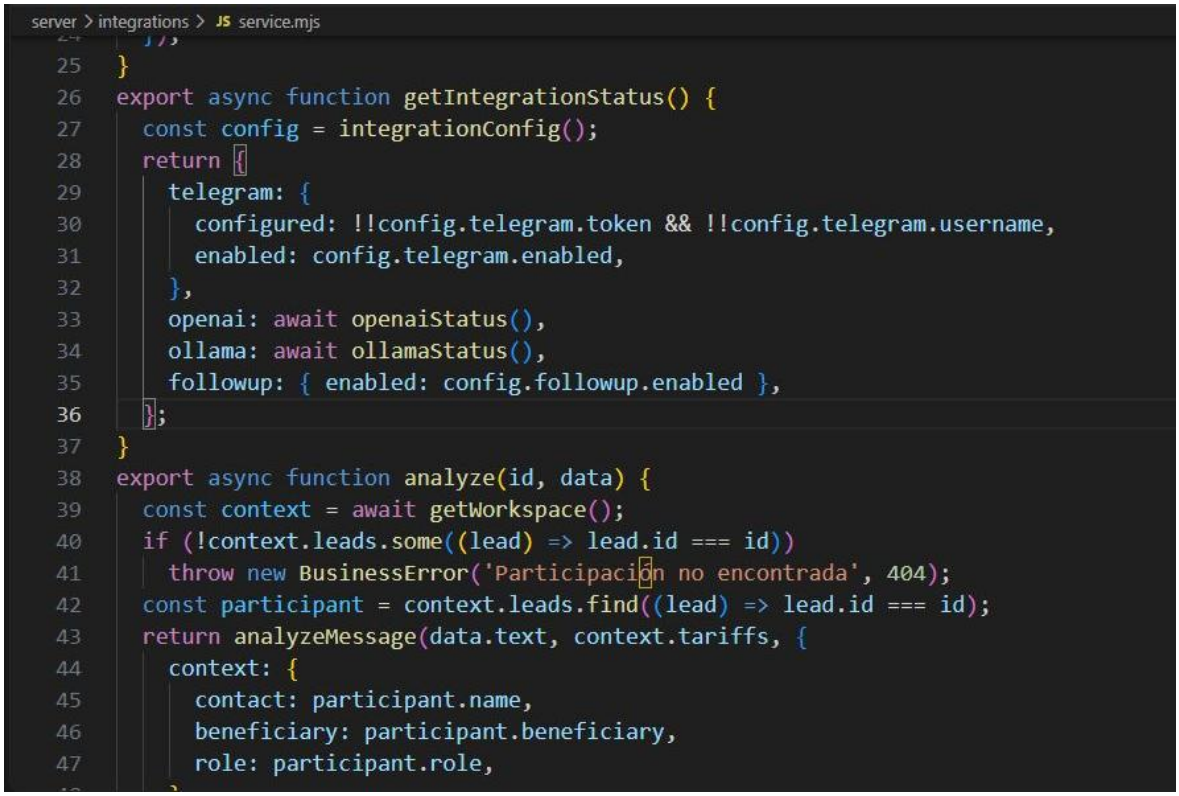


Figura 85

server/integrations/service.mjs líneas 25 a 47



Recorrido técnico Integrations → GET /api/integrations/status → getIntegrationStatus

Implementación La respuesta distingue configuración y habilitación de Telegram, disponibilidad de OpenAI y Ollama y ejecución del seguimiento. integrationConfig lee variables de entorno; el código documentado utiliza nombres de configuración, sin copiar secretos. analyzeMessage aplica el proveedor principal y el respaldo configurado. Un servicio disponible no prueba la entrega de un mensaje: la confirmación pertenece al conector, al despacho persistente y al registro de Mensaje.

11.2 Vincular Telegram con un contacto existente

Figura 86

Preparación de un enlace personal para Telegram



Figura 87

server/integrations/service.mjs líneas 9 a 31

```
server > integrations > JS service.mjs
 9  export const hashCode = (code) => createHash('sha256').update(code).digest('hex');
10  export async function createTelegramLink(id) {
11    const config = integrationConfig().telegram;
12    if (!config.username)
13      throw new BusinessError(
14        'Configura TELEGRAM_BOT_USERNAME después de crear el bot en BotFather.',
15      );
16    return transaction(async (db) => {
17      const participation = await getParticipation(db, id);
18      const code = randomBytes(24).toString('base64url');
19      await repository.saveLink(db, participation.idContacto, hashCode(code));
20      return {
21        url: `https://t.me/${config.username.replace(/^@/, '')}?start=${code}`,
22        expiresInMinutes: 10,
23      };
24    });
25  }
26  export async function getIntegrationStatus() {
27    const config = integrationConfig();
28    return {
29      telegram: {
30        configured: !!config.telegram.token && !!config.telegram.username,
31        enabled: config.telegram.enabled,
```

Recorrido técnico TelegramCard → POST /api/participations/:id/telegram-link → createTelegramLink

Implementación El servicio exige el nombre del bot y genera 24 bytes aleatorios codificados como base64url. Guarda el hash SHA 256 asociado al contacto y devuelve una URL personal de inicio con vigencia de diez minutos. El adaptador Telegram consume el vínculo y relaciona la cuenta del chat con el contacto. Iniciar el bot y autorizar el seguimiento son eventos diferentes; las decisiones se conservan por canal y contacto, aunque participe en varios casos.

11.3 Configurar las recomendaciones con datos

Figura 88

Vista previa de una recomendación con datos simulados

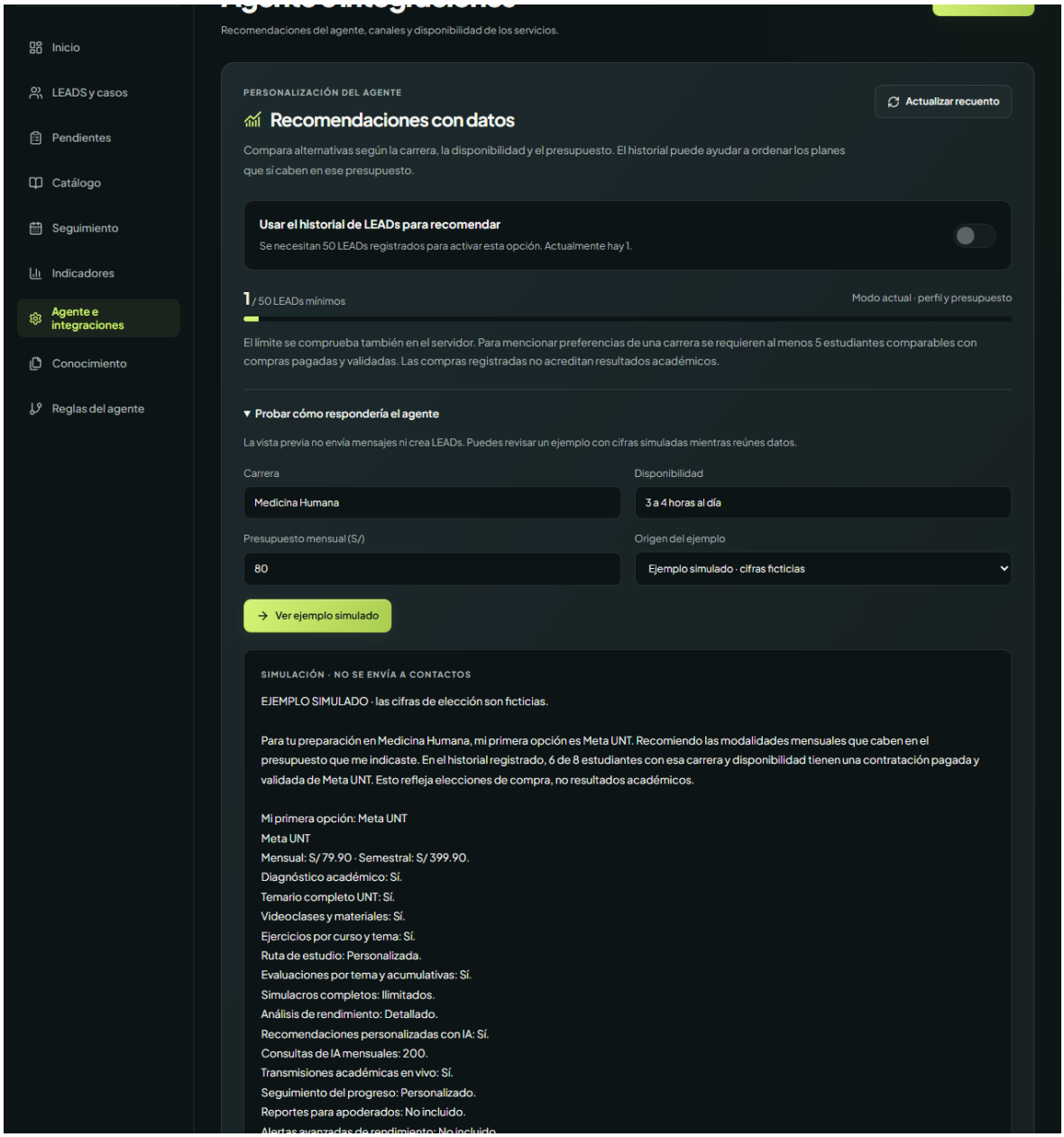
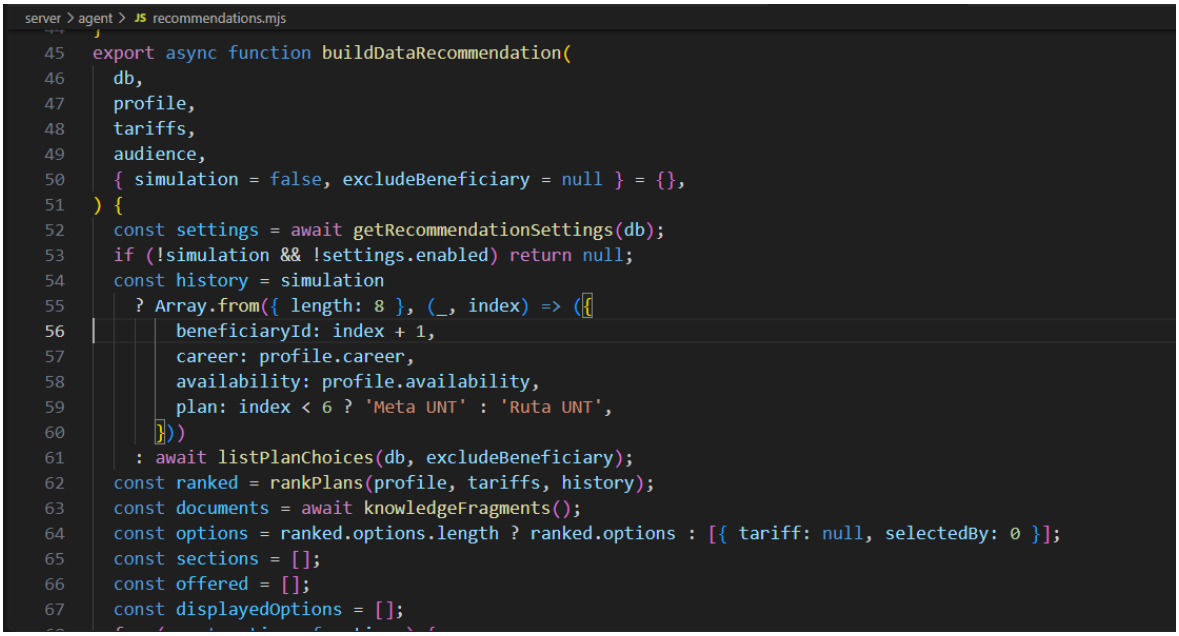


Figura 89

server/agent/recommendations.mjs líneas 45 a 67



Recorrido técnico Integrations → PUT /api/agent/recommendations/settings → buildDataRecommendation → rankPlans

Implementación getRecommendationSettings indica si las recomendaciones con datos están activas. buildDataRecommendation solo actúa cuando lo están, salvo en la simulación de ejemplo, que usa un historial ficticio de ocho elecciones. Con datos reales, listPlanChoices reúne los planes elegidos por beneficiarios de la misma carrera; rankPlans ordena las tarifas según presupuesto, carrera y disponibilidad, y el texto se completa con fragmentos de Conocimiento. previewRecommendation rechaza la vista previa real mientras la opción no esté activa y recuerda que se requieren al menos 50 LEADs.

12 Conocimiento del agente

12.1 Consultar y editar un documento

Figura 90

Edición de un documento de conocimiento

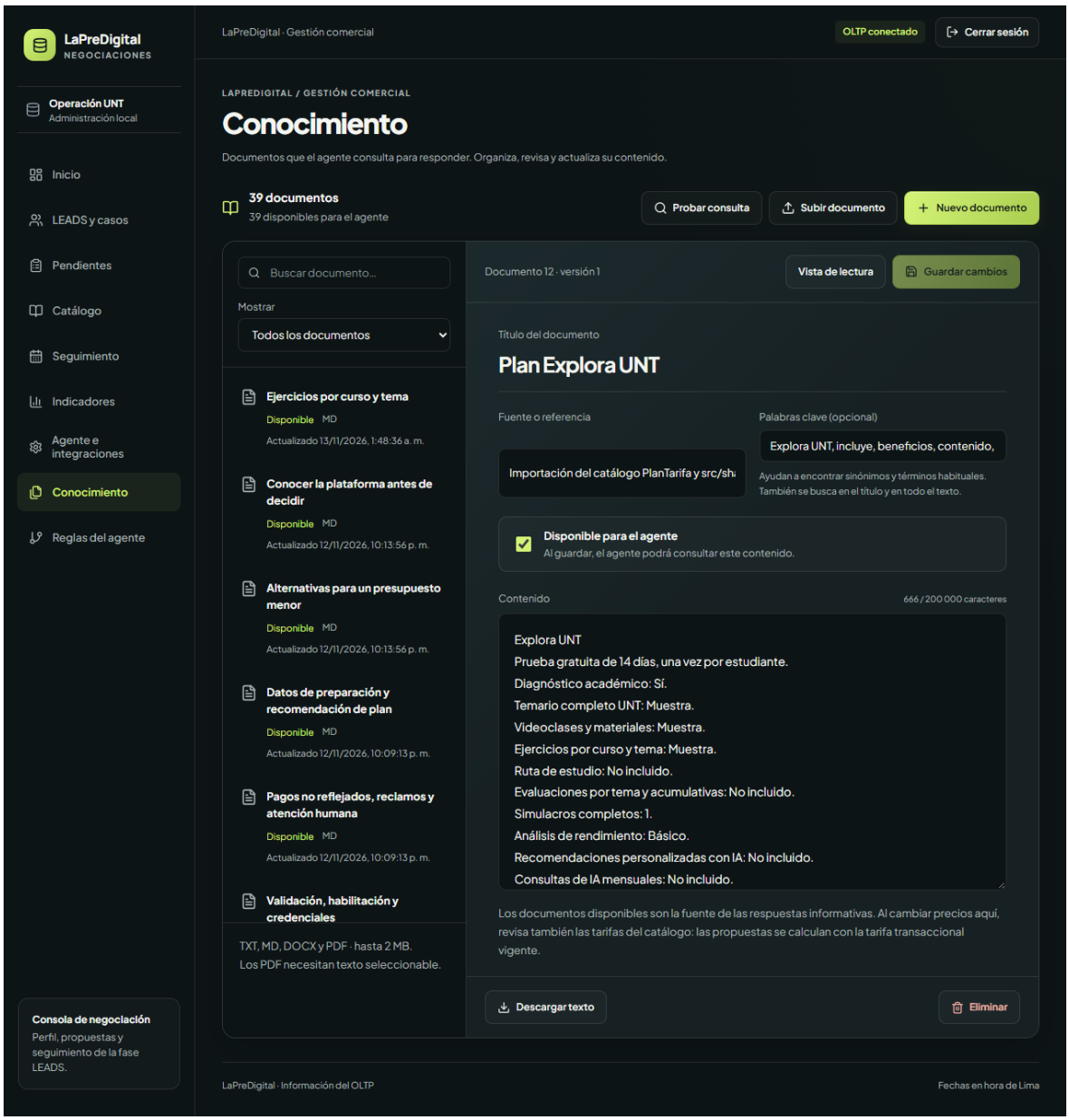
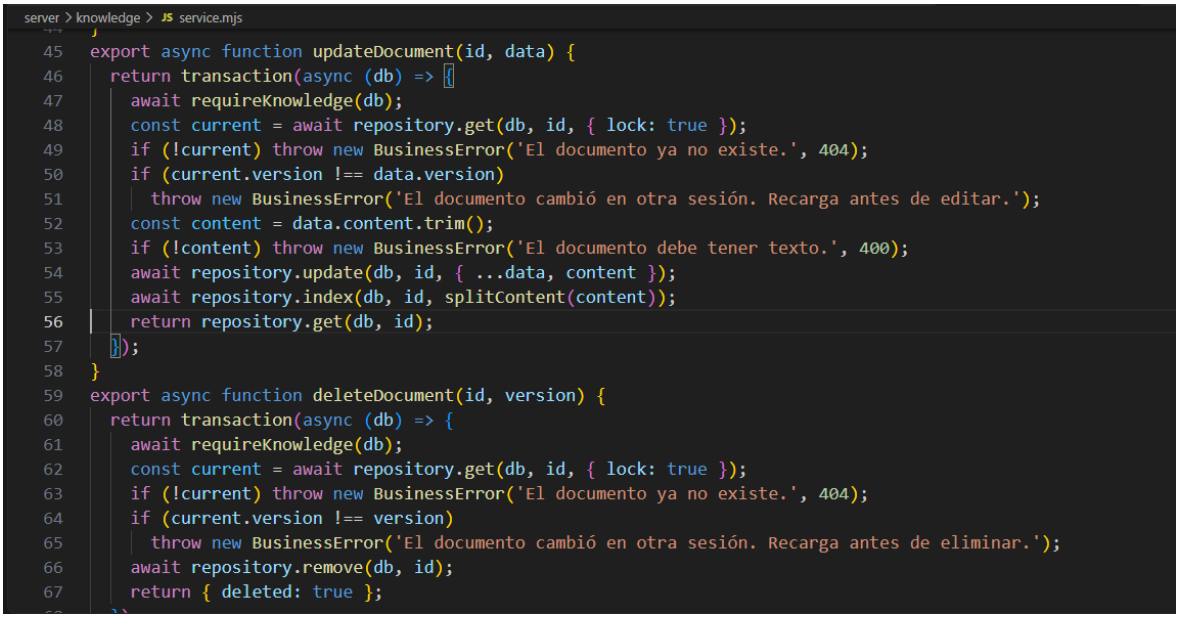


Figura 91

server/knowledge/service.mjs líneas 45 a 67



Recorrido técnico Knowledge → PATCH /api/knowledge/documents/:id → updateDocument → repository.update e index

Implementación updateDocument bloquea el documento, comprueba que siga existiendo y que su versión coincida con la que se editó; si otra sesión lo cambió, pide recargar. El texto no puede quedar vacío. Al guardar, el repositorio actualiza el documento y vuelve a indexar sus fragmentos con splitContent, que son los que el agente consulta. deleteDocument aplica la misma comprobación de versión antes de eliminar.

12.2 Crear o subir un documento

Figura 92

Biblioteca de documentos del agente

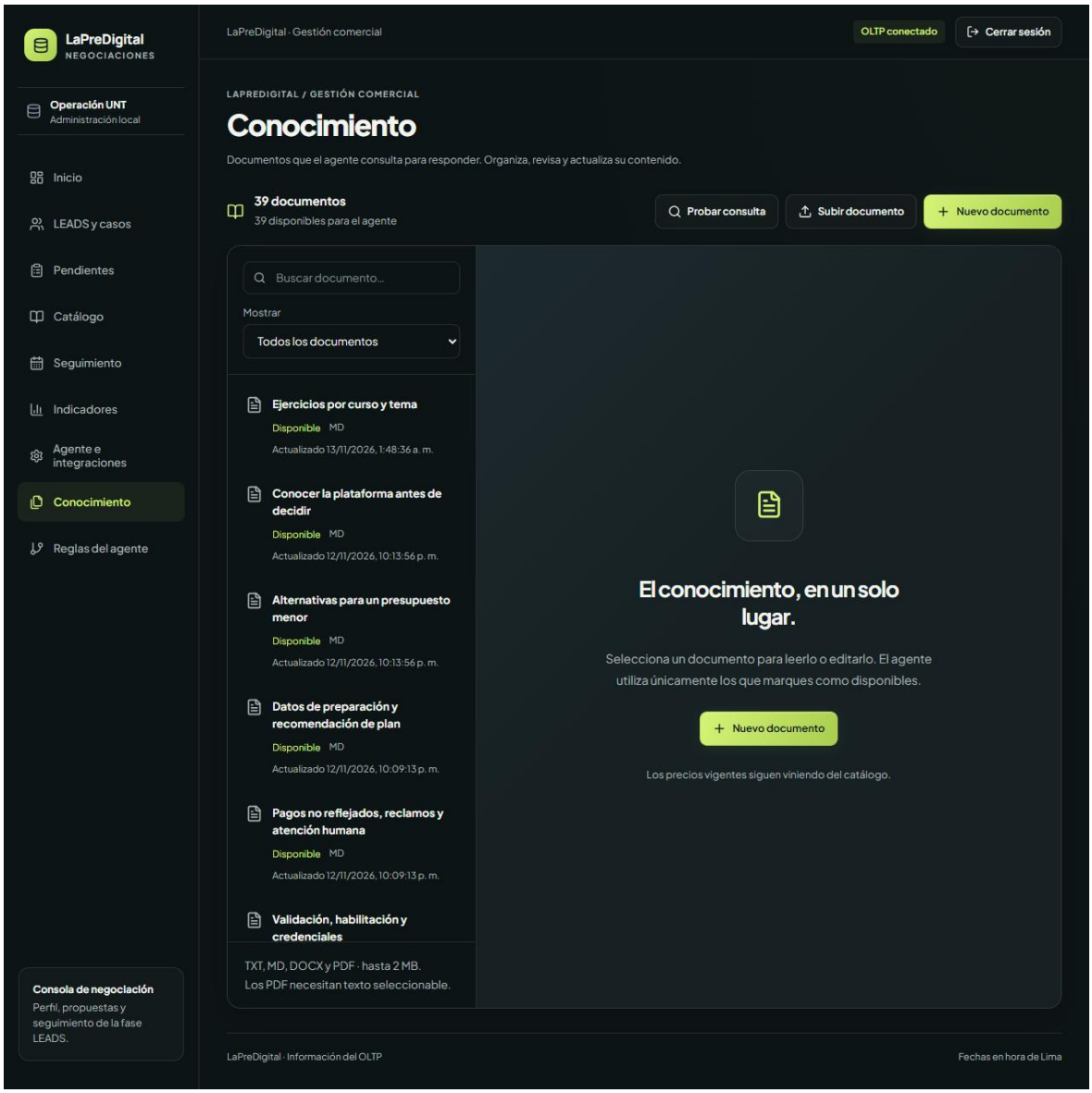


Figura 93

server/knowledge/extract.mjs líneas 5 a 27



Recorrido técnico Knowledge → POST /api/knowledge/documents o /import → extractDocument → createDocument

Implementación extractDocument limpia el nombre del archivo, admite solo TXT, MD, DOCX y PDF y valida la codificación base64, el tamaño máximo de 2 MB y la firma del contenido. La extracción se ejecuta en un proceso separado para que un PDF dañado no detenga la API. createDocument guarda el texto obtenido, el archivo original y sus fragmentos indexados. La ruta /api/knowledge acepta solicitudes de hasta 4 MB.

12.3 Probar una consulta

Figura 94

Fragmentos recuperados para una pregunta de prueba

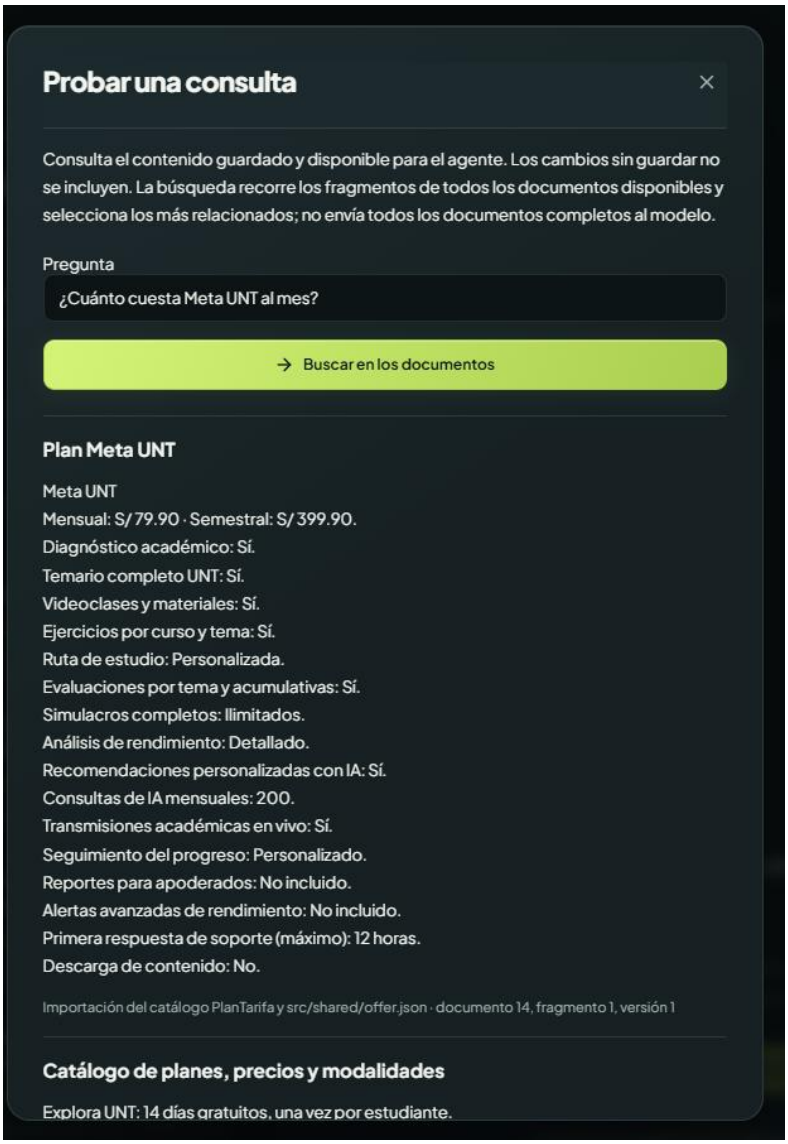
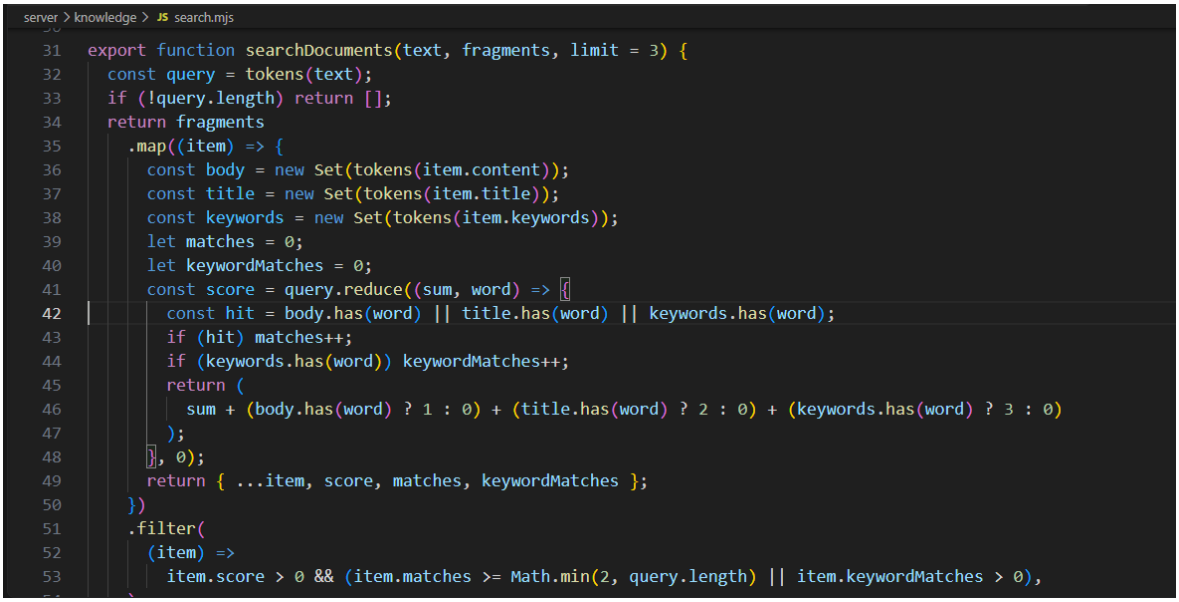


Figura 95

server/knowledge/search.mjs líneas 31 a 53



Recorrido técnico Knowledge → POST /api/knowledge/search → previewSearch → searchDocuments

Implementación searchDocuments divide la consulta en palabras normalizadas y puntúa cada fragmento: una coincidencia en el texto suma 1, en el título 2 y en las palabras clave 3. Solo conserva fragmentos con al menos dos coincidencias o con una palabra clave, y devuelve los tres mejores. La prueba de consulta de la consola usa el mismo criterio que el agente al responder; si no hay fragmentos suficientes, el agente indica que no tiene esa información.

13 Reglas del agente

13.1 Configurar las preguntas y las acciones automáticas

Figura 96

Configuración de las reglas del agente

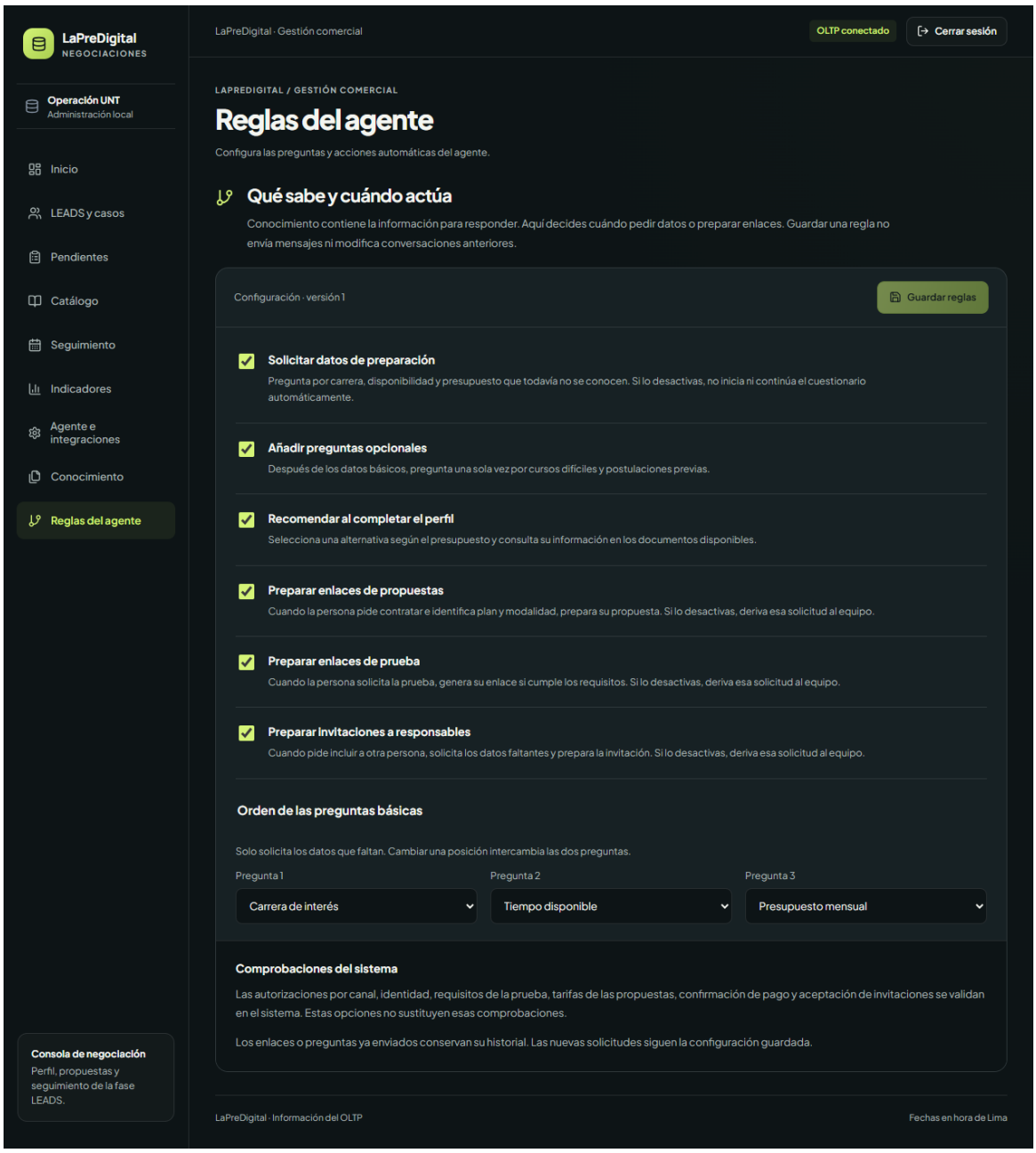
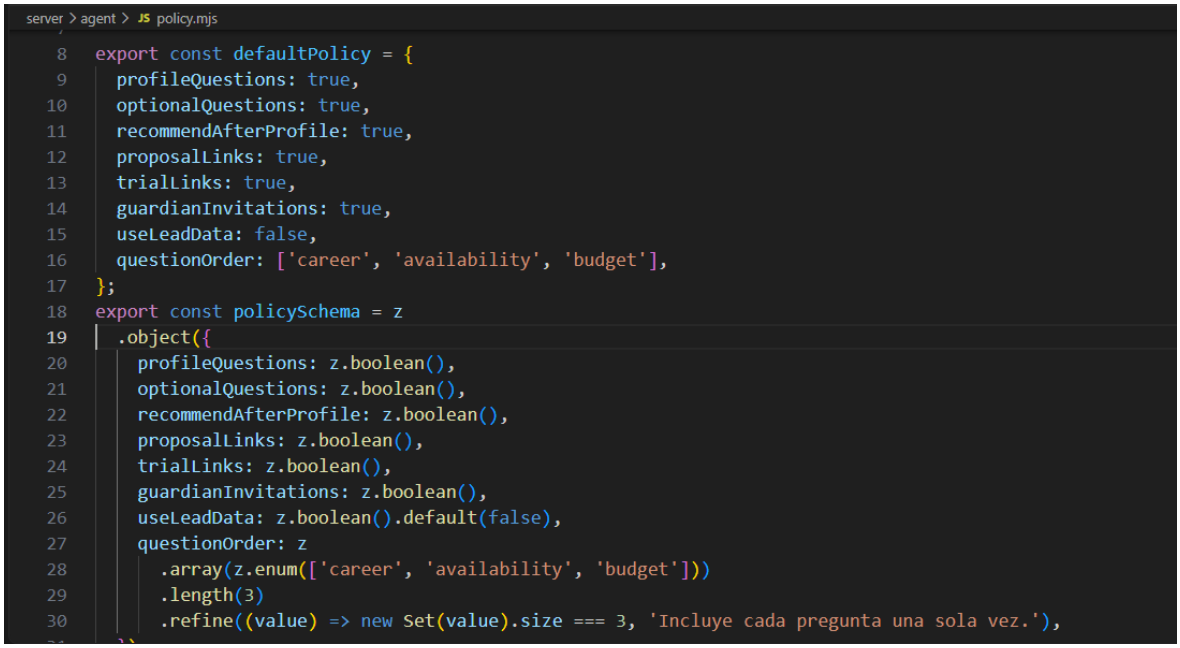


Figura 97

server/agent/policy.mjs líneas 8 a 30



Recorrido técnico AgentRules → GET y PUT /api/knowledge/policy → getPolicy y savePolicy → policySchema

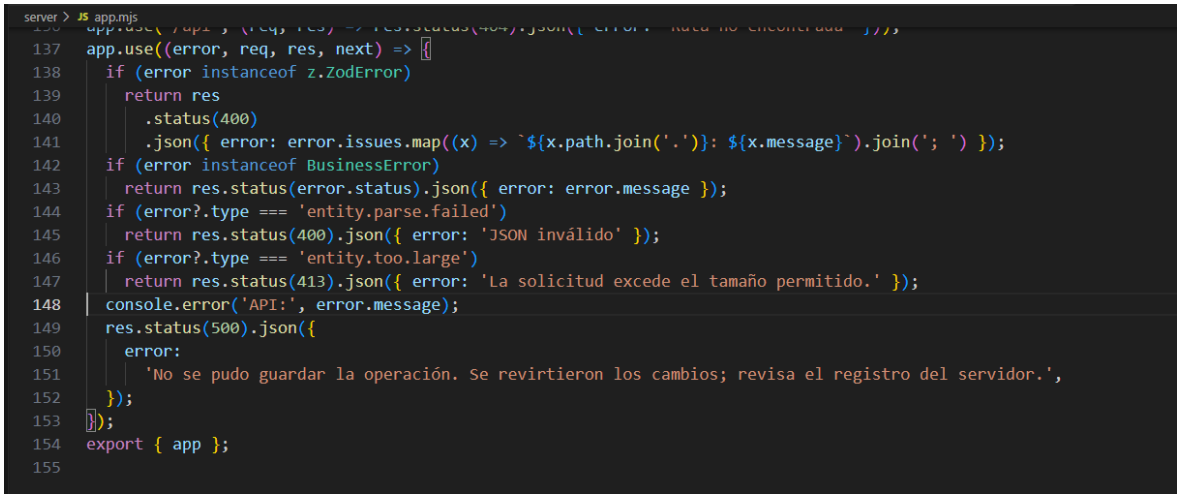
Implementación defaultPolicy define las reglas iniciales: preguntas de perfil y opcionales, recomendación al completar el perfil, envío de enlaces de propuesta y de prueba, invitaciones al responsable económico, uso de datos de LEADs y orden de las tres preguntas. policySchema valida cada opción y exige que carrera, disponibilidad y presupuesto aparezcan una sola vez. getPolicy lee la configuración vigente con su versión y savePolicy guarda los cambios; el agente consulta estas reglas antes de preguntar o enviar un enlace.

14 Referencia de operación

14.1 Resolver situaciones frecuentes

Figura 98

server/app.mjs líneas 137 a 154



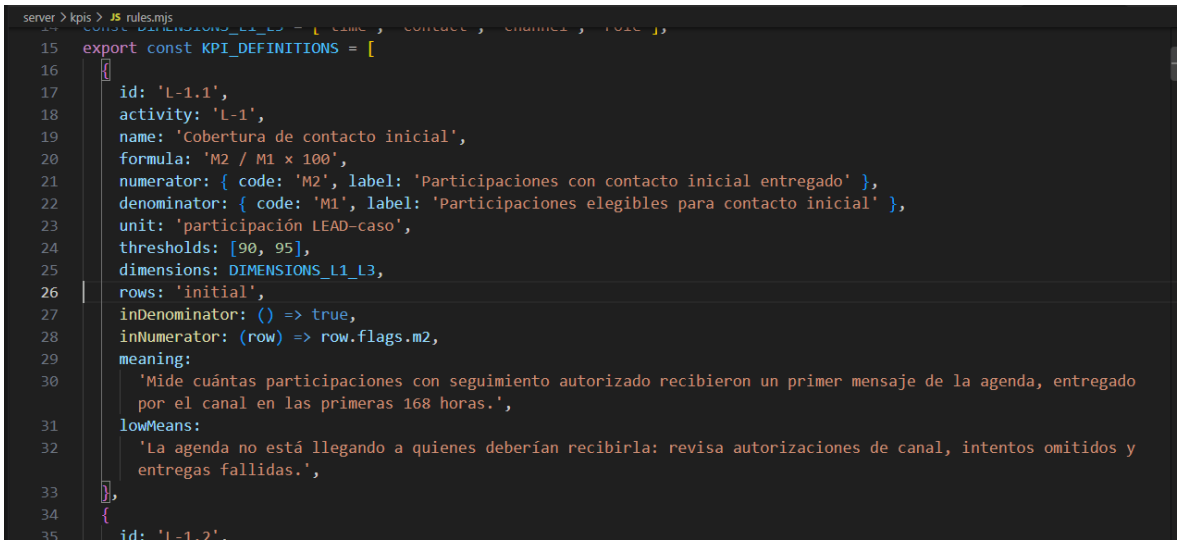
Recorrido técnico http/validation.mjs → BusinessError → middleware final de app.mjs

Implementación Las rutas validan las entradas con esquemas Zod y pasan las excepciones al middleware de Express. Los errores de validación y de negocio devuelven un estado y un mensaje comprensibles; los inesperados se registran y devuelven un error general. transaction intenta rollback antes de propagar el error. Las pruebas en server/tests cubren reglas comerciales, captación y Data Mart con datos controlados; no se ha ejecutado una batería de pruebas operativas para redactar este manual.

14.2 Interpretar los indicadores

Figura 99

server/kpis/rules.mjs líneas 15 a 35



Recorrido técnico KPI_DEFINITIONS → datamart-rows → computeKpi

Implementación Las seis definiciones conservan fórmula, unidad, umbrales, dimensiones y predicados de numerador y denominador. L 1.1 usa M2/M1; L 1.2 M3/M2; L 2.1 M5/M4; L 3.1 M7/M6; L 4.1 M9/M8; L 5.1 M11/M10, multiplicados por cien. Las filas dimensionales conservan las fechas necesarias para comprobar ventanas de 168 horas. L 5 parte de la primera Alta y excluye la beca completa dentro de su ventana; no reutiliza el indicador antiguo de evaluación.

Indicador	Fórmula	Unidad de análisis
L 1.1	M2 / M1 × 100	Participación elegible
L 1.2	M3 / M2 × 100	Participación contactada
L 2.1	M5 / M4 × 100	Contacto único por mes
L 3.1	M7 / M6 × 100	Caso que requiere vinculación
L 4.1	M9 / M8 × 100	Caso con propuesta entregada por mes
L 5.1	M11 / M10 × 100	Caso desde su primera Alta

El informe define D1 a D6 y la tabla 10 determina las dimensiones aplicables a M1 a M11. El código actual añade DimCaso como soporte de las relaciones y permite el nivel de intento de compra al abrir el pago. Estas ampliaciones se distinguen de las definiciones originales. La etapa comercial, la prioridad, el estado de participación y el resultado de pago son atributos de procesos diferentes y no se intercambian para calcular las tasas.

Verificación técnica y reproducción

Para reproducir los recorridos se utilizan las dependencias y variables descritas en el capítulo 1. La base elegida determina los registros que se verán; las pantallas de este manual conservan el conjunto de demostración del manual de usuario. Las comprobaciones siguientes describen la evidencia que debe revisar el mantenedor y no representan pruebas operativas ejecutadas durante esta documentación.

Comprobación	Punto de entrada y evidencia esperada
Preparación	npm install y npm run build preparan dependencias y compilan TypeScript y Vite. npm start inicia la API y sirve la compilación disponible.
Consola y persistencia	GET /api/bootstrap carga los datos iniciales del OLTP. Al abrir una participación, GET /api/participations/:id debe devolver el expediente correspondiente, separado de los indicadores.
Origen analítico	GET /api/kpis devuelve el origen dimensional y los resultados con los filtros elegidos. El resumen de Inicio procede de otra consulta. La ausencia de denominador debe mostrarse como No aplicable.
Actualización dimensional	npm run datamart:init crea y carga el modelo; npm run datamart:refresh repite la carga. El botón Actualizar datos utiliza POST /api/kpis/refresh. Son operaciones de escritura sobre la base analítica configurada.
Contratación y entrega	La evidencia distingue aceptación, pago confirmado, validación, habilitación y entrega de credenciales. Sus registros, identificadores y estados se comprueban en el expediente y en Pendientes.
Pruebas del proyecto	npm test utiliza node:test. server/tests reúne escenarios comerciales, de captación, integraciones y Data Mart; sus archivos y configuración deben revisarse antes de ejecutarlos en el entorno de prueba.